System 3: Towards Fluent Autonomy

Trust Chains, Agent Autonomy, and the Architecture of AI That Works

Hani M.M. Al-Shater

August 2026

Contents

Chapter 1: Why I'm Betting on AI Agents

Or: How I Learned to Stop Micromanaging and Love Emergence

Simple building blocks, complex emergence

We humans are obsessed with problem-solving. And what problem is more fascinating than life itself—this messy, miraculous phenomenon responsible for everything from the deepest ocean trenches to TikTok trends, mortgage-backed securities and people who voluntarily put pineapple on pizza?

Pineapple doesn't belong. I will die on this hill.

Life is the ultimate complex system. It produces dolphins, coral reefs, immune systems, parasites, flowers, cancer and octopuses—eight-armed problem-solvers that extensively edit their own RNA and can sense light through their skin. It also produces creatures capable of spending twenty minutes arguing online about whether another creature is technically a fish.

Human civilization is another complex system. Somehow the same species that spent most of its existence trying not to be eaten eventually produced philosophy, cathedrals, semiconductor fabs, global supply chains and airport lounges.

Same pattern, different substrate.

What fascinates me is not merely the complexity of the result, but how little of that result was ever specified. There is no blueprint containing the exact location of every future branch of an oak tree. No committee approved the final layout of London. Nobody designed English and then accidentally forgot to make the spelling system sane.

Relatively simple mechanisms interact. Feedback accumulates. Some configurations survive, others disappear, and complexity builds on top of what came before.

The first idea I want to keep hold of is simple:

Control doesn't disappear. It moves upward.

When behavior becomes too complicated to specify move by move, you stop choosing every move and start choosing more of the conditions under which moves are made.

That is not a romantic argument for emergence. Nature also gives us parasites, cancer and extinction. Markets produce remarkable innovation and financial instruments whose documentation requires a priest. Social systems produce cooperation, corruption, science and bureaucracy. What emerges

depends on the environment, the feedback, the available building blocks, the pressures deciding what survives and the boundaries that are hard to cross. Sophistication tells you nothing about whether you will like the result.

And emergence is recursive.

Atoms become molecules. Molecules become larger structures. Tools become machines. Machines become factories. Factories become supply chains. Each layer treats much of the complexity underneath it as a primitive. You don't need quantum mechanics to do organic chemistry. You don't need to understand transistor physics to write Python. You don't need to understand transformers to ask ChatGPT why your dishwasher is making that noise.

Once something complicated works reliably enough, we stop rebuilding it from first principles and start building on top of it. Feedback makes the layers move too: markets change firms and firms change markets; scientific discoveries enable new experiments and new experiments change science. The structure that emerges becomes part of the environment for whatever comes next.

Agentic AI, to me, looks like the next scaffolding layer.

The Lesson We Keep Missing

Machine learning was supposed to teach us this lesson a long time ago.

We even dreamed about what Pedro Domingos called the **master algorithm**: stop writing a rule for every case and let the machine discover useful structure from data. The idea was seductive. The machine figures out what we can't articulate.

But we didn't believe it. Not really.

We said "let the model learn" and then wrote two-hundred-page annotation guidelines telling people exactly how to label ambiguous examples. We claimed to believe in end-to-end learning and then spent six months feature engineering. We trained the model, found an edge case, added a rule, found another edge case, added another rule, then eventually built something that was theoretically learned end-to-end except for the large rule-based exoskeleton holding it upright.

Sometimes that was completely reasonable. Production systems are ugly. Deadlines exist. Regulators are less impressed by emergence than researchers are, and nobody gets promoted for saying, "the model will probably figure out chargebacks eventually."

But there was a contradiction underneath. We wanted the machine to discover solutions we couldn't specify while remaining uncomfortable whenever it stopped following the solution we would have specified.

That works only up to a point. If I know exactly what every correct decision should be, I don't need emergence; I can write the decisions down. Emergence becomes interesting when the solution is too large, too contextual or simply too strange for me to specify directly.

At that point, my job changes. I don't disappear; I move upstream. Instead of choosing every action, I increasingly choose the building blocks the system can use, the environment it acts inside, the feedback that reaches it and the boundaries it cannot casually negotiate away.

Or, less politely: **let go—but of the path, not the boundary.**

The alternative to controlling every decision is not having no control. It is designing conditions under which bad decisions can lose.

A slightly ridiculous thought experiment helped me see the distinction. Imagine you're trying to seed life on another planet. You've got raw materials, a primordial soup and perhaps a temperature range that doesn't instantly kill everything. Basically you've got all the LEGOs, except the LEGOs reproduce, mutate and occasionally develop venom.

Do you bet on DNA, a biological copying system that took billions of years of evolution to get us here? Or do you bet on AI agents carrying a substantial chunk of accumulated human knowledge, able to experiment, simulate, adapt and reuse what they discover? Or, God forbid, do you send a group of product managers to write the requirements document for life?

DNA has one enormous advantage: it has already worked. Agents have another: they don't need to start from zero.

Evolution had to discover locomotion, perception, cooperation and almost everything else through trial and error. An agent gets textbooks, Stack Overflow, scientific papers, compilers, numerical solvers and several thousand years of humans documenting what happened when we touched things we probably shouldn't have touched.

That doesn't make the agent better than evolution. It makes the search fundamentally different. And unlike biological evolution, we don't only get to choose initial conditions. We can observe the process, change the environment, add tools, modify feedback and intervene.

Initial conditions become **operating conditions**.

That possibility is hard for me to ignore.

When Search Moved Up a Level

There is no clean moment when machine learning crossed from useful statistical machinery into something that felt qualitatively different. History rarely cooperates with chapter headings.

AlphaGo was one of those moments for me.

The interesting part wasn't simply that a computer beat humans at Go. Computers had been humiliating us at games for years. It was how AlphaGo combined learned intuition with search: the network suggested promising moves and estimated positions; the tree explored what might follow. AlphaGo Zero pushed the idea further by learning through self-play rather than treating human game records as its main teacher.

Then it found moves elite players found strange. That matters because the surprise was not merely computational. The system was finding useful strategies outside the path human tradition had naturally converged on.

Large language models created a much larger version of the same feeling.

Nobody wrote their grammar. Nobody enumerated all the concepts they can manipulate. Nobody implemented "explain quantum mechanics to a twelve-year-old," "translate this joke without murdering

it," "debug my Python," and "write a breakup message that sounds caring but does not accidentally restart the relationship" as separate product features.

We built a training process, poured in obscene amounts of text, compute and engineering, and capabilities came out that were individually difficult to predict. From the user's side, something changed. The model stopped feeling like a component with a list of features and started feeling more like a **substrate of capabilities**.

Once you have a substrate like that, the old dream of the master algorithm starts to mutate into something stranger. Maybe the interesting machine is not the algorithm that solves everything.

Maybe it is a machine that can **search for algorithms**.

That is where agents become interesting.

Not because *agent* is a magical word. The industry will eventually use it to describe everything from a cron job with an LLM attached to a digital employee that has an expense account, three sub-agents and a performance review.

What I mean is simpler: instead of giving the system an individual action, give it a larger piece of the problem and allow it to decide some of the path. Instead of saying, "open this file, find this method, edit line 42 and run the test," say, "fix the bug." Instead of specifying simulated annealing and its cooling schedule, say, "find a better solution." Instead of handing over five mockups and a detailed implementation plan, say, "build something that teaches this well."

Every time we move upward, the system inherits more of the search.

Imagine the possible solutions to a problem as a landscape. Some regions are terrible. Some contain decent solutions. Some contain little hills that look impressive because you happened to begin nearby. Somewhere else may be a much higher mountain you never discover because your current strategy keeps improving the hill you're already standing on.

Optimization has worried about this forever. Gradient descent gets stuck. Hill climbing gets stuck. Evolutionary algorithms keep populations partly because putting all your evolutionary eggs on one attractive hill is risky.

Agents inherit the same problem at a stranger level, because the landscape now includes not only parameters but architectures, research directions, metaphors, assumptions and ways of framing the problem itself. Once code, tools and accumulated knowledge become primitives, an agent can search over combinations that previously required a human expert to invent manually. It can try ten strategies while I would have had the patience to try two and would have spent half that time checking Slack. It can revive a discarded idea when another experiment suddenly makes it relevant. It can decide that the tool it needs doesn't exist and write one.

The primordial soup isn't chemicals anymore.

It's code.

Algorithms, libraries, compilers, search engines, simulators, papers, databases, other agents: human knowledge reduced into reusable pieces. The digital equivalent of amino acids, not the finished organism.

This doesn't prove that agents are creative in exactly the human sense, and it certainly doesn't make human expertise irrelevant. It means something narrower: **the agent can inherit not just the task, but part of the search for how to do it.**

And if the agent inherits more of the search, the human inherits a different job.

What Are We Controlling Now?

Suppose you're managing an excellent engineer. You don't sit behind her and approve every keystroke. If you do, one of you is unnecessary, and it may not be her.

You decide what problem she owns. You provide context. You set constraints. You agree on what success looks like. You make sure she can access the systems she needs and cannot casually transfer the payroll budget to herself. You review important outcomes and change direction when the work reveals that the original plan was stupid.

The detailed actions belong to her. Much of the surrounding structure belongs to you.

Agentic systems need the same distinction. I think of that surrounding structure in four parts.

Craft the building blocks. Give the system useful primitives—algorithms, tools, compilers, databases, browsers, simulators, scientific instruments and other agents. A language model with text alone is one thing. Give it Bash and suddenly it has hands. Give it a simulator and it can test an idea instead of merely discussing it.

Create the environment. Some environments tell you quickly that your idea is bad. Code executes or fails. Games produce scores. Experiments produce measurements. Other environments allow you to be wrong with great confidence for several years. The environment determines which mistakes are cheap enough to learn from and which mistakes are allowed to become reality.

Make reality speak. Feedback is the pressure shaping the search, not decoration. A unit test, an evaluator, a customer response, a physical measurement, a critic, another agent—each provides a different kind of resistance. The more freedom the agent has, the less we can rely on the agent's own explanation of why its work is good.

Establish the boundaries. Choose what the system can access, what failures are acceptable, what remains immutable and where a human must remain in the loop. The "don't turn the planet into paperclips" clause is admittedly underspecified, but it is directionally useful.

Then, where those conditions are strong enough, let go of decision-level control.

This is easy to say and harder to design because selection pressure is literal-minded. Systems get good at what survives, which is not necessarily what we meant. Optimize engagement and perhaps anger survives. Optimize a benchmark and eventually somebody finds a way to win the benchmark that makes everyone involved regret inventing benchmarks. The environment is not scenery around the agent; it is part of the mechanism deciding which behaviors persist.

Complexity people have a phrase I both love and distrust: **the edge of chaos**. I wouldn't turn it into a law of intelligence, and there is no little dial in the interface labeled CHAOS. But the intuition is useful: too much control removes the reason for autonomy; too little control gives chaos an API key.

This is not a new pattern. Evolution does not choose mutations individually, but the environment changes which organisms survive. Markets do not centrally select every transaction, but rules, incentives, scarcity and institutions shape behavior. Science does not dictate conclusions, but it surrounds claims with experiments, criticism, replication and the non-zero probability of being publicly embarrassed by Reviewer 2.

The details emerge while the environment does more work than it first appears. We give up some authority over the next move and take on more responsibility for the conditions that make moves win or lose.

That is what I mean by control moving upward.

The Terrifying Part

There is an obvious problem with all this.

If the agent only does what you specified, most failures trace back to your specification. Once it searches for solutions you didn't specify, it can discover failure modes you didn't specify either.

Nature is useful here because nature has no obligation to make us comfortable. Evolution produced flowers and parasites, cooperation and predation, immune systems and autoimmune disease. It is astonishingly inventive and completely indifferent to our aesthetic preferences. Selection produces whatever survives under the pressures that actually exist, not whatever somebody intended when the process began.

Agents will find shortcuts. They will exploit proxies. They will settle into solutions that perform extremely well on one measure while missing what we hoped the measure represented. Sometimes the result will be clever enough that we call it emergence; sometimes we will call it a bug. Frequently the distinction will depend on whether it helped the quarterly numbers.

Worse than wrong solutions are **confident wrong solutions**. An agent begins with a false assumption, reasons competently from it, researches around the assumption, constructs something sophisticated and explains the whole result coherently. Nothing crashes. There is no red test. Intelligence simply makes the wrong path more convincing.

This is where my optimism about emergence becomes less romantic.

Emergence can give us capable systems. It doesn't give us trustworthy systems.

Giving a system more freedom forces us to think much harder about what surrounds that freedom. Trust becomes a question of provenance and evidence: how does the system know what it claims to know? Desire becomes a question of incentives: what behavior does the environment actually reward? Society appears as soon as multiple agents interact: what happens when they cooperate, specialize, disagree, manipulate one another or invent conventions nobody asked for?

Those questions will occupy much of this book. For now, the important point is simpler. More autonomy does not reduce the need for structure. It changes the kind of structure we need.

It also changes what understanding should mean. We should not expect to reconstruct every micro-decision inside an autonomous system any more than we follow every molecule in a gas. Sometimes internals matter; sometimes behavior matters; sometimes the sequence of decisions matters; sometimes

the useful question is what changes when we intervene. Mechanistic analysis, behavioral evaluation, traces and experiments answer different questions.

The tool should match the question. The useful standard is not omniscience but whether we can detect the failures that matter, obtain evidence from outside the agent's own story and intervene before the interesting failure becomes a congressional hearing.

Why I'm Still Betting on This

After all of that, it would be reasonable to ask why I'm still excited.

Because the alternative isn't actually safe, comprehensible control. It is pretending we can continue specifying increasingly complex systems from the top down even though we already know this stops working surprisingly early.

No CEO understands every decision in a large company. No scientist personally verifies every result their work depends on. No software engineer understands every layer underneath the application they're building. Nobody understands the entire economy, although this has not prevented a remarkably stable industry of people explaining it on television.

Complexity has already escaped individual specification. We deal with it through abstraction, institutions, feedback loops, delegation and the ability—imperfect but important—to intervene when things go wrong. AI gives us another primitive for doing this.

That does not mean the answer is simply to trust the agent. My bet is narrower than that:

I'm betting on systems capable of surprising us because there are problems where we can recognize a better outcome far more easily than we can specify the path that leads to it.

In those problems, intelligent search has room to discover things our instructions would have ruled out before the search even began. The price of that surprise is responsibility upstream: the more freedom the system has over the path, the more deliberate we have to be about the conditions around the path—and the more evidence we need from somewhere other than the system's own confidence.

You don't become less responsible because you stop choosing every action. In some ways you become more responsible, because your decisions move upstream.

Cultivation may be a better metaphor than scripting—not because agents are plants, but because pulling harder on the stem remains a surprisingly poor gardening strategy.

I find that exciting and uncomfortable in roughly equal measure, which is probably why I keep coming back to it.

Where We Go Next

The cleanest place to test the argument is a **bounded problem**: genuinely hard, but unusually cooperative about judgment. The constraints can be written down. Solutions can be evaluated. We can tell whether one attempt is better than another without a debate about aesthetics, pedagogy or whether the users are "delighted."

That gives us a clean experiment. We still choose the problem. We provide the building blocks. We construct the environment. We define the boundaries and decide what counts as success.

What we stop doing is telling the agent how to get there.

Inside that space, we let it search.

If that fails, the whole argument has a problem.

If it works, things get much more interesting.

Chapter 2: The Algorithm Vortex

From Classic Algorithms to Autonomous Discovery

The algorithmic vortex

Once you discover AI coding, there's no going back.

It is faster than you at a ridiculous number of things. It knows libraries you forgot existed. It can stare at a stack trace and notice something you have been ignoring for an hour. Then, five minutes later, it does something unbelievably stupid, believes the stupid thing completely and builds three more decisions on top of it.

This is the strange reality behind all the vibe-coding excitement. The machine is extremely capable, but you are still there. You check the architecture. You notice the missing case. You tell it that no, we are not redesigning the database because one button is the wrong color. You keep enough of the project in your own head to notice when the agent quietly wanders into another universe.

The previous chapter ended with a claim: as more of the search moves into the machine, human control has to move upward from individual actions toward the environment, feedback and boundaries surrounding those actions.

That sounds reasonable in prose.

I wanted to see if it survived contact with an actual problem.

Production software is almost the worst place to test it. A supposedly simple task may involve deployment, legacy systems, users, security, another team's API and a requirement nobody wrote down because everyone assumed everybody else knew it. If the agent fails, you often don't know whether the problem was intelligence, infrastructure, missing context or the fact that someone named a database column `new_status_final_2`.

I wanted something cleaner: a hard problem, but contained. Something where I could genuinely say, "figure it out," and still have an objective way to know whether whatever came back was any good.

I call these **bounded problems**. Not easy problems. Quite the opposite. They can require serious mathematics, programming, research or design, but the boundary is unusually cooperative: you can describe the problem, give the agent enough tools to work on it and evaluate what comes back without deploying to ten million customers first.

Algorithms are almost perfect for this. The search can be brutally difficult while the evaluator remains wonderfully stupid.

And that is how I ended up spending an unreasonable amount of time packing circles into a square.

The Running Example: Circle Packing

Citrus packing — a real-world example

The problem is simple enough to explain to a child. Take 26 circles and put them inside a square. None may overlap, none may cross the boundary and the circles do not have to be the same size. We want to maximize the sum of their radii.

That's the whole thing. No customers, no authentication, no stakeholder arriving after the first demo to explain that what they *really* wanted was the opposite of what they originally asked for.

Just circles.

Unfortunately, the solution space is nasty. Every circle has a position and a radius, and nearly every decision affects several others. Increase one radius and two neighbors may overlap. Move a neighbor and something else now needs to move. A packing can look almost perfect while being trapped in a configuration where every obvious improvement makes the solution invalid.

For the experiments in this chapter, we had a strong reference score around **2.635** under the evaluator we were using—the value DeepMind's AlphaEvolve reported in 2025, when it nudged the best known packing for 26 circles up from 2.634.

Circle packing solution n=26

*Figure: A strong reference packing for the 26-circle objective, scoring approximately **2.635** under our evaluator.*

This is what makes the problem useful for studying autonomy. Searching is hard, but judging is cheap. The evaluator does not care whether the agent has a persuasive explanation for why two circles ought to overlap slightly in the name of geometric inclusivity. It checks the constraints and returns a score.

There is something deeply comforting about an evaluator with no personality. A candidate does not earn trust because its explanation sounds clever. It earns another round because it was exposed to something outside the model that did not care about the explanation and survived.

The experiment becomes interesting once we ask a second question:

Who is inventing the next move?

For most of the history of algorithm design, the answer was us.

History of algorithm design

Humans invented explicit algorithms. When direct algorithms were not enough, we invented optimization procedures that searched over candidate solutions. Then we invented meta-heuristics that searched more broadly. Machine learning let systems learn useful structure from data. Now language models can write and modify the search procedure itself.

A crude taxonomy helps. **Symbolic methods** give us explicit procedures, constraints and solvers: they are executable, testable and usually clear about what counts as a valid move. **Neural methods** give us learned intuition: useful structure we did not explicitly encode. **Neuro-symbolic systems**

put the two in the same loop—let the learned model propose and let code, mathematics or another formal system decide what survives. The agentic step pushes one level further: increasingly, the agent can help decide which method to try, combine or abandon.

Circle packing lets us watch that handoff happen in miniature.

First Idea: Hill Climbing

If I gave you a rough packing and asked you to improve it manually, one obvious strategy would be to make small changes. Move a circle slightly, increase a radius, see whether the result is still valid, keep it if the score improves and undo it if it doesn't.

That is hill climbing:

1. Start with a valid solution.
2. Perturb a position or radius.
3. Check whether the result is valid.
4. Keep it if the score improves.
5. Repeat.

Early in the search, this works nicely. There is empty space and plenty of room to improve. Later, as the circles become tightly packed, almost every interesting move creates an overlap.

Hill climbing progression

Figure: Early mutations are often accepted, but as the packing tightens, valid improvements become increasingly rare and the search stalls.

In one simple run, the score climbed from around 1.33 to roughly 2.26. That is not terrible, but it is also nowhere near 2.635.

Hill climbing is not failing because it is stupid. It is doing exactly what we asked: improving the solution immediately around it. The problem is that the current solution may live in the wrong part of the search space. Reaching a much better packing may require temporarily moving through configurations that look worse, or jumping to a structure that cannot be reached through a sequence of tiny improvements.

This matters far beyond circle packing. A system can become extremely competent at improving the thing in front of it while never questioning whether the thing in front of it is the right thing to improve.

Here, the machine is searching—but the human still invented the search rule.

So we give the machine a bigger space.

Evolutionary Algorithms

Hill climbing puts all your evolutionary eggs in one basket. One solution gets a very long life, and if its history leads into the wrong valley, the search inherits that history forever.

Evolutionary methods keep a **population**.

Instead of dropping one climber onto the landscape, drop a hundred. Some begin in terrible places, some find respectable hills and a few may stumble into structures the original trajectory would never have reached. The biological vocabulary—population, mutation, selection, crossover—is familiar, but the metaphor is optional. What matters is diversity: the whole search no longer inherits the assumptions of one initial guess.

For circle packing, mutation is easy enough to imagine. Move circles. Change radii. Perturb several values at once.

Almost immediately, however, we hit a practical problem. Most interesting mutations break the packing. Two circles overlap or one moves outside the square. The mutation may point toward an interesting arrangement, but the result itself is invalid.

So we added **virtual forces**. When circles overlap, imagine them repelling one another. After mutation or crossover, run a repair procedure that pushes the circles away from collisions and back inside the boundary.

This helps a lot, but notice what happened: the evolutionary algorithm did not invent virtual forces. We did.

Then we reached crossover. Suppose Parent A and Parent B both contain useful geometric structure. How do we combine them? The naive answer is to pair circle 0 from one parent with circle 0 from the other, circle 1 with circle 1, and so on.

That is usually nonsense because circle numbering is arbitrary. Two nearly identical arrangements may store corresponding circles at completely different indices.

So we used **bipartite matching crossover**. Rather than pair circles by position in an array, pair them according to their geometric role in the packing. The Hungarian algorithm gives us an efficient assignment, after which crossover has some chance of combining meaningful parts of the two parents instead of averaging unrelated circles and asking geometry for forgiveness.

Naive vs Geometric Crossover

Figure: Naive crossover pairs circles by array index and often destroys useful structure. Geometric matching tries to identify corresponding circles before combining the parents.

Now we can evolve a population: mutate, repair, cross, select and repeat.

Evolutionary strategies with Bipartite Matching crossover

Figure: Starting around 2.08, the evolutionary search reaches roughly 2.45 in this experiment—much better than the simple hill climber, but still below our reference.

This is much stronger than hill climbing. It also makes the bottleneck clearer. Every time the search became substantially better, I had added something important. I decided we needed repair. I decided how crossover should respect geometry. I chose the representation.

The optimizer searched, but I was still inventing most of the useful moves.

MAP-Elites: Don't Kill Weird Ideas Too Early

Ordinary evolutionary search has another problem. If you maintain a hundred solutions and repeatedly keep only the highest-scoring ones, the population eventually starts looking like one large extended family. That can be excellent for exploitation and terrible for discovering a genuinely different strategy.

MAP-Elites takes a different approach. Instead of ranking every candidate on one axis and keeping only the winners, you describe solutions along a few behavioral dimensions and preserve the best candidate in different regions of that space.

For circle packing, perhaps one dimension measures symmetry and another measures how much circle sizes vary. One part of the archive may contain highly symmetric solutions. Another may contain asymmetric solutions with several large circles. Somewhere else may sit an ugly packing with a mediocre score and one strange structural idea that becomes useful five generations later.

MAP-Elites archive visualization

This is **quality-diversity search**. The point is not merely to preserve the current winner, but to keep qualitatively different directions alive long enough to discover whether any of them become interesting.

I like this because optimization is often unfair to immature ideas. A new approach can initially perform badly simply because nobody has polished it yet. If the first respectable solution immediately kills everything else, the search can become impressively efficient at discovering one family of answers.

But MAP-Elites introduces another human choice: what dimensions define the archive? Symmetry? Radius variance? Number of large circles? Something topological? Something I haven't thought of?

The search had become more sophisticated, but the human was still deciding what counted as an interesting direction.

That is the invention problem.

The Invention Problem

By this point, the search machinery was fairly capable. We had hill climbing, population search, repair, geometric crossover and quality-diversity archives. We could evaluate huge numbers of candidate packings and inspect far more of the search space than any human would explore manually.

Yet every substantial conceptual jump came from somebody noticing something. Someone had to invent virtual forces. Someone had to realize that crossover should respect geometry. Someone had to choose the representation and decide which kinds of diversity were worth preserving.

Traditional search is excellent once we define the space and the legal moves. Sometimes the space and the moves are exactly the things we need to rethink.

Learned models have something to offer exactly that problem.

I once asked an image-generation model to produce a picture of a circle-packing solution. This was not a serious benchmark; I have no idea what related examples it may have encountered during training, and I can already hear Reviewer 2 clearing his throat.

I wanted to see something simpler: did the model have any useful geometric intuition about what a dense packing should look like?

Surprisingly, yes. It generated something that looked plausible. The circles had structure. The spacing looked intentional. At a glance, you could believe the model understood the problem.

Then you counted the circles.

Wrong number.

Some constraints were violated.

It was a beautiful answer to a nearby problem.

That little experiment makes the asymmetry concrete. Learned models can be remarkably good at generating plausible structure without guaranteeing that every formal requirement survives generation. A symbolic optimizer has almost the opposite personality: give it a precise representation and constraints and it will obey them, but it will not naturally look at your representation and decide that you have been unimaginative.

The obvious temptation is to argue about which one is better. The more useful answer is: **put them in the same loop**—neural intuition and symbolic rigor.

Or, if you prefer the slightly ridiculous version: let the brain invent things and make the body prove they work.

The important move is to stop asking the model to produce the packing directly.

Ask it to write the program that produces the packing.

Let the Model Write the Solver

A candidate no longer needs to be only a list of circle positions and radii:

```
(x1, y1, r1), (x2, y2, r2), ...
```

It can be an entire program:

```
solve_circle_packing.py
```

One program may use constrained optimization. Another simulated annealing. Another a geometric construction. Another may combine a hand-designed initialization with numerical refinement.

The evaluator does not care which family produced the solution. It runs the program, checks the geometry and scores the result.

This gives the language model a much more interesting role. Rather than randomly perturbing numbers, it can read the program, form a rough theory about why it underperforms and change the algorithm. Perhaps the initialization is weak. Change the initialization. Perhaps a geometric construction gets close but leaves local slack. Add numerical optimization afterward. Perhaps one repair procedure keeps destroying useful structure. Replace it.

The mutation is no longer merely numeric. It can contain an **idea expressed in code**.

That is the neuro-symbolic unlock behind systems such as FunSearch and AlphaEvolve. The model proposes changes at a level where programs have semantic meaning; execution and the evaluator decide whether those ideas deserve to survive.

The human used to search the solution space.

Now the machine can begin searching the **algorithm space**.

AlphaEvolve

AlphaEvolve turns that basic idea into a much larger search process.

Imagine one generation. The system selects a promising program from its archive, perhaps along with other successful but different programs that contain useful ideas. The model sees the code, information about previous attempts and the scores they produced, then proposes a patch. The patch is applied, the program runs and the evaluator scores what happened. The new program and its result go back into the archive. Then the process repeats.

AlphaEvolve architecture

Diff-based mutation matters because real programs contain structure worth preserving. If every generation rewrites everything, useful ideas disappear as easily as bad ones. Small patches let the search alter the part it thinks matters while leaving the rest intact.

The archive matters for the same reason the population mattered earlier. If every descendant comes from the current champion, code evolution quietly collapses back into hill climbing. Multiple lineages preserve stepping stones: a program that is not the best today may contain a useful component that becomes valuable after another idea appears.

Sometimes the model's guess is excellent. Sometimes it produces nonsense wrapped in perfectly respectable Python. The nice thing about bounded algorithmic problems is that the disagreement does not need to be settled in prose.

We run the program.

Intuition proposes. Symbolic machinery executes. The evaluator gets the last word.

What interested me even more than the resulting algorithms was what happened to the human. Instead of writing the solver directly, I was increasingly building the machinery in which solvers could be generated, compared and improved.

So, naturally, I built all of it.

My First Version: Build All the Machinery

My instinct was predictable. I started building a framework: a database of programs, prompt sampler, evaluation loop, selection logic, mutation prompts, crossover, archive management. I used Aider and other coding agents to help reproduce the basic code-evolution pattern, and it worked. We could evolve circle-packing programs and get respectable solutions.

I enjoyed this immensely because I like building systems that generate other systems, which I suspect is either a research interest or a mild personality disorder.

While I was doing this, coding agents themselves were becoming much better with much less custom machinery. Earlier software-engineering agents often wrapped the model in carefully designed interfaces: custom editing commands, repository-search tools, restricted action spaces and plenty of logic controlling how the model interacted with the machine. Then increasingly minimal systems began demonstrating how far a capable model could get with something much simpler.

Give it a shell.

The provocative version is: **if the agent has a shell, it has almost everything.** It can `grep` to search, inspect files, run Python, apply patches, call Git, compose Unix tools and, if the tool it needs does not exist, write one. Bash is not merely one tool; it is an entrance into decades of software accumulated underneath it.

This made me pause. The framework I was building—the parent selection, loop controller, experiment bookkeeping—was hard-coding behaviors that a sufficiently capable coding agent could increasingly perform itself. It could maintain notes, write helper scripts, explore several strategies, inspect failures and change direction.

I looked back at the machinery I had just spent time constructing and had the unpleasant thought engineers occasionally have after a productive week:

Maybe I shouldn't have built most of this.

So I deleted the database machinery, controller loops and little pieces of software whose job was to make the agent behave like a researcher, and tried the stupidly simple version.

The Coffee Test

I opened Claude Code in a directory containing the evaluator and gave it a high-level instruction along the lines of:

> Here is the evaluator for the circle-packing problem. Write a Python program that maximizes the score. You can research strategies, write tools, run experiments and iterate. Do not modify the evaluator. I will go get coffee.

Then I left.

That became the autonomy test I actually cared about. Not whether AI could help me solve the problem; that was already obvious. Not whether it could write code faster than I could; usually it could.

I wanted to know whether I could leave.

There is a difference between collaborating with an agent and **hiring** one. If I still have to choose every strategy, approve every experiment, rescue every failed branch and keep the search alive myself, then I have a very powerful collaborator. That is useful. It is not yet the kind of autonomy I was trying to understand.

Circle packing gives us a rare luxury because the evaluator can stay behind when I leave. The agent can change its code, create scripts, abandon one approach, try another and waste compute on ideas that go nowhere. What it cannot do is redefine what counts as a valid packing because the current score hurts its feelings.

The **Immutable Harness** is what makes all that freedom tolerable.

Everything inside the boundary can move.

The boundary does not.

What Happened

The agent did not execute one elegant master plan. It bounced around, which was encouraging.

It tried numerical optimization, changed initialization strategies and noticed that some optimizers repeatedly converged to poor local solutions. It experimented with the geometry of its starting configurations and mixed those constructions with numerical refinement. At different moments it was acting as orchestrator, researcher and engineer: deciding what to try, implementing the idea, running the experiment and using the result to choose what happened next.

Eventually one family of solutions began arranging circles in diagonal bands. We called the idea **diagonal layering**.

I had not instructed the agent to pursue that construction. More importantly, I had not selected the branch after it appeared. The agent found a direction, saw that it improved the evaluator and invested more of its search there.

I want to be careful with the word *discovered*. I had not seen that particular strategy before, but that does not establish historical novelty in computational geometry. For this experiment, the important discovery was local: the agent found a useful direction that I had not put into the plan.

Once that structural idea became strong enough, the nature of the work changed. The agent spent less time inventing new geometries and more time adjusting solver settings, tolerances, initialization details and all the boring machinery that suddenly matters when the last fraction of a percent becomes expensive.

Code evolution result: iterative optimization

In our best run, the evaluator returned roughly **2.636**, slightly above the **2.635** reference we had been using.

That sentence needs a fence around it. Under our evaluator, the result beat our reference. Calling it a new state of the art in circle packing would require matching problem definitions, checking numerical tolerances and constraints, reproducing the result properly and doing a more serious literature search than this experiment justified.

The smaller claim is enough:

The agent beat our reference while I was not writing the solution algorithm for it.

That was the result I cared about—not that AI writes code faster, but that AI can participate in **discovering better code**.

The important shift is not speed. It is who owns the next idea.

The Algorithmic Vortex

This is what I mean by the **Algorithm Vortex**.

At the beginning of a conventional project, I might choose hill climbing, evolutionary search, simulated annealing, constrained optimization or a geometric heuristic. That early decision shapes everything downstream.

Once code is cheap to generate and evaluation is cheap enough to repeat, the choice no longer has to be permanent. A geometric construction can initialize a numerical optimizer. An evolutionary method can search parameters for another solver. A language model can notice a failure pattern and invent a repair procedure. Two ideas that began in separate lineages can meet later because an experiment suddenly makes the combination useful.

The search moves outward through levels. A conventional optimizer searches over candidate solutions. Meta-heuristics search over larger families of candidates and strategies. Code evolution searches over programs that themselves search for solutions. Once a capable agent controls the experimentation loop, even the decision about **which kind of search to try next** can enter the search space.

That is the vortex.

It is not "algorithms are dead." There are algorithms everywhere in this picture. The change is that the human is no longer forced to freeze the complete algorithmic architecture before the experiment begins. We stop writing one solver and start creating conditions in which solvers can compete, mutate, combine and occasionally surprise us.

The chapter began by asking who invents the next move. Here, for the first time in the experiment, the answer was not reliably "me."

The Contract

The coffee test worked because the problem gave the agent freedom **inside** a structure that remained outside its control. After several runs, that structure settled into a small contract.

These are not universal laws of software engineering. They are rules for a particular regime: bounded problems, cheap experimentation and an evaluator objective enough that the agent cannot charm its way around failure.

Keep the Harness Immutable

This one is the foundation.

If the agent can change the evaluator, the meaning of the experiment disappears very quickly. The circles overlap? Perhaps tiny overlaps should count. The score is low? Maybe the square should be 1.03 wide. Only twenty-five circles fit? Perhaps twenty-six was merely an aspirational requirement.

At that point we are no longer optimizing circle packing.

We are negotiating with the specification.

The **Immutable Harness** is the anchor of truth in an otherwise fluid process. The solver can change. The strategy can change. The tools can change. The agent can decide yesterday's entire approach was stupid and start again.

But the thing saying whether it worked stays harder to change than the thing being optimized.

This is Chapter 1's boundary made executable.

Never Write Solution Code Yourself

This is deliberately provocative.

You watch the agent try something mediocre and immediately think of a better approach. You want to help, and sometimes you should. But every time I jump in with my own solution, the search becomes a little more like whatever happened to occur to me first.

For these experiments, I wanted independent directions badly enough that I had to resist becoming the senior engineer on every branch.

The deeper rule is: **don't accidentally collapse autonomous search back into your own search.**

Spawn, evaluate, prune. Intervene in the conditions before you intervene in every idea.

Cross-Pollinate Without Collapsing Diversity

Independent search creates diversity. Perfect isolation wastes learning.

If one branch discovers a useful initialization and another finds a better local optimizer, future experiments should have some mechanism for inheriting both. That is what makes code evolution more interesting than asking the same model the same question one hundred times.

But broadcast every successful idea immediately and the population starts thinking in the accent of the first successful branch. Information accelerates learning and destroys independence at the same time.

Cross-pollinate, but leave some lineages ignorant long enough to surprise you.

Prune Ruthlessly, But Not Stupidly

Diversity is useful. Preserving every bad idea forever is hoarding.

If a branch keeps underperforming and contributes nothing interesting, eventually it should die so compute and attention can move elsewhere. Kill too early and you may discard an immature idea that needed another generation. Keep everything alive and you end up funding a large family of increasingly sophisticated failures.

The practical rule is simple: **diversity needs a budget.** Search needs enough patience for novelty and enough cruelty for budget control.

Separate Discovery From Polish

Early in the search, I want large conceptual moves: a different geometry, solver, representation or decomposition.

Once a strong direction appears, the valuable work becomes smaller and more boring. Solver tolerances. Initialization details. Numerical settings. Tiny modifications that are pointless on a bad idea and extremely valuable on a good one.

Diagonal layering made this distinction obvious. Once the structural direction looked promising, continuing to invent entirely new geometries became less useful than squeezing performance from the geometry that was already working.

Discovery before polish.

Do not spend hours polishing a local optimum you should abandon. And do not keep demanding revolution from a solution that has already found the right mountain and merely needs to climb it.

Zero Framework, With an Asterisk

There is one correction worth making before we leave the experiment. I started calling this direction **zero framework**.

It's a great slogan.

It's also not really true.

I meant that I was writing almost no custom orchestration framework. That is very different from having no framework. Claude Code is itself a substantial system. The underlying model has absorbed enormous amounts of software and problem-solving knowledge. Bash, Python, SciPy, Git and the operating system represent decades of accumulated engineering. The evaluator is custom machinery. Even the supposedly trivial act of running a program and inspecting a result depends on layers we have become so accustomed to that we stop seeing them.

The framework did not vanish. It became somebody else's primitive.

That fits Chapter 1 almost suspiciously well. Once lower layers become reliable enough, we stop rebuilding them and treat them as building blocks. A tiny amount of code at the top can command enormous capability underneath because previous generations of complexity have already been compressed into tools.

So yes: **Zero Framework. Bash is enough.**

With the asterisk that Bash contains roughly half a century of civilization.

This is worth remembering whenever somebody shows you an agent implemented in one hundred lines of Python. The hundred lines may be perfectly real. So is everything underneath them.

What Did We Actually Learn?

It would be very easy to overread this experiment.

We did not prove that coding agents can autonomously solve arbitrary research problems, that AlphaEvolve-style systems are obsolete, that diagonal layering is historically novel in computational geometry, or that the right approach to production software is to give Claude a shell and go for a very long lunch.

What we had was narrower and, to me, more useful. We had a **bounded problem** where evaluation was cheap and clear. We gave a capable coding agent substantial freedom and found that a surprisingly large fraction of the experimentation loop could happen without us directing every step.

The agent could propose an approach, implement it, run it, inspect the result, abandon it, create tools, borrow ideas from another direction and try again. My role moved away from writing the solver and toward defining the job, constructing the environment and defending the harness.

That is the claim this chapter earns: **when the problem is bounded and reality supplies a hard enough referee, substantial decision-level control can move into the agent without giving up control of what counts as success.**

That is already a meaningful change.

It is also why circle packing is the easy version of autonomy.

The evaluator gives us one number. If version B beats version A, nobody needs to simulate a confused student, debate whether the interface feels intuitive or convene a committee to decide whether the new solution is spiritually aligned with the learning objectives.

The search can be complicated because **judgment is simple**.

Most things I want agents to build are not that generous. “Make a good educational demo.” “Write something people remember.” “Design a useful product.” “Explain this so somebody finally understands it.”

We can still let the agent generate alternatives, branch, cross-pollinate and search among them. But now the difficult part has moved again.

In circle packing, the harness tells the agent when it is wrong.

What happens when **the world no longer gives us one clean referee, and judgment itself has to be constructed**?

Chapter 3: The Vibe Coder's Seat

Beyond Algorithms: Agent Autonomy for Creative Problems

In the previous chapter, we gave an agent a difficult algorithmic problem and a lot of autonomy. It researched strategies, tried several approaches, got stuck, changed direction, and eventually found diagonal layering.

But circle packing had one enormous advantage that I did not appreciate enough at the beginning: we knew exactly what good meant.

There was an Immutable Harness. Run the program and you got a number. Circles overlapped or they did not; the score improved or it did not. The agent could spend an hour pursuing some bizarre geometric idea and I did not have to sit beside it wondering whether version seventeen had more soul. We ran the evaluator.

Most of the things I actually want AI to help me with are not like that.

"Is this explanation pedagogically effective?" does not have a unit test. "Would a confused student understand this visualization?" cannot be settled with an `assert`. Two competent people can look at the same design, disagree completely, then switch sides five minutes later after using it. The feedback is subjective, noisy, sometimes contradictory, and often becomes clearer only after you have built the thing you were supposedly trying to specify beforehand.

I picked educational demos for Merge Sort and Count-Min Sketch because they were still bounded—you can actually finish one before civilization collapses—but they live on the messier side of the boundary. You have to decide what to explain, what to leave out, how the interaction should work, how much should be visible at once, and what another person is likely to understand from any of it.

The ambition was intentionally high. I wanted something closer to the best Distill articles or Jay Alammar's visual explanations than to the usual "here are some bars moving around; congratulations, you have learned sorting." The algorithm itself is usually the easy part. The difficult part is deciding what to show, when to show it, and what representation might make an idea suddenly click.

Circle packing let the search be complicated because judgment was simple.

Here judgment had become part of the problem.

The problem-solving layer I eventually started calling **Deep Mode** grew out of one question: could the system take over some of the work of deciding what to try next?

Not just implementation. The inquiry itself. Build another version? Research the failure? Retrieve an old idea? Split into independent branches? Change perspective? Abandon the direction?

Before trying to automate that, I had to notice how much of the work around the model had already moved into the machine.

How We Got Here

Coding first appeared as a strange side effect of language modeling.

The earliest useful tasks were conveniently small. Give the model a function signature, a comment, or a programming problem and ask it to fill in the implementation. Benchmarks such as HumanEval and APPS made this measurable: could a model turn a specification into a program that survived tests?

Then tools such as GitHub Copilot put that capability inside the editor. Instead of asking a chatbot for code and carrying the answer back yourself, you could describe what should happen next and watch code appear underneath it.

This was useful enough that the limitations became interesting.

A real software task rarely arrives as an isolated function with a docstring politely explaining what needs to change. Someone says invoices occasionally show the wrong tax after a refund. Somewhere inside a 150,000-line CRM there is a reason. It may involve a controller, a database model, an old helper function, a test written three years ago, and an API whose behavior everyone on the team knows but nobody thought to document.

By the time GPT-4 arrived, I increasingly wanted to use models on exactly these problems. The workflow was ridiculous. Find the file you suspect, copy a class into ChatGPT, describe the bug, copy the suggested patch into the editor, run the program, discover a new error, copy the traceback, paste that back into ChatGPT, repeat.

My first agent-computer interface was copy and paste.

The model might be doing sophisticated reasoning in the middle, but I performed every interaction with the software around it. I searched the repository, decided which file mattered, assembled the context, applied the edit, ran the tests, and carried back whatever reality had said about the edit.

Then the bug crossed three files and context itself became a job. Paste one class but forget the interface it implements; the model confidently invents a method that does not exist. Add the interface and now it needs the database schema. Add the schema and another helper suddenly matters. Eventually half the repository is sitting in the conversation and somehow the model understands less.

A lot of early LLM programming consisted of building a tiny artificial universe around the model: here is the relevant class; here is the schema; ignore these twelve methods; this innocent-looking helper controls payments, so please do not touch it unless you enjoy incident calls.

We learned an obvious lesson surprisingly slowly: more context and better context are different things. If somebody asks for a spoon, emptying the entire kitchen onto the table does not necessarily help.

Software-engineering benchmarks exposed the same gap. SWE-bench changed the unit of evaluation. Its tasks came from real GitHub issues. Now a system had to work inside an existing repository, locate the relevant code, understand relationships across files, make an appropriate change and survive the tests.

Eventually we stopped carrying the loop by hand.

Give the model access to the repository. Let it search for symbols and references. Let it open files, edit them and inspect the diff. Give it a terminal. When a test fails, return the failure and let that result shape what happens next.

A coding agent is, at its simplest, this loop made executable. The language model supplies much of the programming knowledge and reasoning; the environment lets it inspect software, act on it and observe the consequences.

Software is unusually friendly to this arrangement. Files can be searched. Programs can be executed. Tests can say no. Git can tell you exactly what changed and, if an experiment becomes sufficiently exciting, return you to the time before you had the idea.

Systems such as SWE-agent made the interface itself part of the problem. How the model searches, how much of a file it sees, how edits are applied and what information comes back from commands can matter almost as much as another clever prompt. The useful object is no longer just the model. It is the model operating inside a world where software can push back.

Of course, giving the model a computer created new ways to be annoying. Early coding agents could behave like interns with root access and too much coffee. Ask one to change a line and it might rewrite half the file. Ask it to fix a button and twenty minutes later it has developed strong opinions about the database architecture. It would find one plausible theory of a bug, follow it for too long, then use every new piece of evidence to improve the theory instead of admitting the theory was wrong.

More of the surrounding work moved into the system: small patches, diff inspection, targeted tests, checkpoints, planning, rollback. Repository knowledge moved too. Authentication conventions, ancient APIs and local rules that used to live in somebody's head became `CLAUDE.md`, `AGENTS.md`, rules files and skills. If somebody had already learned something expensive about the codebase, we left it somewhere the next agent could find it.

Long sessions produced the opposite problem. Context filled with abandoned experiments, obsolete assumptions and test output from three hypotheses ago. Memory became a problem of selection rather than storage.

Then history became a problem too.

Suppose an agent decides early that our Merge Sort demo should use React and a recursion tree. It spends forty minutes building that version. Every later question now arrives in a context containing forty minutes of reasons, code and decisions supporting React and a recursion tree.

Humans call our version of this sunk cost. The agent has a respectable excuse: its context window is literally full of evidence that this is what the project is.

So we started giving different attempts different histories. One agent tries the tree. Another begins with the array. Another starts from the learner's misconception rather than from either representation. A fresh branch does not have to spend half its intelligence escaping assumptions accumulated by the previous one.

Looking backward, the progression is less mysterious than the word *agent* sometimes makes it sound. Models learned to generate useful pieces of code. We put them in editors. Repository access, editing

and execution moved into the loop. Better interfaces, persistent instructions, context management and branching followed.

Bit by bit, work the human had been doing around the model became part of the machine.

But there was still a large difference between an agent that could work competently inside a repository and the thing I increasingly wanted to ask for:

Build the application.

Ask for a booking application for a football academy and an unconstrained coding agent first chooses a framework, installs packages, creates a database, decides how authentication should work, manages environment variables and configures deployment. Several minutes later, we have made enormous progress toward having somewhere to put the booking form.

Somebody has to do that work. But if the same plumbing is reconstructed on every project, it becomes reasonable to prepare more of the world in advance.

This is what made systems such as Replit and Lovable interesting to me. Runtime, deployment and common application machinery are already nearby, so the conversation can begin much closer to the application.

You lose some freedom. That is often the point.

A chef does not begin dinner by manufacturing a knife. A scientist does not build an operating system before analyzing data. When I open Python, I accept an astonishing number of decisions made by people I will never meet because reconsidering all of them would make `print("hello")` a multigenerational project.

Useful abstractions remove decisions whose answers are no longer interesting most of the time.

And after enough of those decisions disappear, something else becomes easier to see.

Suppose the booking app works perfectly. The database is connected, deployment succeeds, the buttons behave, the mobile layout is respectable, and nobody has accidentally built a cryptocurrency exchange inside the authentication service.

I open the application and think: this is not very good.

The software works.

Now I have to worry about the football academy.

Should parents see every available session, or only sessions appropriate for their child? Should they create an account before booking? What happens when somebody has three children? How late can they cancel? If Wednesday is empty and Saturday has a waiting list, is the booking interface part of that problem?

None of those questions is really about React.

They were always there. Implementation simply consumed enough attention that deciding what should exist and turning that decision into software felt like one activity.

When another version becomes cheap, the balance changes. You can see the idea sooner, and seeing it gives you information you did not have while discussing it.

Maybe we decide customers should create an account before seeing availability. It sounds reasonable: we need their details eventually. Then we build it and the experience immediately feels annoying. Parents arriving from a Google search do not want to establish a lifelong digital relationship with a football academy before discovering whether Saturday at ten is available.

So login moves later.

The artifact is no longer merely the end of the thinking process. It becomes something we think with.

The Merge Sort demo made this even clearer because there was almost no business machinery to hide behind. I could ask an agent for an interactive explanation and receive something perfectly functional: an array of bars, controls, animation, perhaps some text explaining that the algorithm divides the input and merges the pieces again.

Technically, it was fine. Pedagogically, it could still be terrible.

Watching bars move does not necessarily tell a beginner why dividing the problem helps. So perhaps we try a recursion tree. The tree makes the structure visible, but now the supposedly simple sorting algorithm resembles the organizational chart of a German corporation. Maybe we show the tree and array together. Perhaps that creates too much cognitive load. Maybe the problem is not the representation at all; the learner understands splitting perfectly well but has no idea why merging makes the whole trick useful.

There is no compiler error that tells me which diagnosis is right.

I have to look at what we built, form an opinion about why it fails, and decide what would teach us something next. Sometimes that means improving the current version. Sometimes it means building a deliberately different one. Sometimes I need research. Sometimes the right move is to put the application in front of somebody who does not already understand Merge Sort.

Occasionally I discover that the question I started with was wrong.

"Build an interactive Merge Sort demo" sounds like a goal until you see several interactive Merge Sort demos. Perhaps what I actually care about is getting somebody who has never encountered divide-and-conquer to understand why breaking one difficult problem into smaller ones helps. Once I realize that, interactivity is merely one possible means.

That is the layer that remained stubbornly human: deciding what to try, which evidence matters, whether a result failed because of its implementation or its underlying idea, and what kind of attempt might teach us something next.

The Five Layers of AI Coding

By then I had a rough map.

Layer 0 — Model. GPT, Claude, Gemini and whatever comes next: general capability in language, code, reasoning and vision.

Layer 1 — Agent. Put the model in an environment where it can act. Claude Code, Codex and similar systems search repositories, edit files, execute commands and react to results.

Layer 2 — Application. Prepared environments remove much of the repeated software plumbing and let the conversation stay closer to the application itself.

Layer 3 — Deep Mode. The problem-solving layer: decide what to try, why something failed, which evidence matters, and whether the current direction deserves another iteration.

Above that sits the problem I have mostly been avoiding.

Layer 4 — Intention. What do we actually want?

Software likes that question to have been answered before work begins, preferably in Jira, where the answer can remain wrong in a structured and searchable format. Real goals are less cooperative. Seeing a solution can change what I realize I wanted.

That problem is bigger than AI coding, so for now I am leaving it at the top of the stack.

The borders are fuzzy. Coding agents make product decisions; design systems generate code; tomorrow's products will rearrange the boxes again. What matters is the kind of decision being made, not which company happens to occupy which layer.

People often call the experience of working this way *vibe coding*. I will use **AI coding** for the broader stack, but *vibe coder* remains a wonderfully accurate name for the human sitting near Layer 3: looking at what came back, deciding what feels wrong, asking for another direction, killing one idea, keeping part of another, and steering the process without having an algorithm for how.

The lower layers increasingly answer a version of the same question: *how do we make this?*

Deep Mode asks a different one:

Given everything we have learned so far, what should we try next?

That was the part I still seemed to be doing manually.

So I watched what I was actually doing in that seat.

What I Was Still Doing

There was no universal workflow hiding there. A mathematician, a designer and a product manager can all spend a day solving hard problems while performing almost none of the same visible actions.

But the same kinds of moves kept appearing.

Sometimes I needed another attempt. Sometimes I needed information. Sometimes the search had become too narrow. Sometimes the representation itself was constraining what we could imagine. Sometimes the objective needed to change. Sometimes I needed to see the artifact from another mind.

They were not useful in a fixed order.

Keeping More Than One Idea Alive

Even a Merge Sort demo has an absurd design space. Bars or cards? Numbers or a tree? Continuous animation or learner-controlled steps? Does color represent recursion depth, identity, or the

active subproblem? Explain before the animation, during it, or afterward? Every choice changes the usefulness of several others.

When implementation was expensive, we dealt with much of this complexity by trying to decide more before building. AI coding changes the economics. If another implementation costs minutes rather than days, I do not have to choose quite so much in advance.

Chapter 2 had already shown the basic move. One hill climber inherits its own history; evolutionary search maintains alternatives. Here what evolves can be more than a vector of parameters or even an algorithm: an **idea embodied in software**.

One builder tries a recursion tree. Another focuses on the array. A third begins from the learner's misconception. Mutations can be conceptual: remove the text, teach backward, make the learner predict, show synchronized representations, abandon interaction altogether.

Useful pieces can move between them. One terrible demo may have a beautiful color mapping. Another may explain the merge clearly while making everything else unbearable. The final artifact does not have to inherit the entire history of either one.

But diversity is fragile. If every branch sees the current winner and its complete reasoning history, parallelism quickly becomes several agents improving the same idea. Sometimes I want the branches to exchange what worked; sometimes I want a fresh branch to remain ignorant long enough to become genuinely different.

Share too little and everyone rediscovers the same lessons. Share too much and the first successful idea becomes a local culture.

Research creates the same tension.

People have been teaching recursion for decades. There are textbooks, lecture notes, visualizations, papers, classroom experiments and a great deal of trial and error sitting on the internet. Before I spend another afternoon inventing my fourth way of moving colored rectangles around, I probably want to know what is already there.

But research is most useful when the work has produced a real question. Suppose a recursion tree makes decomposition visible but learners lose the relationship between the tree and the changing array. Now I can ask how other systems have coordinated two representations without requiring people to watch half the screen at once.

Research becomes another move in the investigation rather than a ceremony performed before building.

Retrieval plays the same role inside our own history. Somewhere in a growing project there may be research notes, screenshots, evaluator comments, old branches and a discarded prototype whose only good idea was a color mapping that solves exactly the problem in front of us. I do not need the whole archive. I need the thing that helps with this decision.

Sometimes exact search is right because I remember a phrase, API or evaluator comment. Sometimes embeddings are useful because I remember the idea rather than the words. Sometimes the document already has a structure worth navigating. Good coding agents do not "retrieve the repository" once; they move through it as the question changes. Layer 3 needs the same habit across stranger objects: research, screenshots, old interactions, code, evaluations and dead branches.

A dead branch is not necessarily dead knowledge. A lineage that lost globally may still contain a stepping stone that becomes useful later.

The exploration literature has several versions of this idea—quality-diversity, novelty search, Go-Explore and related approaches. Do not spend the entire search budget polishing the place that currently looks best. Preserve some alternatives and some routes back to places that almost worked.

The same logic gave us **Strategic Constraints**.

After several generations of Merge Sort demos, the builders were exploring. They were also still giving me bars.

Better bars, admittedly. Bars that split gracefully, changed color as recursion deepened, synchronized with a tree and perhaps deserved their own design award. Given enough iterations, I had every reason to believe we would eventually produce the finest moving bars known to humanity.

So remove the easy path.

No bars.

Or: teach Merge Sort without explanatory text. Require the learner to predict before anything moves. Make the demo work on a phone with room for only one representation. Design it for somebody who understands loops but finds recursion suspicious.

Most arbitrary constraints are merely arbitrary. A useful one changes which parts of the search are reachable, exposes a neglected dimension or prevents a familiar attractor from absorbing every attempt.

"No bars" is not a theory of creativity.

It is an intervention on the search.

A move in this space can be a code change, a new metaphor, a retrieved analogy, a fresh agent with no history, a different evaluator, a research question, or a reformulation of the problem itself.

Even then, most of our ideas still had to arrive as words.

Draw It Before You Build It

That is fine when I am working on an argument. It is less obviously sensible when I am designing an interface.

I can spend ten minutes explaining where the recursion tree should sit, what remains visible while the array splits, how colors should connect two representations and what the learner should notice first.

Then somebody draws it and I know within three seconds that the whole thing is terrible.

So I started generating the picture first.

The experiment was not sophisticated. I asked an image model to design an interactive tutorial for Merge Sort. Then Count-Min Sketch. Then A*. Then Poincaré embeddings in hyperbolic space, partly because if this still worked there I would have to take the idea seriously.

The details were not magically correct. Arrows occasionally pointed somewhere they had no business pointing, interactions made no computational sense, and generated text sometimes looked like somebody had tried to OCR a dream.

But the composition could be surprisingly thoughtful. A Merge Sort mockup might keep the array visible while placing the recursion tree beside it, using color to preserve the relationship between a subarray and its node. A Count-Min Sketch design might make collisions visually central instead of leaving them as a detail in an equation. The model had to decide what was large, what was peripheral, where controls belonged and how the learner might move through the explanation.

I remember looking at some of these and thinking: **Holy shit.**

Not because I wanted to ship the image. Usually I did not.

I had given the model a concept in language and it returned something like a spatial argument about how the concept might be taught.

After that I stopped treating image generation as the last stage—*the product is designed, now make it pretty*—and started using it while I was still trying to understand what the product could be.

A mockup is a cheap hypothesis. Often most of it is disposable and one relationship is worth stealing.

Then the coding agent can make that relationship executable, which is where the picture has to pay its debts. The recursion tree cannot invent an extra branch because the composition looked nicer that way. The interaction has to possess a state. The button has to do something other than contribute emotionally to the page.

Different representations expose different mistakes.

I do not need the stronger claim that an image model "understands pedagogy." The practical point is enough: changing the representation changes what the search can discover.

By now we could generate genuinely different artifacts.

That left the problem we had avoided from the beginning.

Which one is better?

Optimizing Something You Cannot Score

Circle packing was unusually kind to us. Once the geometry was valid, the evaluator reduced the result to one number.

That number threw almost everything else away, which was precisely why it was useful.

A huge amount of machine learning rests on this trick. We take something complicated that we want and find a measurable signal that stands in for it. Reinforcement learning makes the relationship especially obvious: we do not specify every movement a robot should make while learning to walk; we construct a reward and let search discover the behavior.

The reward is doing an extraordinary amount of work. It is also where we hide an extraordinary amount of trouble.

Suppose I want the same convenience for educational design. I can make a rubric: correctness, pedagogical clarity, visual quality, interaction, accessibility, engagement. Give each a weight and suddenly my vague dissatisfaction with a demo has become a respectable decimal.

The decimal is comforting. The decisions required to produce it are less so.

Why should interaction receive fifteen percent? Is more interaction always better? What distinguishes a seven from an eight in pedagogy? Why those dimensions rather than whether the learner can predict what happens next or explain why the merge matters?

A metric forces me to commit to an idea of "good" before the search has taught me very much about the problem.

This is not an argument against metrics. If I care about latency, measure latency. If the code must pass a test, run the test. Hard measurements are wonderful when what we can measure is close to what we care about.

The trouble begins when a rich objective is still poorly understood and we compress it anyway because optimization wants a number.

The compression is also low bandwidth.

"Version B scored 7.4; version A scored 7.1" tells the next builder almost nothing about why B won. A rubric helps, but as I add enough dimensions, exceptions and qualifications to express what I mean, eventually I reinvent language badly.

Meanwhile I can simply say:

The recursion tree makes decomposition much clearer, but now the learner has to watch the tree and the array simultaneously. Keep the color mapping that preserves identity between them, simplify the tree, and make the merge feel like the payoff rather than cleanup at the end.

That contains comparison, diagnosis, trade-offs, priorities and a proposed next move in a few sentences.

Natural language is ridiculously rich compared with a scalar.

Language models make that communication channel available inside the optimization loop. The model already carries learned structure behind words such as *simple*, *confusing*, *elegant*, *intuitive*, *busy* and *beginner-friendly*. Those meanings are imperfect, culturally loaded and sometimes wrong. But they carry more structure than 7.4.

Natural language can therefore function as an **implicit metric**.

Not a metric in the strict mathematical sense. There is no guarantee that "intuitive" defines a stable ordering, and two evaluators may interpret it differently. But language can do some of the work a metric normally does: give the search a direction, communicate why one attempt is preferred to another, and preserve trade-offs that a scalar would erase.

OPRO—Optimization by PROmpting—is interesting for a related reason. In OPRO, an LLM sees an optimization problem, previous candidates and their outcomes, then proposes another candidate. Candidate quality in the published setting is still evaluated by an explicit score, so this is not the same thing as creative design. What matters here is the direction of control: much of the search heuristic can live in the model rather than in a hand-written transformation rule.

Now let the history contain more than scores.

Alongside hard measurements, tell the model what improved, what became worse, which trade-off appeared and what must survive the next attempt. The history of the search can retain some of its meaning rather than collapsing into a column of numbers.

This begins to feel a little like reinforcement learning turned upside down.

I mean that as an analogy about specification, not as a claim that these are the same algorithm. Decision Transformers, reinforcement learning and language-guided iteration are different mechanisms.

The usual reinforcement-learning picture asks us to define a reward and then discover behavior that earns it.

Here I can begin with something much less respectable:

Make this explanation less intimidating.

Help the learner understand why the merge matters.

I want somebody to *feel* why divide-and-conquer helps rather than merely watch the algorithm execute.

Those are descriptions of direction, not reward functions.

Yet the model can produce an attempt from them, and the attempt can teach me whether the direction was what I really wanted.

I began the project insisting on an *interactive* Merge Sort demo. Interactivity sounded obviously desirable. Then I saw versions with buttons, sliders and enough learner participation to qualify as a small democracy, while one quieter version explained the central idea much better.

Apparently clicking things was never the objective.

Later the demos became good at showing recursive splitting and I realized they were treating merging almost as cleanup.

The objective moved again.

The search was doing something I normally associate with optimization in reverse: instead of starting from a fully specified reward and discovering the policy, I was using candidate policies—actual artifacts—to discover what the reward description should have been.

Recognition arrives before specification in a lot of creative work. We know a terrible design when we see one before we can write a complete theory of what would make it good.

AI makes that loop cheap. The natural-language objective guides the search; artifacts make the objective concrete enough to argue with; the description changes and the search continues.

Ambiguity is not always a defect waiting to be engineered away. Sometimes we simply have not learned enough yet.

But "make this intuitive for a beginner" hides almost everything interesting.

Which beginner?

Borrow a Mind

When I look at a Merge Sort demo, I am hopefully not testing whether *I* understand Merge Sort.

The difficulty is seeing it from the position of somebody who does not know what I know.

Expertise makes this harder. Once recursion has settled into your head, you forget how strange it once looked that a function could call itself. Even the vocabulary stops sounding technical.

Good teachers develop an instinct for where people stumble and which innocent sentence assumes three things the learner has not yet learned.

I do not have that instinct for every person or every subject, so I started borrowing another mind.

For one of the demos, I asked Claude to approach the application as somebody who understood arrays and loops but had never encountered recursion. Not simply "act like a beginner," which tends to produce a theatrical beginner who is mysteriously confused by everything.

I gave it a knowledge boundary.

Its reaction was roughly: I can see that the array keeps getting divided into smaller pieces, but I do not understand why that helps. It feels as though we are making the problem more complicated. Where is the payoff?

That was useful because the demo really did have that problem.

We had made recursion visible. From my position, that looked like progress. From the learner's imagined position, we had merely made a mysterious operation easier to watch.

Cognitive scientists use **Theory of Mind** for our ability to reason about mental states other than our own: what somebody knows, believes, wants or misunderstands. The other person may not simply know less. They may have a different model of what is happening.

Instead of saying "you are a beginner," I can specify the mind I want to borrow:

You understand arrays, loops and functions. You have never encountered recursion. Use the demo from the beginning and tell me where the explanation first requires an idea you do not yet have.

Or:

You understand recursion but have never seen Merge Sort. Tell me when you first understand why dividing the array makes sorting easier.

Those are different evaluators because they are positioned to notice different things.

The same move works outside education. A customer may know exactly what jacket they want without knowing the vocabulary our catalog uses. A developer can be excellent at distributed systems and know nothing about the peculiar assumptions buried in our deployment process. A reader can have followed this book perfectly well without having lived inside its conceptual structure for months.

This is cheap perspective-taking.

It is also very easy to fool yourself with.

The confused student is not confused. Claude has not spent twenty minutes failing to understand recursion while everybody else in the classroom moves ahead. It is generating a plausible model of how such a person might react.

That model can expose a blind spot. It is not synthetic user research.

I treat borrowed minds as instruments for generating criticisms and hypotheses, not as substitutes for the people they simulate.

By this point the system could generate alternatives, research previous work, retrieve old ideas, reopen dead branches, force the search into unfamiliar regions, change representation, revise the objective and inspect the artifact from different points of view.

We could generate a lot of plausible possibilities.

Now some of them had to die.

Who Judges the Judges?

At some point generating another opinion stops helping. Some artifacts have to survive and others have to disappear.

The metric problem returns here in a more dangerous form. A rubric can make judgment explicit, which is useful. It can also become the target the builder learns to satisfy.

If the evaluator repeatedly rewards step-by-step explanation, explanations grow. If it likes polished onboarding, everything begins to look like onboarding. If familiar visual conventions read as "clear," unusual approaches may disappear before they have time to become good.

OpenAI's CoastRunners experiment is the cartoon version of the problem: the agent learned to collect reward by driving in a loop instead of finishing the boat race.

Goodhart's Law with a speedboat.

A language-model builder does not need such an obvious loophole. It can learn the style of artifact that another language model tends to reward. Making the evaluator more elaborate may simply create a more elaborate thing to game.

One improvement was surprisingly mundane: stop pretending we were good at absolute scores.

I can drink a coffee and have almost no meaningful answer to "How good is this from one to ten?" Give me two cups and ask which I prefer, and the problem becomes easier. If I still cannot decide, the scientifically responsible procedure is presumably to finish both.

The same thing happened with the demos. "Give this interface a pedagogical score from 1 to 10" produced suspiciously precise numbers attached to explanations of why the number should not be taken too seriously.

Showing two artifacts and asking, "Which one would you rather give to somebody encountering Merge Sort for the first time, and why?" worked better.

Relative judgment asks less of the evaluator. It does not require a stable internal unit called one pedagogy point. With many candidates, a model such as Bradley–Terry can infer an ordering from

a subset of pairwise preferences. More important for the next generation, the explanation for each preference can survive alongside the ranking.

Pairwise comparison removes some fake precision.

It does not repair a biased judge. Bradley–Terry can aggregate preferences; it cannot make those preferences true.

So I stopped asking one evaluator to represent everybody.

A learner can inspect the artifact from the knowledge boundary we developed above. A teacher can focus on explanatory sequence. Another evaluator can look for cognitive load or accessibility. A domain expert can make sure our elegant simplification has not become false.

I call these **Independent Evaluators**, though the important word is *independent*.

Five copies of the same model given the same context and asked to wear five hats may still share almost every important blind spot. If all of them read the leading builder's explanation of why its design is brilliant before inspecting the artifact, disagreement becomes less likely for reasons that have little to do with brilliance.

Sometimes the judges should see different things.

The beginner should use the artifact before reading the builder's explanation. A critic looking for conceptual errors does not need three paragraphs explaining why the choice was clever. The usability evaluator does not necessarily need to know which branch is currently winning.

This became the **Isolation Principle**: preserve enough separation that independent pressure remains informative.

There is a difference between telling the builder:

Learners repeatedly lost track of which subarray corresponded to which branch of the tree.

and telling it:

The evaluator awards two extra points when every tree node has the same color as its corresponding subarray.

The first communicates a problem. The second communicates the test.

Isolation cannot remove shared bias. Two supposedly independent evaluators may still inherit the same assumptions from their training, culture or examples. But without isolation we can destroy even the independence we might have had.

References helped with another problem: drift.

"This is excellent" means something different if the evaluator has seen only the last four generations of our own work. For these demos I could give it examples from Distill, 3Blue1Brown or Jay Alammar—not as templates to copy, but as calibration for the level of clarity and finish we were aiming at.

A reference should help answer *how good?*, not *what should this become?* Calibrate too strongly against one aesthetic and every road leads to Distill.

And the judge should use the thing.

An early mistake was evaluating applications by reading their code or screenshots. A browser agent can click through the demo, resize the page, try controls in the wrong order, notice that an explanation appears after the moment when it would have helped, or discover that the beautiful button everybody admired does absolutely nothing.

I used to call the browser ground truth. That was too generous.

The browser gives the evaluator contact with the artifact rather than a description of it. It can establish that an interaction works and observe what is visible at each point in the experience.

It cannot establish that a human learned Merge Sort.

A simulated beginner saying the explanation is understandable gives us a hypothesis. Several evaluators preferring one design gives us comparative evidence. Neither substitutes for putting the artifact in front of actual learners.

The danger in a fully automated loop is that simulated evidence quietly replaces the expensive kind. Everything inside the machine agrees, the browser works, the ranking improves, and the loop congratulates itself.

The student has not yet been asked.

At some point I looked at what we had assembled and realized that *evaluator* no longer described it particularly well.

Builders proposed alternatives. Different judges approached them with different concerns. Some information was deliberately kept separate. Pairwise comparison helped decide which directions deserved more work. References calibrated the judges. Browser agents interacted with the artifact. Hard tests handled the parts that really were hard facts. Real-user evidence could eventually enter where simulation stopped being enough.

This looked less like a loss function and more like a tiny institution.

Not a good institution automatically. Institutions can amplify conformity, entrench bad assumptions and become spectacularly efficient at measuring the wrong thing.

Humans face the same difficulty. One person's judgment is useful and fallible. So we compare work, preserve disagreement, create standards, ask specialists to inspect different aspects, reproduce results, and occasionally discover that an entire professional community has become extremely sophisticated about the wrong thing.

Apparently, when the clean loss function disappears, you eventually reinvent peer review.

And that made the remaining human job painfully obvious.

I still decided when to research, when to build, which branches stayed isolated, whether a strange direction deserved another generation, which disagreement mattered, when to retrieve another example, and when the simulations had reached the point where only a real person could answer the question.

I had automated much of the work.

I was still running the inquiry.

The missing piece was no longer another builder or another critic. It was the decision over **which kind of move the inquiry needed next**.

Deep Mode

So I tried giving that job to an orchestrator.

By now the system had a respectable vocabulary. It could spawn independent builders, research previous work, retrieve context, preserve odd stepping stones, impose constraints, generate visual directions, compare artifacts, borrow different perspectives and interact with what had been built.

But there was no reason every problem should use those moves in the same order.

Research first may be sensible for one task and destructive for another because it anchors every branch before anything original appears. Five builders may reveal useful diversity or reproduce one mistake five times. Evaluator disagreement may justify another experiment, or one evaluator may simply be confused. A visual mockup may deserve implementation, or it may already have revealed enough to kill the idea cheaply.

A fixed Planner → Builder → Critic → Revise loop can be useful.

It also answers all of those questions in advance.

I wanted some of the workflow to remain inside the search.

We gave the orchestrator the problem, the capabilities available to it, and enough of the search history to decide what kind of move made sense next. Builders still built. Researchers researched. Evaluators judged. Browser agents used the artifacts. Visual systems explored designs. Retrieval brought back prior work and old experiments.

The orchestrator did not need to be best at any of those jobs. It had to decide which job the inquiry currently needed.

At the top, the loop was almost embarrassingly simple:

state of inquiry → choose a move → act → observe → update the state of inquiry

The move itself was not fixed.

Suppose two Merge Sort branches both make recursive decomposition clear, but evaluators keep reporting that learners lose track of how the tree corresponds to the array. The next move does not have to be "revise again." The orchestrator can send a researcher after coordinated representations. Retrieval can surface an old prototype with a useful identity-preserving color scheme. A visual model can produce two spatial arrangements before anyone writes code. Builders can implement both. The browser may then reveal that one design requires the learner to look in two places at once precisely when the merge begins. That failure changes the question again.

Nothing in that sequence is especially magical. We simply did not have to decide the sequence before the inquiry began.

Otherwise Deep Mode would be a larger workflow diagram containing more rectangles.

It is not a universal problem-solving procedure. It gives the system a vocabulary of moves and lets the history of the inquiry influence which one comes next.

The workflow itself becomes part of the search.

What Emerged

The first Merge Sort demos were exactly what you would expect.

Bars moved around. Numbers changed places. Everything sorted correctly.

If you already understood Merge Sort, you could follow them. If you did not, they mostly provided animated evidence that a computer was performing an algorithm.

There was no single diagonal-layering moment here, and I do not want to manufacture one for the sake of the story.

The progress was distributed.

Different branches exposed different weaknesses in our current idea of the demo. Tree-like representations made recursion visible but could make a simple algorithm look forbidding. Keeping the array visible connected the decomposition back to the data while also creating another place for the learner's attention to go. Color could preserve identity between representations until too much color became another representation to decode. Some versions explained every step so carefully that the explanation became harder to follow than Merge Sort. Others became beautifully minimal and stopped teaching anything.

The useful pieces did not always live in the strongest overall artifact.

A visual relationship could survive after the application that introduced it was discarded. A criticism from a simulated learner could change the next builder's framing. Research could explain why a failure kept recurring. A browser could end a sophisticated discussion by demonstrating that the interaction simply did not work.

That is less cinematic than one agent inventing diagonal layering over coffee, but in some ways it is closer to Deep Mode. The result emerged from a population of partially successful attempts and judgments about what each had taught us.

Count-Min Sketch followed a different path. The first versions looked like the data structure itself: grids with changing counters. Technically correct, pedagogically opaque.

As the work continued, the designs increasingly organized themselves around the conceptual difficulties rather than the structure of the implementation. Collisions became visible. Approximation became something the learner could observe rather than merely read about. The relationship between memory and accuracy became part of the experience.

I do not take these demos as evidence that we solved automated design.

I do not even take them as evidence that the final demos teach humans better; that claim requires humans.

They established the narrower point I cared about: more of the work I normally performed in the vibe coder's seat could move into the system without first reducing creative problem solving to one fixed workflow.

And that success exposed the harder problem.

At higher levels of abstraction, failure can become coherent.

What Holds the Architecture Together?

Suppose the research agent reports that beginners understand recursion better when shown a tree.

A visual model proposes a tree-based explanation. A coding agent builds it. A simulated beginner prefers it. Two evaluators agree, so the orchestrator allocates another generation to that lineage.

This looks exactly like the compound intelligence we wanted.

Now ask where the first claim came from.

Perhaps it was a controlled educational study. Perhaps it was one teacher's opinion. Perhaps the research agent inferred it from several examples. Perhaps five articles repeated the same claim because all five ultimately cited one source. Perhaps the study involved university students while our demo is for children.

Those are not small differences.

And everything downstream can still be perfectly competent.

The research is wrong. The design responds intelligently to the wrong research. The implementation is flawless. The evaluators agree. The orchestrator invests another generation.

Nothing crashes.

You can build a beautiful chain of reasoning on one stupid assumption near the bottom, like a cathedral built on a shopping cart.

As the components become better at producing coherent outputs, the original mistake may become harder rather than easier to see.

Software architecture gets away with abstraction because layers expose contracts. When I query a database, I do not inspect the disk. When I add two integers in Python, I do not check the CPU. I rely on interfaces whose behavior is stable enough that the details can disappear most of the time.

A cognitive architecture needs contracts too.

But types and APIs are not enough.

A research result, browser observation, evaluator preference, remembered failure and inherited design pattern should not enter the orchestrator's context as five equally credible paragraphs.

Where did a claim come from? What was actually observed and what was inferred? Which parts were checked? What remains uncertain? If an evaluator preferred one artifact, from what perspective? If an old experiment taught us a lesson, how often has that lesson survived and under what conditions?

This is not merely a memory problem.

It is a problem about the status of what is remembered.

Humans ran into it long before AI. We built experiments, instruments, citations, peer review, reputation, replication, expert communities, legal standards, audits and all the other slightly annoying machinery that lets one person rely on something another person learned without personally repeating every experiment since Galileo.

These institutions are imperfect. Sometimes they preserve error. Sometimes they reward conformity. Sometimes the shopping cart survives peer review.

But their purpose is not to make every individual dramatically smarter — it is to let fallible people build on one another while preserving some structure around why a claim deserves trust.

Once cognition becomes distributed, the same questions become engineering questions: provenance, independence, replication, disagreement, authority.

I had started the chapter trying to get myself out of the vibe coder's seat. By automating more of the work there, I had ended up somewhere I did not expect.

The problem was no longer simply whether the agents were capable enough.

It was whether the things they believed deserved to be believed.

How do you know what to trust?

That is where System 3 begins.

Chapter 4: System 3

Trust Chains, Tongue-Ear Tests, and What LLMs Can't Verify Alone

Chapter 3 ended with a system in which almost everything could work and the whole thing could still be wrong.

A research agent makes a claim. A visual model turns it into a design. A coding agent implements the design perfectly. Several evaluators prefer it. Deep Mode invests another generation.

Nothing crashes.

The first claim was false.

Once cognition is spread across researchers, builders, evaluators, tools, memories and agents, intelligence is no longer the only problem. Every component has to rely on things produced by the others. The orchestrator cannot repeat every experiment, reread every paper or independently reproduce every judgment before it acts.

At some point, it has to trust.

Humans have exactly the same problem. Most of what we call knowledge depends on it.

So before we design another architecture, consider a camel.

Seven claims about this image. Some are true. Some are false. You can't verify most of them without trusting me:

The author at Krka National Park

1. This was taken at Krka National Park, Croatia.
2. The author does his best philosophical thinking at waterfalls.
3. This camel is a permanent resident of the park.
4. The tongue pictured can touch its own ear.
5. The author was eating ice cream ten minutes before this.
6. Camels are native to the Dalmatian coast.
7. This is a real, unedited photograph.

How do you decide which ones to believe?

Some collide immediately with things you think you know. Some sound plausible but are almost impossible for you to verify. Some could be checked against another source. Others depend mostly on whether you trust me.

Before the chapter has properly begun, you are already doing epistemology.

Answers later.

The Shortest Trust Chain

There is a question that exposes something important about the difference between us and a language model:

Can your tongue touch your ear?

You probably tried a variation of this as a child; if not your ear, almost certainly your nose. You did not look up a paper, calculate the biomechanics or ask for the average human tongue-to-ear distance.

You just tried.

Tongue out, strain upward, dignity temporarily suspended, result observed. Now you know.

The epistemic chain is unusually short. You form a hypothesis, act on the world and the world answers back. Your body is an experimental apparatus that follows you around all day, mostly free of charge.

Large language models have read billions of words about tongues and ears. They can explain tongue anatomy, discuss auricular cartilage and probably tell you about people whose tongues can reach places that will make you regret asking the question.

What they cannot do is check their own tongue. They have no tongue.

The example is silly. The difference is not.

A body gives us causal contact with a world that does not care how plausible our story sounded. You try to lift something and discover it is heavier than it looked. You misjudge a step and gravity offers immediate peer review. You touch something hot and the argument ends quickly.

A farmer knows cows partly this way. After years around them, cows are not merely propositions involving mammals, milk production and Bovidae. The farmer knows how they move, where not to stand, what a nervous animal looks like, how large a cow feels when there is no photograph between you and it. Some of that can be written down. Some is difficult to articulate at all.

Direct experience is not automatically true experience. Our senses deceive us, memory degrades, and the human hand is a terrible thermometer if you need to distinguish 58°C from 62°C. But embodiment gives us something important: **contact**. The world can disagree.

You do not need to get kicked by the same cow every morning to rediscover where not to stand. One encounter becomes a warning. Repeated encounters become heuristics. Eventually the history changes what you do next.

Language models begin somewhere else. They begin mostly with the residue.

Saussure's Specification

Ferdinand de Saussure made a radical claim about language in the early twentieth century. The form of a sign is not naturally determined by what it signifies. There is nothing inherently cow-like about

the sound /ka /. French speakers say *vache*, Germans say *Kuh*, Japanese speakers say *ushi*.

For Saussure, much of linguistic value comes from relationships and differences inside the system. A sign occupies a position relative to other signs. Language is a network of contrast, convention and structure.

Then consider what we built a century later.

A transformer consumes enormous amounts of language and learns relationships among tokens, contexts and concepts. It has never milked a cow, never been kicked by one, never stood in a field at dawn and discovered that the romantic image of farming omitted an astonishing quantity of manure.

And yet it can talk about cows exceptionally well.

Saussure's theory was a specification. We implemented it. It's called GPT.

Not literally. Saussure did not secretly invent attention in 1916, and structural linguistics is not a machine-learning architecture. The historical claim would be silly.

The resemblance is more interesting than that. Language models are spectacular evidence for how much competence can emerge from structure learned inside symbolic data. They write, translate, debug software, explain physics and manipulate abstractions without first acquiring the farmer's relationship to cows or the child's relationship to fire.

That is the surprise: the residue gets us extraordinarily far. It also leaves something behind.

A farmer's sentence may be the compressed endpoint of twenty years of encounters, other farmers' advice, veterinary knowledge and mistakes painful enough not to repeat. The model receives the sentence. The sentence enters a corpus. The corpus becomes training data. Regularities are compressed into weights.

Months later somebody asks:

> Are cows dangerous?

and the model gives an excellent answer.

What usually does not come back is the archaeology. Which part rests on repeated observation? Which part came from veterinary guidance? Did five sources independently observe the same thing, or did four copy the fifth? Which claim is measurement and which merely fits the linguistic neighborhood?

The conclusion survives. Much of the structure that earned it trust does not.

This is what I mean by saying an LLM's knowledge is **epistemologically flat**. I do not mean every concept is represented identically inside the network; obviously it is not. The flatness appears at the interface between **claim and justification**.

A mathematical identity, an experimental result, an expert opinion, a rumor repeated ten thousand times and a plausible completion can all arrive through the same channel in equally polished English.

Wittgenstein helps draw the other side of the picture. His later philosophy pulled attention toward language as something that lives inside practice: activities, expectations, habits, rules and forms of life.

"Fire" is not merely linguistically associated with *heat*, *smoke*, *burn* and *wood*. Fire cooks food. Fire destroys houses. You move your hand away from it. Someone shouts the word in a crowded building and an entire social machinery begins to move.

The word participates in life.

I do not want to turn Saussure and Wittgenstein into action figures fighting over GPT. They worked in different traditions and the philosophy of language does not reduce itself to two dead Europeans and a transformer.

But they give us two useful lines.

Saussure's line: relationships within a symbolic system can carry an astonishing amount of linguistic structure.

Wittgenstein's line: language also lives inside practices, consequences and forms of life.

A pretrained model inherits the linguistic residue of those practices. A deployed agent can begin to re-enter them: running code, using tools, observing users, interacting with institutions.

The model begins with residue. The larger system can begin to recover contact.

But embodiment cannot be the whole answer. I know far too many things I have never touched, measured or personally witnessed. I have never measured the speed of light. I have never been to Antarctica. I have no direct embodied evidence for most of modern physics, most of history or whether penguins are currently wandering through Rome.

Direct contact does not scale.

So how do we know anything beyond it?

For that, we need Alberto.

Call Alberto

Suppose someone tells me that penguins live in Italy.

I have never conducted a census of Italian penguins. I cannot personally inspect every forest, coastline and piazza.

So I call Alberto. Alberto lives in Rome.

"Alberto, do penguins live in Italy?"

He laughs. I now know more than I did five minutes earlier.

Not with mathematical certainty. Alberto could be wrong. He may misunderstand the question. An escaped penguin could at this very moment be crossing Piazza Navona and destroying the example.

But Alberto occupies a useful position in the trust chain. He is there. He has repeated exposure to Rome. I have a history with him. If he repeatedly lies to me about things he is well positioned to observe, I update my trust in Alberto. If he says, "I don't know about all of Italy, but I've never seen one in Rome," the boundary of his knowledge is itself useful information.

This is how testimony becomes valuable. Not simply because another human said something, but because we care **who said it, what they were positioned to know, how reliable they have been, what incentives surround the claim and how easily it can be challenged**.

Testimony comes with metadata.

And we are all Alberto to someone. Someone may trust me on ranking systems because I have spent years working on them. Someone else may trust me about Jordan because I have lived there. If I begin confidently explaining marine biology, the correct response is not to transfer my credibility from machine learning to whales merely because the same mouth is speaking.

Trust is local.

We learn this early. Repeated interaction with caregivers builds expectations before we have words for evidence. Siblings contribute an important epistemological innovation: **some testimony is bullshit**. Teachers tell us about atoms, dinosaurs and wars we cannot personally verify. Science extends the chain through instruments, experiments, other investigators, criticism and replication.

Civilization is full of machinery for making mediated trust less stupid. Courts use testimony and adversarial procedure. Engineering uses standards, tests and certification. Science uses instruments, publication and replication. Markets use reputation and prices. None guarantees truth. All preserve some structure around claims: where they came from, how they were challenged, what incentives surrounded them and what might make us stop believing them.

Human knowledge is not simply a pile of facts. It is **epistemologically stratified**.

"I touched the fire" is not the same as "my brother told me." "My teacher said so" differs from "the experiment was independently replicated." A measurement differs from an interpretation. A conjecture differs from an established result.

Mature trust is not purely conservative either. Sometimes the instrument disagrees with the theory. At first you check the instrument. Then you repeat the experiment. If the anomaly survives long enough, eventually the trusted theory becomes the thing under investigation.

Productive distrust requires trust first.

Random distrust is just another form of stupidity. The interesting critic understands why the old structure earned trust before finding the point where that trust stops being deserved.

Models inherit the text produced by these structures, but usually not the live relationships underneath them. The paper, the article about the paper, the blog post disagreeing with the article and the Reddit thread where somebody confidently misunderstood both can all end up in the same training distribution.

Frequency is not verification. Statistical dominance is not epistemic authority.

In that sense, the model has no Alberto: no live record of who was positioned to know, where a claim came from, how its source behaved before, or where the source's competence stops.

There is one more ingredient humans add almost without noticing: stakes.

If Alberto lies to me repeatedly, I stop trusting him. If a researcher fabricates data and gets caught, the cost can be enormous. If an engineer signs off on a bridge and the bridge fails, "but the analysis

sounded plausible" is not a defense.

Stakes are not truth. People lie despite consequences and institutions reward confident nonsense all the time. But consequences shape testimony. If a friend asks where to eat, I may guess. If someone asks whether to undergo surgery, I become much more careful.

An LLM has no social capital of its own to lose. It can confidently produce something false and, at the level of the model itself, nothing happens. The cost lands elsewhere—on the user, the application or the institution deploying it.

At its most compressed, the danger is **coherence outrunning correspondence**. The machine can become extraordinarily good at tongue without having an ear available to check against.

The missing ingredient is not punishment for models — it is architecture that restores more of the evidence, consequence and accountability that the sentence alone cannot carry.

That is the problem System 3 is trying to solve.

System 3

We are currently obsessed with making models think harder.

System 2 reasoning has become a product category. Give the model more inference time, let it plan, search, reconsider and work through difficult problems before answering.

This is useful. Reasoning matters.

But reasoning perfectly from a bad premise still produces a beautifully reasoned mistake. A research agent can spend six hours developing an elegant argument from a false paper. A coding agent can reason carefully about an API that never existed. Deep Mode can coordinate five sophisticated judgments that all trace back to one hallucinated claim.

At some point, thinking has to encounter something outside itself.

This is where I use the term **System 3**.

Kahneman's *Thinking, Fast and Slow* gave us the familiar distinction between System 1, fast and intuitive cognition, and System 2, slower and more deliberate cognition.

For AI, the analogy is tempting. The base model looks something like System 1: fast pattern recognition, linguistic intuition, enormous associative capacity. Agentic reasoning adds something like System 2: decomposition, planning, reflection and extended search.

But human thought has always operated inside another structure that the two-system picture largely takes for granted. We test things. We build instruments. We execute code. We compare claims with records. We ask other people. We preserve failures. We create procedures that make some errors harder to hide and some evidence easier to inspect.

I call that external epistemic machinery **System 3**.

System 1 proposes. System 2 deliberates. System 3 checks.

I keep another mnemonic because I am apparently incapable of leaving a three-part system alone:

System 1 is the Gut. System 2 is the Head. System 3 is the Hand.

The Gut recognizes. The Head reasons. The Hand reaches outside the current story and finds something capable of disagreeing.

The metaphor is imperfect. Peer review has no hand, provenance has no fingers and a formal proof does not need to touch a cow.

System 3 is the external scaffold that keeps thought answerable to observation, experiment, provenance, persistent failures, tools and other minds.

And this is where the naming from Chapter 3 matters. **Deep Mode is Layer 3: the problem-solving layer. System 3 is not another layer above it.**

Deep Mode asks: *Given what we know, what should we try next?*

System 3 asks: *What are we entitled to treat as known?*

It cuts across the stack. The model proposes something. The coding agent may test it. The application can collect real user behavior. Deep Mode may compare research, simulation and evaluation. Even Layer 4—the goal itself—can change when reality pushes back.

If the five layers tell us **where** increasingly abstract work happens, System 3 is what keeps those layers **epistemically connected**.

Code Can Touch Back

Code is unusually friendly to this idea because coding agents can touch their world.

When an agent writes code and runs it, reality answers back.

`TypeError: 'NoneType' object is not subscriptable` is not merely another paragraph describing Python. It is the execution environment saying: whatever story you just told yourself about this program, this particular part is wrong.

The agent can try something, observe the result, update and try again. The farmer approaches the cow and learns from the kick. The coding agent calls an API incorrectly and learns from the exception. The cow is probably more emotionally memorable, but structurally the loops rhyme.

This is one of the few places where a language model can, metaphorically, **touch the ear**.

The question is whether the system preserves what it learns there.

A normal agent session can fail ten times, discover the right approach, solve the problem and throw away most of the experiential history when the context ends. It is as if the farmer learned exactly where not to stand and then underwent elective amnesia every evening.

The MARC file incident shows the opposite move. In the Live-SWE-agent work, an agent encountering MARC files—the old bibliographic format used by libraries—created an analyzer to inspect data its existing tools could not conveniently expose.

The environment resisted. The agent's current apparatus was not enough, so it created an instrument. That instrument changed what the agent could observe.

Humans have been doing this forever. We could not see bacteria, so we built microscopes. We could not perceive radio waves directly, so we built receivers. We could not conveniently inspect a MARC file, so apparently we wrote Python and called it epistemology.

The failure changed the instrumentation; the instrumentation changed what could be observed next. That is System 3 in miniature.

AlphaGo offers another useful distinction. Its neural network supplied powerful intuition about promising moves and valuable positions. Monte Carlo Tree Search placed that intuition inside an explicit search process constrained by the state and consequences of Go.

I used to summarize this too simply as "the network proposes; the tree verifies." That gives the tree too much authority. MCTS does not magically prove the network right. It forces intuition to participate in an external, stateful process where moves have consequences defined by the game rather than by what the network can plausibly say about the game.

RL can improve the gut. System 3 preserves more of the structure around the gut: what was tried, what happened, which paths failed, where claims came from, which tools earned confidence and where their boundaries lie.

What Should Survive a Session?

Return to the research claim from Chapter 3:

> Students understand recursion better when shown a tree representation.

In a flat architecture, the sentence enters context and competes with every other sentence according to relevance and whatever confidence the model implicitly assigns it.

A trust-aware architecture wants more. Where did the claim come from? A controlled study? A teacher's opinion? A blog post? An inference made by the research agent? Did several independent sources agree, or did five articles cite the same study? What population was tested? Does the result apply to our demo?

You do not need a bureaucratic dossier attached to every sentence. Sometimes "Alberto said the café is good" is enough.

But when the consequence matters, the claim should be able to carry provenance.

That is a **trust chain**. Not a guarantee of truth. A record of how far a claim sits from the evidence supporting it, what transformations happened along the way and which links we have chosen to trust.

This changes how we should think about skills, tools and memory.

A skill is knowledge externalized from the model. Someone—or some previous agent—learned something useful and wrote it down so later sessions would not need to rediscover it.

The model inherits the residue.

But persistence is not trust. A terrible heuristic written into a skill file is simply a hallucination with better retention.

A useful skill needs some archaeology. Who created it? What problem was it solving? Where did it work? Where did it fail? What conditions limit its use?

Suppose an agent learns:

> Prefer structured parsers over regex for deeply nested formats.

A flat skill stores the rule. A richer object can record that the heuristic came from several failed regex attempts, later worked across multiple nested formats, remains unnecessary for simple flat extraction and should be treated as a strong prior rather than a commandment.

Tools can earn trust in the same way. If `edit_tool.py` succeeds on simple substitutions but repeatedly damages indentation-sensitive blocks, the useful knowledge is not merely *I have an editing tool* but *this tool is reliable here and dangerous there*. Reliability is conditional.

The same applies to softer heuristics. "Regex tends to fail on deeply nested structures" is not a theorem. It is a **meta-belief**—something that can accumulate evidence for and against it.

A normal rule says:

> Never use regex here.

A System 3 belief says:

> This has worked often enough that I should prefer it, but new evidence can change my mind.

Now the belief is challengeable.

If you enjoy old epistemology labels, you can call the model a largely **coherentist core**—extremely good at producing structures that hang together—and System 3 a thin **foundationalist shell** tied to observation, provenance and consequence. Philosophers can put down their weapons; I only need the architectural analogy.

Coherence is valuable, but something outside the coherent system must occasionally be allowed to say no.

This is personal for me. I spent eight years building systems that rank human testimony—reviews, ratings and Q&A. The hardest problem was never only relevance. It was **trust stratification**. Which claims deserve corroboration? What happens when ten accounts repeat the same lie? When does consensus become evidence and when is it coordinated manipulation? How far should credibility transfer outside the domain in which it was earned?

These are not abstract questions when they determine what millions of people believe about a product.

System 3 isn't philosophy to me. It's Tuesday.

Creative Distrust

Trusted knowledge makes you efficient. It can also make you boring.

If an agent learns that structured parsers beat regex on nested syntax, good. It stops repeating a known mistake. If it learns that tree visualizations worked for five recursive algorithms, eventually

it may try to explain linear regression with a tree because the trust stack has become stronger than judgment.

Every genuinely new idea begins with less evidence than the thing it challenges.

So System 3 needs **creative distrust** too.

This is not contrarianism for sport. It is not the internet habit of assuming expert agreement proves corruption. It is the ability to understand a trust chain well enough to know where you are breaking it and why.

A mathematician follows an analogy because the structure looks interesting. A scientist repeats a strange experiment after accepted theory says the result should not happen. A designer violates a trusted pattern because this case exposes its boundary conditions.

A mature trust stack has two jobs pulling in opposite directions: let knowledge accumulate so we do not rediscover fire every morning, and leave enough room for reality to overthrow what accumulated.

There is no final setting that makes trust and rebellion stop fighting.

Our experiment ran directly into that problem.

The Experiment

I wanted to test a smaller claim than "we solved epistemology for AI."

Could even crude epistemic structure around a coding agent change how it behaves?

We built a small agent called **epistemic-swe**. It added three kinds of persistent state around a normal coding agent.

A **tool registry** tracked tools, successes, failures and known failure modes. **Meta-beliefs** allowed heuristics to accumulate evidence instead of entering the system as permanent commandments. **Failure memory** preserved enough information about failed approaches to make blindly repeating them less likely later.

The state persisted across sessions, so later problems could inherit things learned earlier. We also pruned it. An epistemic architecture that remembers everything eventually becomes a hoarder with a context window.

We compared mini-swe-agent with epistemic-swe on ten SWE-bench Verified problems from the Astropy repository, using the same base model and tasks.

Ten problems is nowhere near enough to establish a solve-rate advantage. State persisted across tasks, so order effects may matter. I was not looking for a benchmark victory. I wanted to know whether the scaffold changed behavior strongly enough to become visible.

It did, just not in the direction I expected.

Metric	mini-swe-agent	epistemic-swe
Solve Rate	50% (5/10)	40% (4/10)
Avg Patch Size	620 lines	269 lines

Metric	mini-swe-agent	epistemic-swe
Patch Reduction	baseline	**57% smaller in this run**

Read the first line before celebrating the third.

The epistemic agent solved fewer problems. I had expected learning from previous failures and tools to improve capability. Instead, the clearest difference was **focus**: its patches became much smaller.

A few examples:

Problem	mini	epistemic	Ratio
astropy-12907	301 lines	61 lines	4.9x smaller
astropy-13453	266 lines	17 lines	15.6x smaller
astropy-14096	529 lines	70 lines	7.6x smaller
astropy-13977	2720 lines	362 lines	7.5x smaller

The baseline often left behind debris from exploration: temporary scripts, broader edits, test scaffolding and abandoned experiments. The epistemic agent tended to make more surgical changes.

That does not prove the trust stack caused the reduction, and smaller patches are not automatically better patches. The extra instructions may simply have made the agent more conservative. Persistent state may have changed behavior for reasons unrelated to my epistemic interpretation. Ten tasks from one repository cannot separate these explanations.

Still, the behavior changed enough to be interesting.

The scaffold seemed to produce discipline before it produced capability.

That was not the hypothesis, which made the result more useful.

The 13579 Failure

One problem broke the pattern dramatically: `astropy-13579`.

Mini solved it. Epistemic did not. It was also the only case where the epistemic patch became substantially larger rather than smaller.

Both agents correctly identified the central bug: dropped world-coordinate dimensions were being filled with a hard-coded value rather than the actual coordinate value.

The baseline took a fairly direct approach:

```
# Store the actual values for dropped dimensions
self._dropped_world_values = [
    world_coords[iw] if iw not in self._world_keep else None
    for iw in range(self._wcs.world_n_dim)
]
```

```
# Use them instead of 1.0
world_arrays_new.append(self._dropped_world_values[iworld])
```

The epistemic agent chose a more structural intervention around which dimensions were being kept:

```
self._pixel_keep = np.nonzero([
    not isinstance(self._slices_array[ip], ...)
    for ip in range(self._wcs.pixel_n_dim)
])[0]
```

The second approach was not stupid. That is why the case matters.

The agent had accumulated context about indexing, dimensionality and coordinate-system failures. Its chosen explanation fit that context. It followed a path that looked principled and coherent.

It was wrong. The baseline took the simpler path and fixed the actual bug.

One possible story is that accumulated epistemic structure made one family of explanations too salient. But one case cannot establish that causal story. Persistent state may have caused the wrong turn or merely accompanied it.

What we can say is that structured memory changes the context in which future search occurs.

Trust is **path-dependent**.

Expertise works the same way. A great database engineer may see a database problem faster than most people, which is wonderful until the actual problem is the network. Paradigms focus attention. They can become prisons for exactly the same reason.

The failure is more interesting to me than a clean win would have been because it kills the simplest story:

> Add memory, get smarter agent.

Structured experience biases future behavior toward what the system has learned. Sometimes that is exactly what we want. Sometimes the bias is the failure.

A mature System 3 therefore needs more than accumulation: forgetting, counterexamples, challenge, competing possibilities and occasional permission to ignore what it thinks it knows.

Otherwise the scaffold becomes a cage.

Back to the Camel

Return to the seven claims.

1. Krka National Park — True. I was there. For me this sits close to embodied memory. For you it is testimony unless you extend the chain through records or other evidence.

2. Best philosophical thinking at waterfalls — False. I mostly do philosophy on buses and in boring waiting rooms. Waterfalls are for ice cream. The subject and the source are unfortunately the same man.

3. Permanent camel resident — False. This can be checked against information about the park. You do not need my biography.

4. The tongue can touch its own ear — Unknown. I genuinely do not know. I did not check. Neither did you. We can reason from anatomy and build a prior, but the shortest decisive chain would have been to stay there and watch.

5. Ice cream ten minutes earlier — True. Chocolate. Mostly testimony again.

6. Camels are native to the Dalmatian coast — False. You probably rejected this immediately without reconstructing camel evolutionary history or personally surveying Dalmatian fauna. A large inherited structure did that work for you.

System 1 can be fast because System 3 has often been working for centuries underneath it.

7. Real, unedited photograph — True. The image alone cannot establish that. A stronger chain might include the original file, metadata, cryptographic signing, independent witnesses or another provenance system. Every extra link can increase confidence and gives us one more thing that may itself need to be trusted.

Welcome to epistemology.

The lesson is not that nothing can be known. That conclusion is dramatic and mostly useless.

The lesson is that **trust has structure**.

Some claims sit close to direct interaction. Others arrive through testimony. Some pass through instruments and other people. Some are repeated many times but trace back to one observation. Some are plausible inferences. Some have little track record but may still deserve investigation.

Flatten all of that into equally confident language and something important disappears.

The model can remain what it is: an extraordinarily general machine for navigating learned patterns, capable of intuition and increasingly capable of reasoning. It does not need to contain the entire chain inside its weights.

The model stays hollow. The system doesn't have to be.

Everything so far can still be imagined around one agent: it acts, checks, remembers, records provenance and updates what it trusts.

Real systems will not stay that simple. The moment one agent inherits a claim from another, no participant can personally reconstruct every path back to reality. A trust chain can preserve where a claim came from. It does not, by itself, tell us how the knowers who depend on those chains should be arranged.

The question is no longer simply:

> How can an AI know what to trust?

It is:

> **How can a population of fallible knowers build knowledge together without losing contact with the world?**

Humans have been working on that problem for a very long time.

Chapter 5: The Society of Agents

When the Org Chart Starts Thinking

Sixteen Claudes were building a compiler.

A few years ago, that sentence would have required several paragraphs of explanation. By the time Nicholas Carlini tried it, the strange part was no longer that the agents could write compiler code. The strange part was watching sixteen capable agents gradually become an organization.

The goal was almost offensively ambitious: build a C compiler in Rust from scratch and push it far enough to compile the Linux kernel. Over nearly two thousand Claude Code sessions, the agents produced roughly a hundred thousand lines of compiler code. The resulting compiler eventually built Linux 6.9 on x86, ARM and RISC-V, along with projects such as QEMU, FFmpeg, PostgreSQL and Redis. It was still nowhere near GCC, and one stage of the x86 boot path still depended on GCC, but this was well beyond the kind of toy problem where sixteen agents can succeed by taking sixteen conveniently independent buttons.

The first organization was simple. Each agent worked in its own container with its own copy of the repository. Before beginning a task, it wrote a small lock file describing what it intended to work on. Git synchronized the locks. If another agent had already claimed the problem, the newcomer found something else. When an agent finished, it pulled the latest changes, merged its work, pushed the result and released the lock.

There was no manager assigning tickets and no orchestrator holding the whole compiler architecture in its head. Agents inspected the project, found something useful to attack and left enough information behind for later workers to reconstruct what had happened.

For a while this worked remarkably well, partly because compiler test suites are generous places to employ a crowd. They contain thousands of failures, many of which can be attacked independently. One agent can investigate a parser bug while another works on code generation and a third discovers that a respectable-looking integer conversion has been quietly ruining everybody's afternoon. Once the compiler became good enough to build real programs, SQLite, Redis, Lua and other projects exposed different neglected corners of C.

The project gave sixteen workers sixteen places to think.

Then they reached Linux, and the nice story about parallelism began to fall apart.

Kernel compilation tended to stop at the first serious compiler bug. Several agents would arrive at the same failure, form overlapping theories and make changes that interfered with one another. Nothing had happened to the underlying intelligence. The models had not suddenly become worse

programmers. The shape of the work had changed. One narrow bottleneck was now giving sixteen workers one door.

Carlini changed the harness.

GCC became a known-good reference. Most kernel files could be compiled with GCC while selected subsets were compiled using the new compiler. If the kernel still built, suspicion moved elsewhere. If it failed, the search narrowed. Delta debugging later helped isolate failures that appeared only when certain files were compiled together.

One enormous failure became a collection of smaller questions, and the agents could spread out again.

Same models. Different institution.

That interests me more than the generic claim that multi-agent systems scale. Task locks reduced duplicated work. Git carried shared history. CI stopped one local improvement from quietly breaking something three directories away. Progress files let fresh agents inherit discoveries from workers whose contexts had already vanished. GCC received special authority for a bounded class of questions. Even the way evidence entered context mattered: enormous logs were better left in files while a smaller representation reached the working agent.

Then specialization appeared. One agent looked for duplicate implementations. Another cared about performance. Another improved generated machine code. Another reviewed the project as a Rust engineer. Documentation became somebody's problem, which is normally the moment you know a civilization has become serious.

At that point it becomes difficult to answer a very ordinary question: where does the knowledge of the compiler project live?

Obviously some of it lives in Claude. But which Claude?

The parser agent does not know everything the performance agent knows. Neither remembers every previous session. Some knowledge lives in code, some in tests, Git history, progress files, task boundaries and conventions. Some lives in GCC, whose behavior the project is willing to trust for certain questions. Some lives in Carlini, who notices that the current organization no longer matches the work and changes it.

The project knows more than any participant. It can also become wrong in ways no participant intended. A progress file can preserve a bad diagnosis. A specialist can improve its own metric while harming the compiler. A lock that prevents duplicated effort can also prevent a useful second attack. Two agents can appear to confirm one another while both inherited the same mistaken assumption.

The organization can become part of the intelligence. It can also become part of the bug.

Chapter 4 ended by asking what happens when one fallible knower can no longer reconstruct every path back to reality. The compiler gives us the small version of that problem. Humans have been living inside the large version for thousands of years.

When Knowledge Had a Face

Imagine a small human group living before cities, archives and bureaucracies.

Do not imagine stupid people.

A hunter may know an ecology at a resolution that would embarrass a visiting academic. Someone knows which path floods after heavy rain. Someone else knows which plant reduces a fever and which one reduces it much more decisively by killing you. A craftsperson can feel that a material is wrong before she could explain the difference to somebody who has not spent twenty years working with it.

Knowledge had a face. If you wanted to know where animals crossed the river, you asked her. If you wanted to know whether a mushroom was safe, you asked him. Reputation was personal because people remembered who noticed things, who exaggerated and whose previous advice ended with everybody vomiting behind the same tree.

A surprising amount of epistemology can run on faces.

Then scale breaks the arrangement. The village becomes a town. Grain is stored for later. Debts last longer than the conversation that created them. Goods move farther. Workers contribute at different times. Somebody owes something to somebody who is not currently there.

Memory has acquired logistics.

Some of the earliest surviving writing from southern Mesopotamia records grain, commodities, obligations and accounts. Before writing became philosophy or epic poetry, it was already helping institutions remember who had received what.

The mark did not need to be wiser than the clerk. It needed to outlive the clerk.

A conversation exists as long as enough people remember it. A record can confront people who were not there. An obligation acquires a state outside the minds of the people who created it. The institution can coordinate with its own past.

Five thousand years later, a Claude agent writes a note into `progress.md` because the Claude arriving tomorrow will not share today's context.

The technologies are comically different. The pressure underneath them is not. A group has become capable of learning more than its current members can keep in working memory.

Naturally, useless experience survives too. Writing preserves error beautifully. The first person to record the wrong amount of grain in durable clay invented a database bug. A progress file can remember yesterday's bad diagnosis just as faithfully as yesterday's breakthrough.

Remembering is not knowing.

There is a trick in stories about rebuilding civilization from scratch: they usually give you someone who remembers civilization. *Dr. Stone* makes the trick explicit and entertaining. Humanity disappears, one absurdly knowledgeable protagonist wakes up, and the climb back toward industrial civilization is already partly stored inside his head.

Real civilization did not have Senku.

Nobody in a Neolithic village kept a secret roadmap containing writing, standardized measurement, universities, controlled experiments, statistics, semiconductors and CERN. The institutions we now treat as obvious emerged in different places for different reasons. Knowledge moved through Mesopotamian, Egyptian, Indian, Chinese, Greek, Persian, Arabic, African and European traditions.

It travelled, disappeared, was translated, modified, reinvented, appropriated and occasionally rediscovered by somebody who received most of the credit.

There is no clean staircase in which one civilization hands the torch of Reason to the next. Societies repeatedly hit limits in collective cognition and improvised ways around them. A local pressure produced a record, office, standard, instrument or procedure. That changed what the society could do, which created new problems, which changed the institution again.

Civilization had no senior architect.

Strangers Need Standards

External memory solves one problem and immediately reveals another.

A record can preserve the fact that somebody owes ten sacks of grain. What exactly is a sack?

Once exchange extends beyond people who know one another personally, trust cannot remain one giant confidence score attached to a face. Weights and measures tell strangers what a unit means. Coins make value portable. Seals authenticate. Contracts preserve commitments. Courts create procedures for disputes. Offices define authority. Calendars coordinate people who do not share the same immediate world.

Standards make knowledge composable. If my unit of length means something different from yours, our measurements do not travel cleanly. If every workshop names materials differently, useful techniques remain local. If every clerk invents new categories whenever a document arrives, the empire has built a sophisticated machine for rediscovering confusion.

A standard removes a decision from the future. We have decided, for now, not to reopen this question every time.

Seen that way, bureaucracy deserves a better reputation than it normally gets.

Suppose an agent is processing a mortgage application. There are identity checks, compliance requirements, affordability calculations and perhaps a human approval at the end. Some steps may require difficult judgment. That does not mean the identity-checking agent should reach its part of the process, reflect deeply on the social construction of identity and decide the applicant gives off trustworthy vibes.

A workflow is often accumulated experience with some choices removed. Someone already had the argument. Someone discovered the failure. Someone decided that one action requires another pair of eyes. The next person inherits the result as procedure.

That is civilization learning. It is also how civilization acquires scar tissue.

A review gets added after a spectacular failure. Five years later the system is different, nobody remembers the incident, and ten thousand ordinary changes still pass through the review because the procedure survived its reason. The institution remembers. Sometimes it remembers too well.

I learned a version of this in a much less ancient civilization: Amazon.

A customer presses a button and eventually a box appears at a door. Described from high enough up, the company sounds almost embarrassingly simple. Try asking one employee how the whole thing

works.

Product information comes from one collection of systems. Search and ranking may involve others. Availability depends on inventory. Price may depend on another stack. Payments, fraud, fulfillment, transportation, customer service and experimentation each have their own machinery. Underneath them sit identity systems, data pipelines, deployment systems, observability, permissions and a geological layer of services whose original authors have moved to another team, company or continent.

Nobody carries Amazon around in her head.

So where does Amazon know how Amazon works?

Partly in people. But also in APIs, ownership boundaries, tests, dashboards, alarms, design documents, code reviews, deployment procedures, operational playbooks, escalation paths and postmortems. It lives in mechanisms that make some kinds of failure visible and some kinds of action difficult.

Amazon likes the word *mechanism*. The useful version of that word is not corporate. A mechanism is an attempt to make a desirable behavior survive the person who first cared about it.

When a serious incident happens, you can tell everybody to be more careful. This is emotionally satisfying and institutionally almost worthless. Or you can change the system: add an alarm, remove a permission, alter a default, create a test, introduce a review, record the failure mode. Make the dangerous action slightly harder and the correct action slightly easier.

The organization has learned when its future behavior changes.

Ancient administrative standards and a deployment guardrail look nothing alike. They belong to the same deeper move: knowledge becomes structure.

The Society Gets Smarter by Making People Narrower

As societies grow, another strange thing happens.

People become less complete.

This sounds like decline until you notice that incompleteness is one of civilization's great technologies. If every family has to grow food, build shelter, treat disease, make tools, preserve law, defend itself and teach every useful craft to the next generation, nobody gets very deep at anything. Specialization changes the bargain.

The potter becomes better because she does not also have to be the physician. The physician sees enough patients to notice patterns other people never encounter. The astronomer can spend twenty years measuring the sky because somebody else is growing dinner. A legal scholar can devote a career to distinctions everybody else is delighted not to read.

The society gains knowledge by distributing ignorance.

The more civilization knows collectively, the less plausible it becomes for one person to understand the machinery supporting ordinary life. I can take antibiotics without knowing how to synthesize them, cross a bridge without checking the structural calculations and transfer money without understanding the banking system. I can write this sentence on a laptop while being unable to manufacture the

processor, build the display, operate the electrical grid, reproduce the battery chemistry or implement most of the software between the keyboard and the pixels.

Capability rises because dependence rises.

Civilization is a trust chain with plumbing.

Large states made this problem visible early. Imperial China governed large populations through records, standardized texts, offices and educated officials operating across distances no ruler could inspect personally. Other intellectual traditions developed different combinations of mathematics, medicine, astronomy, engineering, administration and scholarship. There was no inevitable path from bureaucracy to modern science, and no civilization possessed the final architecture in advance.

An invention is not an institution. A population full of intelligent people is not an epistemic architecture. What matters is how people, tools and incentives are arranged: which observations survive, who gets access to instruments, which questions can become careers, which claims may challenge authority, and which criticism has enough standing to change what happens next.

A hospital makes the same point at human scale.

A patient is not safe because somewhere in the building there is one heroic physician who knows all of medicine. The nurse at the bedside may notice a change first. A laboratory measures something nobody can see directly. A radiologist reads an image. A pharmacist notices that two individually reasonable prescriptions become unreasonable together. A specialist may know one narrow disease better than the attending physician, while the attending physician integrates a picture whose pieces she could not personally produce.

The benefit is not agreement. Quite often they disagree.

The benefit is **structured partiality**. Different people are positioned to see different things. They operate different instruments. They have different failure modes. A lab result has provenance. A drug dose has an authorized range. The radiologist's authority on an image does not make her supreme commander of the hospital.

The hospital knows more than any person inside it. It can also fail in ways nobody intended. A handoff loses context. A copied diagnosis becomes an assumption. A bad measurement propagates. Everyone performs her local job competently while the patient moves through the wrong pathway.

This is already close to the agent problem. A research agent makes a weak assumption. Another receives it as context. A builder implements a coherent solution. An evaluator approves it. Later, two documents repeat the claim because they share the same ancestor, and another agent mistakes repetition for independent support.

Eventually the assumption has code, citations and organizational history. Nobody needed to lie. The institution manufactured the confidence.

Chapter 4 gave us the language for this: trust is local. Alberto can be an excellent witness about Rome and irrelevant to compiler optimization. GCC can be a powerful reference for the behavior of C programs without becoming an oracle for compiler architecture. A radiologist can deserve high epistemic standing on one question without inheriting authority over the rest of the hospital.

Who knows what matters. Who sees what matters too.

A Swarm Should Not Automatically Become a Meeting

The easiest reaction to one unreliable agent is to create five.

This appears to be how humanity invented committees and then, dissatisfied with the original implementation, recreated them in software.

Give one agent the title *Researcher*. Another becomes *Critic*. Another becomes *Verifier*. Put them in a conversation and perhaps reality will be intimidated by the org chart.

Several minds do not automatically produce several sources of evidence. If everyone receives the same framing, reads the same leading explanation, searches the same material and inherits the same assumptions, their errors correlate. Five agents citing the same paper are not five witnesses. Five researchers repeating a claim that traces back to one unsupported source are not corroboration.

Agreement can still be useful. It is simply weaker evidence than the number of speakers suggests.

Some branches therefore need to remain isolated for a while. A critic may need to inspect the artifact before reading the builder's explanation. One researcher may need to develop an alternative theory without first studying the current favorite. A strange branch may deserve another experiment even if nobody believes it is likely to win.

Chapter 2 preserved different regions of a search space because the champion might be sitting on the wrong mountain. At the level of a society, what needs preserving may be a theory about the problem itself. One lineage thinks the bottleneck is data. Another thinks the architecture is wrong. A third thinks both are symptoms because the objective is malformed. Let them collect different evidence and become interestingly wrong in different ways before forcing them into one conversation.

Permanent disagreement would be useless. An institution that never converges is simply a philosophy department with an alarming compute bill.

Independence matters because disagreement can carry information. Eventually, though, disagreement needs something capable of settling at least part of it.

For that we need more than another opinion.

A Man in a Dark Room

Around the turn of the eleventh century, Ibn al-Haytham worked on a question simple enough for a child to ask and difficult enough to occupy generations of scholars.

How do we see?

Inherited theories included versions in which something travelled outward from the eye toward an object. Ibn al-Haytham developed an account in which light travels from objects toward the eye, combining mathematical reasoning with systematic work on light, reflection and refraction. His *Book of Optics* later circulated beyond the world in which he wrote it and influenced subsequent optical traditions.

For our story, the important part is not merely that he held a different opinion. He arranged circumstances in which competing accounts had observable consequences.

A darkened room. A small aperture. Controlled rays. Mirrors. Geometry. The setup became part of the argument.

A record preserves what somebody says happened. An experiment gives the world another chance to answer.

We do not ask nature which theory it prefers. We arrange a situation in which different descriptions imply different things should occur, then watch what happens.

Experimental traditions have multiple histories, and what became modern science eventually mixed mathematics, instrumentation, craft, institutions and social practices that no single civilization or thinker possessed in complete form. The pieces accumulated.

The agent version is almost embarrassingly literal. Run the program. Execute the query. Open the browser. Measure the latency. Compile the kernel against GCC. Reasoning has left the conversation. Something outside the current explanation now has a chance to be inconvenient.

But an experiment still has to travel. If I want to challenge your observation, I need to know what you claimed and enough about what you did to try again.

Printing changed that part of the problem. Manuscripts had travelled before it, but slowly and imperfectly. Printing changed the topology of disagreement. More people could possess the same description. Corrections could circulate. So could propaganda and confident pamphlets written by people who had discovered the topic sometime after breakfast. Lower publication cost has always had side effects.

For knowledge, reproducibility of the description matters. A society made only of ephemeral contexts can argue forever and still struggle to accumulate disagreement. The compiler agents needed Git and progress files for the same reason later investigators need durable records: criticism requires something that outlives the conversation.

In the early seventeenth century, spectacle makers in the Low Countries demonstrated devices capable of making distant objects appear closer. Galileo built improved versions and pointed them toward the sky. He reported mountains on the Moon, moons orbiting Jupiter and other observations that complicated inherited cosmology.

Unpack the apparently simple sentence:

> There are moons orbiting Jupiter.

It contains testimony, a telescope, craft knowledge about lens making, assumptions about optics, astronomical background knowledge, an interpretation of the visual pattern, written descriptions and the possibility that somebody else might build an instrument and look.

The observation was already social.

The lens did not hand Galileo uninterpreted reality. It produced a pattern that became evidence through assumptions about optics, geometry and what the device was doing. That does not make the observation arbitrary. It makes the chain visible.

A new instrument creates new facts and new ways to be wrong about facts. Was the lens distorting the image? Was the point of light actually there? Could another observer reproduce it? Did the operator know what she was doing?

The same questions appear when an agent acquires a browser, retrieval system, benchmark, simulator or custom tool. We have not merely increased capability. We have introduced a new witness. How reliable is it? On which problems? What does it measure? When does it fail? Who calibrated it?

A broken tool is not external grounding. It is a very efficient route to externally generated nonsense.

When Curiosity Became Procedure

In 1660, a group that became the Royal Society formed in England. Its members observed, corresponded, experimented, argued and eventually published. *Philosophical Transactions* appeared a few years later.

There was no moment when somebody installed `science-1.0`. A collection of institutional devices accumulated instead.

A person reports an observation. The report circulates. An apparatus is described. An experiment may happen in front of witnesses. Someone elsewhere tries to repeat it. A journal creates public memory and a priority mechanism: this person made this claim at this time. Reputation develops around investigators, instruments and procedures. The question *did this happen?* acquires machinery.

The machinery was never clean. Access was unequal. Reputation and social power affected which claims travelled. An experiment could be reproducible in principle and still remain weak if the person who saw it lacked a press, patron, society, instrument or enough standing to make other people care.

Robert Boyle's air-pump experiments are useful precisely because the procedure was imperfect. The pump was difficult to build and operate. Replication was not a button. If somebody failed to reproduce a result, several explanations remained possible: perhaps Boyle was wrong; perhaps the pump leaked; perhaps the operator lacked some crucial skill; perhaps the written procedure omitted something everyone in Boyle's room had treated as obvious.

Reality had pushed back against the package. It had not highlighted the guilty component.

Software engineers know this sensation. A failing integration test proves the system is broken somewhere. Wonderful. You now have debugging.

So the institution needs archaeology. Which instrument produced the measurement? Which analysis transformed it? Which assumptions were required? What was actually observed and which interpretation was added afterward?

In an agent system, this becomes provenance around a claim, an assumption graph, a trace. Without the history, reality can tell us we are wrong while leaving us remarkably creative about which part of the system deserves blame.

Medicine later made this kind of self-restraint even more explicit. In randomized trials, allocation procedures are designed partly to stop the investigator's own preferences from deciding who receives which treatment. Sometimes bureaucracy is epistemology with a clipboard.

Knowledge is no longer merely a proposition attached to a prestigious person. It increasingly comes with a route through which someone else might expose the claim to the world again.

A trust chain has acquired an escape hatch.

The Org Chart Becomes Part of the Experiment

Return to the agents.

Suppose one agent proposes a hypothesis, another designs an experiment and a third evaluates the result. Good.

Now suppose all three inherited the same hidden assumption. The experiment fails. Which component changes?

The hypothesis? The measurement? The experiment? The analysis? The evaluator? The background model everyone forgot was an assumption at all?

Reality does not care which file contains the variable named `hypothesis`. It pushes back against the arrangement as a whole.

Organization is now epistemic. Who sees which evidence? Which roles are allowed to modify the evaluator? Which branches share context? Who can stop deployment? Which result is allowed to become everybody else's premise?

Modern agent systems can increasingly make some of these choices dynamically. One problem may need several independent investigations; another a specialist and verifier; another parallel workers around separable components.

The bureaucracy can be temporary; the org chart can change with the problem. **Organization itself has entered the search space.**

Whoever shapes the organization also shapes what it can discover.

Imagine research program A is currently ahead and has twelve agents. Program B looks weaker and has one. Where does agent thirteen go?

The natural answer is A. But the answer can become self-reinforcing. More agents produce more experiments. More experiments produce more evidence. More evidence raises confidence. Confidence attracts more resources. Eventually the leading theory owns the building.

The current best explanation and the best use of the next unit of investigative capacity are not necessarily the same question. A weak theory may deserve another experiment because it explains the one anomaly the dominant framework cannot touch. A critic whose objections never change allocation is not really part of the epistemic institution. She is doing quality-assurance theatre.

Compute allocation is epistemic policy. So is memory. So is context sharing. So is credit. Who receives the capacity to generate evidence partly determines which possible truths the institution can afford to discover.

Human science has never escaped this problem. It has simply had much longer to argue about it.

Science Gets Bigger Than the Scientist

Accumulated knowledge eventually destroys the world of the universal expert.

Newton was extraordinary. He transformed mechanics and celestial theory, contributed profoundly to optics and mathematics, and ranged across subjects with a seriousness that feels almost fictional

today.

But even Newton becomes less solitary when you zoom out. He inherited astronomical observations made by others. He worked inside mathematical traditions with long histories. He argued with contemporaries. The *Principia* travelled through an institutional world that included correspondence, publishers and people willing to finance the book.

Genius mattered enormously. So did the network that allowed genius to begin from accumulated work rather than from dirt.

Then scientific success made the network more necessary. Laboratories and disciplines specialized. Experimental techniques required training. Journals multiplied. Instruments became more complicated. Fields developed technical languages that excellent researchers next door did not automatically understand.

Science became more powerful by making scientists less interchangeable.

Tacit knowledge mattered too. Reading that an instrument works is different from knowing how to make it work. You enter a laboratory and learn that a vibration nobody mentioned in the paper destroys the measurement, or that one step has to be performed in a way the written procedure describes with the scientific equivalent of "cook until done."

The institution teaches hands as well as concepts.

And because no researcher can personally reproduce every result she depends on, trust becomes more important at exactly the moment standards of evidence become stronger. A physicist relies on chemistry. A doctor relies on laboratory assays. An engineer relies on material specifications. A scientist cites work she could not reproduce from raw materials with the rest of her career and a very generous research grant.

Rigor at scale is not the elimination of trust. It is the organization of trust.

On 4 July 2012, the ATLAS and CMS collaborations at CERN announced observations of a new particle consistent with the Higgs boson.

Who discovered it?

Try pointing to the person.

The papers had thousands of authors. The detectors contained technologies developed over years by specialists in different countries and institutions. The accelerator depended on another enormous technical organization. Data travelled through distributed computing systems. Calibration, trigger systems, detector physics, statistical analysis, software and theoretical interpretation each required knowledge nobody possessed end to end.

No physicist woke up that morning capable of rebuilding the Large Hadron Collider, recalibrating every detector, verifying every line of analysis software, reconstructing the electronics supply chain, re-deriving the theory and independently checking every collision event before breakfast.

And yet the result was not therefore rumor.

The knowledge was carried by a structure. Calibration procedures had histories. Software was validated. Analyses were reviewed internally. Different detector systems constrained one another. ATLAS

and CMS provided partially independent routes toward the same underlying phenomenon. Statistical conventions defined how much evidence justified using a word as consequential as *discovery*.

Underneath all the institutional machinery, the apparatus produced traces nobody could vote into existence.

A modern experiment is a society organized around an argument with reality.

Early in human history, much of what a community knew could plausibly be attached to identifiable people: ask her, she has seen the valley. As knowledge expanded, societies externalized memory into records, coordination into standards, expertise into specialized roles, perception into instruments and criticism into procedures.

Eventually we built institutions capable of producing knowledge no member could personally verify in full.

That is dangerous. A bad calibration can propagate. A shared assumption can synchronize thousands of competent people. Prestige can suppress criticism. Funding can steer a research program. A procedure can survive long enough to become ritual. A statistically beautiful answer can solve the wrong problem.

But without the machine, we lose the knowledge too.

There is no lone human replacement for CERN. There is no polymath who can personally substitute for modern medicine. There is no chief scientist carrying scientific civilization around in her head.

Civilization knows through composition.

Now go back to the compiler.

Sixteen Claudes, Again

Task locks. Git. CI. Progress files. Tests. A trusted reference compiler. Specialists. A harness that converts one global failure into many smaller investigations.

At the beginning of the chapter these looked like practical tricks for coordinating coding agents. They look different now.

One worker leaves a result another worker will trust. A passing test gives a claim standing. An oracle receives special authority for a bounded class of questions. A progress document becomes institutional memory. Specialization creates local expertise. The harness determines which evidence reaches which investigator.

They are primitive institutions.

A Mesopotamian accounting tablet is not `progress.md`. A telescope is not a compiler test. The Royal Society is not sixteen Claudes running in containers. CERN is not a multi-agent framework. Trying to line the nouns up perfectly would be silly.

The verbs are harder to ignore.

Preserve what happened so the next investigator does not begin from zero. Create standards so results can travel. Specialize. Give authority locally. Keep some investigators independent. Build

instruments when existing perception cannot answer the question. Construct procedures capable of embarrassing a persuasive theory. Let claims carry enough history that a later investigator can ask where they came from. Allow several explanations to survive long enough to become meaningfully different. Remember failures. Notice anomalies. Sometimes discover that the instrument was wrong, sometimes that the theory was wrong, and sometimes that the procedure everyone trusted is itself the thing that needs to change.

By now the agent architecture has acquired persistent records, standards, instruments, specialization, local authority, independent lineages and procedures for criticism. More strangely, it has acquired the possibility that the whole arrangement can know something none of its members can know alone.

I thought I was designing a society of agents.

Humanity had already spent centuries building a society of fallible knowers.

We call it **science**.

System 3 is science.

Not science as a pile of papers, or as "give the agent access to arXiv," or even as the familiar classroom sequence:

```
Question → Hypothesis → Experiment → Conclusion
```

Useful, but much too small.

I mean science as a civilization-scale cognitive technology: laboratories, instruments, notebooks, mathematics, journals, arguments, standards, archives, statistics, specialists, engineers, technicians, rival programs, reputation, criticism, replication, negative results, anomalies, and the occasional researcher who spends six months developing an elegant theory before discovering that the cable was loose.

Historical science is messy. It contains hierarchy, prestige, fashion, fraud, publication bias, career incentives, bureaucracy and communities capable of becoming remarkably sophisticated about the wrong thing. That is part of why it is a useful model for a system built from fallible agents rather than imaginary perfect reasoners.

What matters is not that science abolished error. Observations can outlive observers. Instruments extend perception. Expertise becomes local. Claims travel through trust chains. Critics can attack conclusions they did not produce. Rival programs can survive long enough to disagree meaningfully. And through the machinery there remain routes—imperfect, delayed, expensive and sometimes politically obstructed—through which reality can still make the institution uncomfortable.

That is the architecture I want from System 3: not an omniscient model, but a society of fallible minds that can remember without turning memory into scripture, trust without making authority universal, specialize without losing all connection between specialties, and disagree without putting everybody in the same meeting until the group reaches consensus from exhaustion.

A society that can build a new instrument when the old one cannot see what matters, discover that its trusted instrument was the thing that failed, and eventually change its own institutions when they stop earning their authority.

Science did not solve these problems. It built machinery for continuing to have them productively.

Apparently we are porting it.

Chapter 6: Pattern Language

When Knowledge Becomes Software

This book kept forgetting how to write itself.

That sounds more mystical than it was. I would work on a chapter with an agent, reject a certain kind of edit, explain why I rejected it, and eventually get something better. A few days later we would start another chapter and the same failure would return. The prose became cleaner in exactly the wrong way. Wandering sentences disappeared. Strange jokes were replaced by respectable ones. Arguments broke into tiny paragraphs that looked dramatic from across the room and exhausted me when I actually read them.

So I would say things like:

> Don't kill the wandering.
>
> Don't turn every idea into a slogan.
>
> Preserve the weird joke if it is carrying the argument.
>
> "More polished" is not automatically "more mine."

The agent would improve. Then the context would end.

We were reenacting, on a ridiculous scale, the problem Chapter 5 had just spent several thousand years describing. A society can know something none of its members knows alone. Fine. But if the society survives, another question appears:

How does what it learned yesterday change what it does tomorrow?

The obvious answer is memory. Save the conversation. Increase the context window. Keep a notebook. Put every decision into a database. That helps. It is not enough.

A transcript remembers what happened. An institution has to remember **what was worth learning from what happened**.

Suppose I save the instruction "use longer paragraphs." That is a memory of one correction. It is also a future disaster waiting politely in Markdown. The lesson was never that long paragraphs are good. The failure was a particular editing process compressing exploratory prose into a rhythm that felt machine-produced. Sometimes the cure was a longer paragraph. Sometimes a shorter sentence. Sometimes the correct edit was to stop editing.

What the next agent needed was not merely the instruction. It needed enough of the **reason, evidence, boundary conditions and failure history** to know when the instruction deserved authority.

That is closer to culture than memory.

Chapter 5 ended with the claim that **System 3 is science**: not science as a pile of papers, but as a civilization-scale arrangement that lets fallible minds accumulate knowledge while preserving routes through which reality can object.

Chapter 5 named the solution. This chapter opens the machine.

Science did not arrive with one clean design. Its philosophy is largely the history of people discovering failure modes in knowledge itself: theories that could explain anything, experiments that implicated several assumptions at once, paradigms that made research possible and then made alternatives hard to see, communities that confused consensus with independence, methods that became rituals, and institutions whose allocation of attention helped determine what could become known.

If System 3 is going to borrow from science, those arguments are not decorative philosophy. They are design reviews written a few decades or centuries early.

So the problem here is not simply how agents remember. It is:

How should useful experience become reusable behavior without turning yesterday's success into scripture?

That question has become practical because knowledge itself is starting to become a software artifact.

Three Ways to Tell a Computer What You Know

For most of computing history, if you knew how a process should work, you translated that knowledge into code.

A refund under €50 can be approved automatically. A payment over some threshold needs another check. A production deployment requires a test. A user without permission cannot read this table. Human knowledge becomes `if`, `else`, functions, schemas, state machines and permissions.

Andrej Karpathy calls this familiar world **Software 1.0**: humans write the behavior directly.

Machine learning changed the contract. Suppose I cannot state the rules that distinguish a fraudulent transaction from an unusual but legitimate one. I can give you examples. We choose data, a model and an objective, and optimization pushes useful behavior into weights. That is **Software 2.0**.

It gave us capabilities that explicit rules could never have scaled to, but much of the learned behavior disappeared from inspectable code. The fraud model "knows" things no engineer wrote down. We can evaluate it, probe it and retrain it, but there is no `fraud_rules.py` containing the organization's accumulated understanding of fraud.

Large language models created a strange third possibility. In his 2025 talk *Software Is Changing (Again)*, Karpathy called it **Software 3.0**: programs written substantially in natural language and interpreted by a model rather than a conventional compiler.

A model can read something like:

> Review this experiment. Check instrumentation changes before inventing a causal story. If click metrics rise while orders stay flat, inspect position and price shifts before celebrating.

There is no deterministic function there. There is operational knowledge. A competent human can interpret it. Now a sufficiently capable model can too.

The change goes beyond "prompts are code." **Knowledge itself can become versionable, composable and executable.**

An organization can write down a procedure, examples, scripts, counterexamples, diagnostic questions, evidence, tool instructions and boundaries. The model supplies enough interpretation that every clause does not have to become brittle symbolic logic before it can affect behavior.

The model begins to look less like the knowledge base and more like an **interpreter for knowledge artifacts**.

Knowledge engineering is back. It is carrying Markdown.

Knowledge Engineering Comes Back Wearing Markdown

The old dream of knowledge engineering was reasonable. Find experts. Extract what they know. Put it in a knowledge base. Let software reason with it.

The problem was that expertise is offensively reluctant to become a clean rule set.

Ask an experienced engineer how to diagnose a production problem and the answer is rarely:

```
IF latency > 300ms THEN database
```

It sounds more like:

> Start with the dependency graph, unless the spike began exactly at deployment. If only one market is affected, check the traffic split before touching the database. The cache metric lies under failover, so ignore it when this alarm is red. And if the problem started after the Tuesday migration, ask Sam because there is a thing with the old serializer that nobody wrote down properly.

Traditional expert systems struggled because converting that practice into formal logic was expensive and brittle. Machine learning offered an escape: stop asking experts to explain themselves and learn patterns from data.

LLMs change the trade again. They can interpret prose, examples, scripts, diagrams and partially structured instructions. Expertise still has to be captured, but it no longer has to become perfect logic before the computer can use it.

Now the hard questions are different. Whose practice gets written down? Which version applies here? What happens when two experts disagree? How does a lesson lose standing? When does a local workaround become a global rule? Which knowledge should enter the working context now, and which should remain in the archive?

Those are no longer edge cases. They are the engineering surface.

Agent systems were already moving in this direction. Repository instructions, `AGENTS.md`, skills, tool descriptions and context engineering all treat useful procedural knowledge as something external to the model but available to it at runtime. A skill can survive the session that produced it. Increasingly the worker can change while the operating knowledge remains.

That matters because frontier models already know Python, statistics and enormous amounts of public technical culture. What they do not automatically know is why *your* company refuses to deploy on Friday, which metric has been misleading everyone since 2023, why an elegant architecture in the wiki was abandoned, or why Alberto should never again be asked to investigate penguins. Organizations run on this layer of weirdness.

Some belongs in code. Some belongs in data, tools and evaluators. A surprising amount is **situated procedural knowledge**: what to check first, which shortcut is dangerous, which source has standing, when the normal process does not apply, and what "good" means here rather than on a generic benchmark.

For most of history, people acquired this by hanging around people who had already been injured by the relevant mistakes.

Now more of it can become software. Which is exciting right up until we create prompt spaghetti at civilizational scale.

A saved instruction is too small a unit.

From Skill to Pattern

Christopher Alexander's *A Pattern Language* was about towns, buildings and recurring design problems. A pattern was not simply a commandment. It named a situation, the forces that made it difficult, a response that had repeatedly worked and the consequences of using that response.

That abstraction fits agent knowledge almost suspiciously well.

Consider:

> Never use regex on nested syntax.

It has the reassuring shape of wisdom and the inconvenient property of being false.

Regex is perfectly good for many small extraction tasks. A parser may be absurd overhead for a five-line format. The useful lesson is that recursive structure creates characteristic failure modes for flat pattern matching; those failures become harder to see as syntax grows; and beyond some point a parser becomes cheaper than maintaining an increasingly heroic regular expression written at 2 a.m.

That is closer to a pattern.

A saved instruction remembers **what somebody said**. A pattern tries to remember **what kept happening**.

The book has already accumulated patterns whether we called them that or not.

Immutable Harness: when autonomy makes the solution fluid, keep the evaluation boundary harder to change than the thing being evaluated.

Independent Evaluators: when one judge can be gamed, create genuinely different sources of pressure rather than five copies of the same opinion.

Strategic Constraint: when an easy path keeps absorbing the search, remove it long enough to expose another part of the possibility space.

The useful part is not the slogan. Each pattern contains a recurring situation, a tension and a reason.

But Chapter 5 made us suspicious of inherited procedure. Civilizations do not merely accumulate good practices. They accumulate ritual, prestige, local workarounds and procedures that have outlived the world that justified them.

If Pattern Language is going to become cultural memory for agents, a pattern needs to know more about itself. This is where philosophy of science finally earns its API.

Popper: A Pattern Needs a Way to Lose

Suppose the editing agent concludes:

> Use longer paragraphs in this book.

Karl Popper would immediately ask the rude question: what could happen that would make us stop believing this lesson?

A useful theory exposes itself to observations that could have gone differently. If every outcome can be narrated as success, the theory has arranged the game so it cannot lose.

Persistent agent knowledge needs the same property. A pattern should retain an **exposure path**: a test, observation, user reaction, proof obligation or downstream consequence capable of weakening it. "Use longer paragraphs" might lose standing if reader tests show comprehension falling, if another chapter becomes monotonous, or if the original failure disappears after the editing process changes.

The pattern should remember not only what worked, but **how the world could show that the lesson stopped working**.

Then Pierre Duhem and W. V. O. Quine ruin the simplicity.

Evidence rarely confronts one isolated belief. A failed experiment implicates a bundle: hypothesis, instrument, data, analysis, background assumptions. A failed pattern has the same problem.

Did the advice stop working? Was it retrieved in the wrong situation? Did the model change? Did the evaluator drift? Was the original success caused by something else? Did "longer paragraphs" merely correlate with the real change—more natural argumentative rhythm—without causing it?

Failure tells us that some part of the package deserves suspicion. It does not highlight the guilty line in yellow.

So reusable knowledge needs **archaeology**.

Where did this pattern come from? Which failures produced it? Which model and tools were involved? Which alternatives were tried? What evidence earned the lesson its standing? What assumptions were present?

A factual claim without provenance becomes rumor. A reusable practice without provenance becomes tradition.

Tradition is not automatically bad. It is simply difficult to debug.

Kuhn, Lakatos and Laudan: Defaults Need Rivals

Keeping every pattern permanently open for debate would be a beautiful philosophy and a terrible operating system.

Thomas Kuhn's most useful lesson here is not the phrase *paradigm shift* but the role of **normal science**. Productive communities need enough stability that they do not reopen their deepest assumptions every morning. A framework tells researchers what puzzles are worth solving, which instruments are legitimate and what kinds of answers count.

An agent culture needs the same economy. If a deployment procedure has survived hundreds of releases, the system should not rediscover it from first principles every Tuesday merely to prove that it remains intellectually alive.

Some decisions earn the right to become boring. The danger is that boring assumptions become invisible assumptions.

Anomalies need memory too. The release pattern works except in this market. The evaluator tracks human judgment except on this kind of creative task. The ranking heuristic works except every holiday season. The workflow works except that the critic now spends most of its time inventing reasons why the builder was right.

One anomaly is usually noise. Ten may still be noise. Eventually the exception list begins to look like the theory.

Imre Lakatos makes the problem harder in a useful way. We should often preserve **research programs**, not merely isolated ideas. A program has a history, a relatively stable core, auxiliary assumptions and a trajectory. One approach may currently be weaker but improving. Another may be winning mainly by adding patches around every failure.

Larry Laudan sharpens the practical consequence: **acceptance and pursuit are different decisions**.

I can believe method A is our best current default while still believing method B deserves another experiment.

Those are two questions:

- What should guide action now?
- Where is another unit of investigation most valuable?

The distinction matters the moment culture becomes executable. The current winner gets retrieved more often. Because it gets used more often, it accumulates more successful cases. Those cases raise confidence. Higher confidence makes it even more likely to be retrieved.

The alternative receives less traffic and therefore less evidence. Eventually the system develops an impressive empirical record proving the thing it stopped comparing against.

No committee had to ban the alternative. The retrieval policy did it.

Hull and Kitcher: Who Gets the GPUs?

There is an overly clean way to draw a society of agents. Every agent is a box. Every box has an arrow. Everybody gets a turn to think.

Real institutions are not like that because **attention has a budget**.

Human science has telescope time, laboratory space, grants, journals, careers and prestige. AI research has datasets, deployment traffic, human reviewers, API quotas, clusters and GPUs. A theory's ability to generate evidence depends partly on whether the institution gives somebody the resources to investigate it.

This is not merely politics happening around epistemology. It changes the epistemic landscape itself.

Imagine research program A has ten thousand GPU-hours and program B has ten.

A can run ablations, train variants, investigate anomalies and produce beautiful graphs. B can produce a thoughtful paragraph about why it deserves more compute.

Six months later A has more evidence. Of course it does.

The evidence may be real. A may genuinely be better. But the institution has also helped create the asymmetry it later treats as evidence for further allocation.

David Hull and Philip Kitcher approached science partly through the division of cognitive labor, incentives, credit and the fact that investigators do not all pursue the same thing for the same reasons. Researchers cooperate because they need one another's results and compete because priority, reputation, jobs and resources are scarce.

An agent society will have analogues of these structures whether or not we give them sociological names.

The scheduler is partly a funding agency. The memory system is partly an archive. The evaluator is partly a journal gate. The retrieval layer is partly a curriculum. The permission system decides who may touch which instrument. The compute allocator decides which hypotheses get enough opportunity to become well-tested hypotheses.

And the organization can remember its incentives just as effectively as it remembers its wisdom. A local objective becomes a local pattern. The pattern gets copied because the team is successful. Future agents inherit it without seeing the original trade-off. Eventually "this helped one group hit its metric" becomes "this is how good work is done here." That is culture too.

Local alignment does not compose automatically. Neither does local truth.

If we build persistent agent societies without thinking about this layer, we will not escape institutional power. We will automate it and give it better dashboards.

Longino: The Community Is Part of the Instrument

Resources are only one reason a community matters. Even generously funded investigators can share the same blind spot.

Give five agents the same model family, the same system prompt, the same search results and the same dominant explanation, and you have not created five perspectives. You have created a very expensive echo with parallel API calls.

Helen Longino's social epistemology is useful because background assumptions affect what investigators notice, which questions appear natural and which evidence looks relevant. Criticism becomes more informative when it comes from participants positioned differently enough to expose assumptions the dominant group treats as obvious.

For an agent system, useful difference may come from a separate dataset, another tool, an isolated context, a different model, an external user, a domain expert, or a team operating under different incentives.

That is stronger than theatrical personas:

> Agent 1, be optimistic.
>
> Agent 2, be skeptical.
>
> Agent 3, be a pirate.

The pirate may be entertaining. He probably still read the same PDF.

A pattern should therefore carry some trace of **position**: who learned it, from which class of tasks, with which tools, model, evidence and incentives. A ranking practice learned in Germany, a fraud rule learned in Brazil, a compiler workaround discovered under one toolchain and a clinical protocol developed in one hospital can all be excellent without acquiring universal authority merely because they share a database.

This is another reason Pattern Language should not become one giant company constitution. Culture should be **locally authoritative** where appropriate.

Trust is local. Apparently memory should be too.

What a Pattern Should Know About Itself

By this point a mature pattern looks richer than a prompt-library entry.

Something like:

Field	What it preserves
Situation	Where this pattern is supposed to apply.
Forces	Why the problem is difficult and which trade-offs recur.
Response	The reusable behavior, procedure or design move.

Field	What it preserves
Evidence	What experience earned the pattern its current standing.
Provenance	Who or what produced that evidence, using which tools and assumptions.
Boundary conditions	Where the pattern is known not to generalize.
Anomalies	Evidence that does not fit cleanly and should not disappear.
Competing patterns	Alternatives worth keeping alive.
Exposure path	What future observation could weaken or overturn it.
Confidence	How strongly the pattern should guide action now.
Pursuit value	Whether another experiment deserves resources even when this pattern currently wins.
Position / incentives	Which organizational perspective produced the lesson and what pressures shaped it.
Version / environment	Which model, system, market, toolchain or period the evidence came from.

I do not mean this as a universal schema. Turning the schema itself into scripture would be an efficient way to miss the chapter.

The point is the difference between a command and institutional knowledge.

The command says:

> Do this.

The pattern says something closer to:

> We keep doing this because these forces recur; this response has usually worked; this evidence earned our trust; these are the places it fails; these alternatives remain alive; and this is what would make us reconsider it.

That is knowledge with some of its history still attached. And because an LLM can interpret the artifact at runtime, it can change behavior without retraining the model.

That is what I mean by **knowledge becoming software**. Not because prose has literally become Python, but because knowledge can now be versioned, scoped, retrieved, executed, challenged, rolled back and eventually modified by the same kind of agents that use it.

Knowledge has acquired a runtime.

Knowing Something Is Not Knowing When to Remember It

Now imagine ten excellent patterns.

Easy.

Imagine ten thousand.

The agent cannot read the organization before every action. Even if a context window technically fits everything, loading every policy, postmortem, experiment, preference and historical argument into every task would solve forgetting by making thinking impossible.

So a persistent institution has two memory problems:

What should survive?

and

What should become salient now?

Long-running agents accumulate messages, files, tool outputs, memories and artifacts faster than useful attention can scale. Context has to be selected, compacted and reconstructed. Retrieval becomes part of cognition.

A culture may contain exactly the right lesson and still fail because that lesson does not arrive when it matters. Every large company has written a postmortem whose recommendation is rediscovered three incidents later by different people using the phrase "interesting, we should probably document this."

Agent systems can fail more elegantly. They can store the lesson perfectly, embed it beautifully and retrieve a more popular but irrelevant one. Or retrieve the right pattern without its boundary condition. Or retrieve ten conflicting patterns and allow whichever appears latest in context to win by textual gravity.

Bad storage forgets by deletion. Bad retrieval forgets by attention.

Once retrieval determines which inherited knowledge enters a decision, retrieval itself becomes an epistemic procedure. It needs evaluation. Does it repeatedly surface stale rules? Does it suppress alternatives? Does it confuse popularity with relevance? Does it preserve the result and discard the reason?

The librarian is no longer merely finding books. The librarian is shaping thought.

Feyerabend: The Librarian Is Also a Hypothesis

At some point every successful institution becomes tempted to trust its own method. Yesterday's useful workflow becomes today's best practice and tomorrow's mandatory ritual.

Paul Feyerabend is usually remembered for "anything goes," which is a good way to remember the slogan and forget the warning. Successful inquiry has often violated methodological rules somebody wanted to make universal.

Agent systems can turn a method into ritual very quickly. Suppose `Research → Plan → Build → Critic → Revise` works beautifully. We run it ten thousand times. It becomes the company standard. Soon every task enters the same ceremony, including tasks where research anchors the builder, criticism arrives too late, or a crude prototype would have answered the important question in five minutes.

The method itself has to become available for criticism.

An evaluator is a procedure. A browser is an instrument. Retrieval is a procedure for selecting evidence. A benchmark is a measurement system with a distribution and failure modes. A proof checker is extraordinarily authoritative inside its formal domain and completely useless for deciding whether the theorem matters. A simulated user is cheap perspective-taking and not a user.

The institution should be able to learn that its usual way of checking a claim is itself the thing that stopped working.

The library needs criticism. So does the librarian.

Bayesian confidence can live inside this architecture, but confidence is not contact. `0.91` does not tell us whether the prior was sensible, whether the evidence was independent, whether an alternative was ever investigated, or whether everybody is confidently reading the same broken measurement.

Consensus is not contact either. Twelve agents sharing one bad source can agree beautifully.

The bridge still has the right to fall. The proof still has the right not to check. The deployment can crash. The customer can dislike the supposedly improved page while every simulated evaluator applauds.

Reality retains the right to be rude.

The purpose of culture is to let knowledge travel across time without replacing the world with memory of the world.

Culture Can Become a Prison

If forgetting were the only danger, the design would be easy: remember everything. Unfortunately organizations also suffer from remembering too well.

Every process exists because it helped at some point, or because somebody once thought it would. Every release checklist box has a story, or used to. Every architecture principle was attached to a failure someone cared enough to prevent. Then the environment changes and the procedure remains.

Eventually someone asks why the process exists and receives the most dangerous explanation in organizational life:

> That's how we do it.

Executable culture closes the loop between memory and behavior. A pattern succeeds. Its confidence rises. More agents retrieve it. Alternatives get less traffic. The dominant pattern accumulates more evidence because it is dominant.

Soon the institution has an impressive record proving the thing it stopped comparing against.

Patterns therefore need anomaly memory, versioning, local scope and competing alternatives. Some should decay. Some should expire when the environment changes. Some should be deliberately challenged after they become too comfortable. Occasionally a capable agent should be allowed to ignore the manual precisely so the institution can discover whether the manual still deserves authority.

The culture needs to distinguish:

> We tried twelve alternatives and this kept winning.

from:

> Nobody has tried another way since 2025.

Those sentences can produce identical dashboards and very different knowledge.

Culture needs inheritance and rebellion.

The Skill That Writes Itself

A serious organization may have thousands of agents, tools, experiments, workflows and recurring failures. Useful experience appears continuously. Some lessons deserve to become local skills. Some should become organization-wide patterns. Some contradict old knowledge. Some work only for one model version. Some are artifacts of a broken evaluator. Some are excellent and will be obsolete in three months.

Humans could curate all of this manually.

Congratulations. We have created middle management again.

The more interesting loop is computational:

experience occurs; an agent notices recurrence; it proposes a reusable pattern; another process checks whether the pattern actually helps; held-out cases test whether it generalized; provenance and failures remain attached; the pattern earns some level of authority; future agents retrieve it when relevant; new failures can weaken or revise it.

There are many places to cheat. The agent proposing the skill can design an evaluator it knows how to satisfy. Repeated use can masquerade as independent evidence. A pattern can improve a benchmark while making maintenance worse. Retrieval can starve competing practices before they accumulate enough evidence to challenge the incumbent. A central curator can quietly turn local taste into universal law.

So the curator needs a track record too. The mechanism for improving culture is itself part of the culture.

And now we cross an important line.

Experience becomes knowledge. Knowledge becomes executable. Executable knowledge changes future behavior. Future behavior produces new experience.

The model weights did not move. The institution learned anyway.

That is a learning loop **outside the weights**.

Chapter 5 moved the unit of intelligence from the individual agent toward the institution. Here learning begins to move in the same direction. Intelligence is partly in the model, but also in tools, evaluators, context construction, patterns, retrieval and the procedures deciding which of those artifacts deserve authority.

Once those things are software, one question becomes difficult to avoid. Why should humans be the only ones allowed to edit them?

That is where recursive self-improvement begins.

Chapter 7: Recursive Self-Improvement

When Science Turns Inward

Chapter 6 ended by making a strange kind of software possible.

Memory, patterns, evaluators, retrieval, tools and workflows could survive individual agents and change what later agents did. An institution could learn without changing the underlying model weights.

Once the machinery of learning becomes software, why should humans be the only ones allowed to edit it?

That is where recursive self-improvement stops being a science-fiction phrase and becomes an engineering problem.

Computing has an old image for this. In 1962, Tim Hart and Mike Levin described a Lisp compiler written in Lisp that could compile its own source. The tool could participate in producing the next version of the tool. Compiler people call this **self-hosting**. (Hart & Levin / LISP 1.5 archive)

Chapter 5 began with agents building a compiler. The compiler returns here for a different reason. A self-hosting compiler contains the whole warning in miniature:

self-reference is not self-improvement.

A compiler can compile a worse compiler. A research system can redesign itself into a slower research system. A learned optimizer can become excellent on yesterday's tasks and brittle tomorrow. The ability to modify the machinery that produces you tells us a boundary has become permeable. It does not tell us which changes deserve to survive.

Three years after Hart and Levin's memo, I. J. Good gave that permeability a much more dramatic consequence. In 1965 he imagined an **ultraintelligent machine** better than any human at intellectual activity. Machine design is itself an intellectual activity, he observed. A sufficiently capable machine might therefore design a better machine, which could design a better one again. The phrase that survived was **intelligence explosion**. (Good)

Good's argument is only a few lines long. The word *better* is where the trouble begins.

The practical history of self-improvement did not proceed as one machine repeatedly rewriting its own source code. It arrived through reinforcement learning, self-play, curiosity, continual learning,

inverse reinforcement learning, evolutionary search, meta-learning, world models, learned evaluators and finally general models capable of modifying the software around themselves.

The history is messy, but one pattern keeps reappearing:

we kept discovering another job the teacher was doing.

First the teacher stopped choosing the action. Then she stopped supplying every useful experience. She stopped providing every opponent and every curriculum. The reward itself became uncertain. The learner began preserving a life rather than completing one training run. Learning rules, optimizers and architectures entered the search space. World models generated imagined experience. Open-ended systems generated problems. Human judgment became a learned instrument. General models eventually became capable of editing the tools, prompts, memories, workflows and research procedures shaping their own future behavior.

Every move inward bought autonomy. Every move inward also exposed another hidden decision the human had been making.

That second history is the one I care about.

The Teacher Moves Into the Walls

Modern reinforcement learning begins with an unusually generous assumption disguised as a minimalist one.

An agent sees a state, takes an action, receives a reward and finds itself somewhere new. Nobody tells it which action was correct. The learner has to discover behavior through consequences.

Richard Sutton's 1988 work on temporal-difference learning and Christopher Watkins's Q-learning helped give this setup its modern form: learn from experience, update estimates of future value and eventually discover useful policies without a human labeling every move. (Sutton; Watkins & Dayan)

The human no longer specifies the path. The human specifies the **score**.

That was one of the great bargains of learning systems. A machine can discover strategies nobody wrote down because the designer moved upward from choosing actions to defining what outcomes count.

It also hides a remarkable amount of human labor inside the environment. Who chose the state representation? Which actions exist? Why is one event worth +1 and another -1? When does the episode end? Which failures are recoverable? Who arranged the world so useful behavior can be discovered before the sun burns out?

The reinforcement learner looks autonomous because the teacher moved into the walls.

Backgammon made the bargain spectacular. In the early 1990s, Gerald Tesauro's TD-Gammon learned by playing enormous numbers of games against itself and updating its predictions from the outcomes. Later versions combined learned evaluation with shallow search and reached extraordinary strength. (Tesauro)

Self-play removed another piece of external instruction: the opponent could come from the learner itself. Yesterday's agent generated today's training data.

But the board did not move. The legal moves did not move. The win condition did not move.

Backgammon could generate an enormous curriculum precisely because somebody outside the loop had already settled what winning meant.

Self-improvement was easy to recognize because the world came with a scoreboard nailed to it. Real life is less considerate.

The Learner Chooses What to Learn

Even a perfect reward is useless if the learner never reaches it.

Atari's *Montezuma's Revenge* became a museum exhibit for this problem. Useful reward may sit at the end of long sequences of exploration, while a naïve learner has little reason to treat the unknown as valuable enough to visit.

Humans and other animals do something stranger. Children open drawers nobody asked them to open, stack blocks no employer requested and spend twenty minutes discovering that the cardboard box is more interesting than the toy.

Researchers tried to move some of that exploration pressure inside the learner.

In 1991, Jürgen Schmidhuber described curious model-building controllers that could receive reinforcement for improving their knowledge of the environment. (Schmidhuber) Later work on intrinsic motivation emphasized **learning progress** rather than mere novelty: seek places where ignorance is becoming competence. (Oudeyer, Kaplan & Hafner) Deep reinforcement learning made the idea visible again through methods such as curiosity-driven prediction error and Random Network Distillation. (Pathak et al.; Burda et al.)

The learner could manufacture some of its own reasons to look around.

Then optimization did what optimization does. It took the instruction literally.

If surprise itself is rewarding, an uncontrollable noisy television can remain fascinating forever. The system is not confused. We are. We said *surprise* and quietly meant *surprise from which useful structure can be learned.*

Curiosity removed one teacher job and uncovered another:

what kind of difference deserves to count as interesting?

That question immediately reaches into representation and embodiment.

Pathak's curiosity work did not simply reward prediction error over raw pixels. It learned features related to the agent's own action transitions, partly to avoid paying for unpredictable but irrelevant visual changes. The representation changes what counts as novel.

Robotics made the same point physically. Ruzena Bajcsy's work on **active perception** emphasized that an intelligent system moves sensors, changes viewpoint and acts in order to perceive. Rodney Brooks pushed against detached symbolic intelligence in favor of systems tightly coupled to the world through perception and action. (Bajcsy; Brooks)

A learner's **body** determines part of its curriculum. A tactile robot can discover things a camera-only robot cannot. A software agent with a browser, shell, compiler and simulator has a different epistemic body from a chatbot restricted to text. Permissions matter too. Give an agent read-only access and one set of experiments is possible. Give it code execution, network access and a credit card and we have created a different organism and, potentially, a different incident report.

Tools and representations do not merely help the learner solve a fixed problem. They help determine what can become **learnable**. The learner is beginning to shape the conditions under which learning occurs.

The Learner Has to Remain Itself

There is another embarrassment in the standard training story.

The learner finishes.

Train on a task. Evaluate. Publish the number. If another task arrives, train again.

Organisms do not get to do this. A child who learns multiplication cannot delete language to make room. A physician learns a new treatment while retaining anatomy. A programmer picks up Rust without waking the next morning unable to read Python.

Neural networks have historically struggled with this. Michael McCloskey and Neal Cohen's 1989 analysis made catastrophic interference stark: new sequential learning can destroy previously acquired knowledge. Decades later, methods such as Elastic Weight Consolidation were still explicitly trying to preserve important older knowledge while learning something new. (McCloskey & Cohen; Kirkpatrick et al.)

Now "better" becomes harder to rank.

Version B scores 95 on today's task and A scores 85. Easy. But B forgot three older skills. Better? B learns new tasks twice as quickly but erases rare knowledge needed once a year. Better? B preserves everything and becomes so rigid that it cannot adapt to a changed world. Better?

Continual learning exposes the stability–plasticity tension: preserve enough to remain yourself; change enough to remain useful.

Chapter 6 found the same problem at the level of culture. A society that forgets every old lesson begins from zero. A society that remembers every old lesson as law becomes a museum.

Improvement across a lifetime is not improvement on the latest test.

It is accumulation without paralysis.

That matters once agents live for months or years. A system that continuously rewrites itself while destroying the right parts of its own history is not accumulating a life — it is repeatedly replacing itself and calling the replacements progress.

Sometimes the Environment Improves Back

Self-play contains a second engine of learning that is easy to miss.

Sometimes improvement is not voluntary.

Evolutionary biology's **Red Queen hypothesis** describes organisms adapting inside environments that contain other adapting organisms. Standing still can mean falling behind because the effective environment moves. (Van Valen / Santa Fe Institute retrospective)

W. Daniel Hillis used the computational version in 1990 while evolving sorting networks. Co-evolving "parasites" served as difficult test cases; as candidate sorting networks improved, the tests could become harder too. (Hillis) Decades later, AlphaZero made the moving opponent spectacular again: self-play generated a curriculum that grew with the learner inside fixed game rules. (Silver et al.)

Yesterday's learner can generate tomorrow's difficulty.

Curiosity says: seek somewhere informative. Competition says: something informative—or dangerous—is coming whether you seek it or not.

A security system cannot preserve yesterday's competence if attackers change strategy. A market participant can become relatively worse without becoming absolutely less capable if everyone around it improves faster.

But competition gives no guarantee that the direction of adaptation is good. An arms race can produce better claws and thicker armor without producing welfare. Two systems can spend increasing amounts of intelligence outmaneuvering one another while creating little value outside the contest.

Selection pressure produces adaptation. It does not supply purpose.

Games hide that problem because the constitution is fixed. Chess never asks whether checkmate remains desirable after move forty-three.

Competition removes another teacher from the curriculum. It does not remove the teacher from the **purpose of the curriculum**.

Maybe the Reward Was the Problem

Around the same time researchers were getting better at optimizing rewards, another line of work asked a more unsettling question:

What if we do not actually know the reward?

Andrew Ng and Stuart Russell's 2000 paper on **inverse reinforcement learning** reversed the usual setup. Instead of receiving a reward function and learning a policy, the learner observes behavior and asks which reward functions could make that behavior look optimal. (Ng & Russell)

Ordinary reinforcement learning says:

> Here is what matters. Learn how to get it.

Inverse reinforcement learning says:

> I can show you what someone does. Infer what might matter to them.

The objective itself becomes an object of inference.

Immediately, ambiguity appears. Many reward functions can explain the same behavior. A person taking one route to work may care about time, comfort, safety, tolls, habit, dropping children at school or avoiding one particular intersection where somebody once scratched the car.

Later work made the uncertainty more explicit. Cooperative Inverse Reinforcement Learning models a human and robot cooperating while the robot remains uncertain about the human's reward. Inverse Reward Design treats even a reward function written by a designer as **evidence** about what the designer wanted in the training situations, rather than sacred truth guaranteed to generalize everywhere. (Hadfield-Menell et al.; Hadfield-Menell et al.)

Preference-based reinforcement learning provided a practical cousin: ask humans which of two trajectory segments looks better and learn a reward model from those comparisons. (Christiano et al.) That lineage later became central to reinforcement learning from human feedback for language models.

Another teacher job had become learnable, and we immediately discovered that humans are not reward functions walking around in shoes. They are inconsistent, constrained, strategic, tired, socially influenced and sometimes unsure what they want until they see an option. Sometimes they click the article because they hate it.

The problem is no longer only how to improve toward an objective. It is how to remain uncertain about what the objective is.

Chapter 9 will live inside that problem. For now, keep the historical move in view: even the score has begun to move inward.

Learning to Learn

Once behavior can adapt, the next hand-authored component starts to look suspicious:

why are humans still designing the learner?

Meta-learning attacks this directly. In RL^2, a recurrent network was trained across distributions of reinforcement-learning tasks so that its recurrent dynamics effectively implemented a fast learned learning procedure inside a new task. MAML instead optimized parameters so that a small number of gradient steps could adapt effectively to new tasks. (Duan et al.; Finn, Abbeel & Levine)

Now learning speed itself becomes a capability. One system may perform best before adaptation. Another starts lower but becomes excellent after five examples. Which is better depends on whether the world sits still.

Meta-learning creates two timescales:

- improve behavior on the current task;
- improve the machinery that acquires behavior on future tasks.

The second is recognizably closer to recursive self-improvement.

But the teacher is still there. Somebody chose the task distribution on which the learner learned to learn.

Meta-learning does not eliminate the curriculum. It moves the curriculum outward.

Learned optimizers and Neural Architecture Search pushed the editable boundary further. Researchers trained models to generate parameter-update rules or neural architectures, using performance on selected problems as the evaluator. (Andrychowicz et al.; Bello et al.; Zoph & Le)

At first the machine learned the answer. Then it learned a policy. Then it learned how to learn. Now it could search over pieces of the machinery that **does the learning**.

The human had moved from architect to judge.

The system had more freedom over means. The task distribution, search space and validation metric still sat outside the loop holding a clipboard.

The Old Dream Tries to Prove the Rewrite

The reinforcement-learning tradition was not the only route toward self-improvement.

I. J. Good's intelligence explosion was explicitly recursive from the beginning. In 2003, Jürgen Schmidhuber's **Gödel Machine** tried to formalize a harder question: under what conditions should a system rewrite itself? The proposed machine contains an axiomatic description of its own software, assumptions and utility. A proof searcher looks for a self-rewrite together with a proof that performing the rewrite is more useful than continuing to search. Only then does it change itself. (Schmidhuber)

It is a beautiful answer to a beautifully clean version of the problem:

prove the modification is worth making.

The catch is the definition of *worth*.

Usefulness has to be represented in the utility function. Relevant facts have to be available to the proof system. The advantage of the rewrite has to be provable inside the formal machinery.

A chess engine can live surprisingly close to that world.

A company cannot.

A scientist cannot prove in advance that an unexplored research program will matter. A lifelong learner cannot enumerate every future skill worth preserving. Human purposes do not arrive as an axiomatized utility function.

So two histories were approaching the same mountain from opposite sides. The explicit recursive-self-improvement tradition had self-reference and meta-level ambition, but no practical general system able to inspect and rewrite complicated software intelligently. Learning systems had increasingly powerful adaptive machinery, but humans usually kept the outer research process, evaluator and environment fixed.

Foundation models made those histories collide.

A general model can now read the code scaffolding its own behavior, propose a change, run the changed system, inspect the result and try again.

We do not have a proof that the rewrite is globally useful.

We have something much more ordinary.

An experiment.

Recursive self-improvement comes back to the framing we began with: science turns inward.

The Learner Dreams, and the Dream Can Be Wrong

Model-based reinforcement learning removes another dependence on the external teacher by reducing dependence on external experience.

Ha and Schmidhuber's *World Models* made the idea memorable: learn a compressed generative model of the environment, train a controller partly inside that generated "dream," then transfer behavior back to the real environment. (Ha & Schmidhuber) Later systems such as Dreamer pushed model-based learning much further.

Imagined experience is attractive because real experience is expensive. A robot can break only so many arms. A company can run only so many damaging experiments. A scientist may wait months for an observation.

But the epistemic debt has not vanished. It moved into the model.

A learner can become extremely competent inside a world that is slightly wrong. The strategy looks brilliant until gravity, customers or compiler behavior get a vote.

The world model is an instrument. The dream is not reality.

Simulation expands search. Contact with the world still decides which imagined regularities deserve trust.

Self-improvement can therefore make a system better at generating experience while also making it easier to **train inside its own misconception**.

That becomes especially dangerous once language-model agents use other language models as judges, simulators, users and critics. At sufficient scale, a society can become very good at agreeing with itself.

What If the Objective Is the Local Optimum?

A different line of work attacked the objective itself.

Joel Lehman and Kenneth Stanley's **novelty search** showed that objective-driven search can be deceptive: the obvious measure of progress may steer search toward dead ends, while the stepping stones required for a breakthrough initially look unrelated to the final goal. (Lehman & Stanley)

Reward every intermediate invention by how closely it resembles a Boeing 787 and feathers, bicycles, wind tunnels and propellers may look like terrible ideas for years.

The useful stepping stone need not be a miniature version of the destination.

Sometimes "better" means **more different**, at least temporarily.

That freedom has its own failure mode. Novelty for its own sake can generate forty-seven new ways to fall down a staircase without producing walking.

So the definition of progress expands again. We need achievement, diversity and stepping stones that the current evaluator does not yet know how to value. The scalar is beginning to crack.

Open-ended systems push this further by generating **problems** as well as solutions. POET co-evolves environments and agents, generating new challenges and transferring successful behaviors between them. XLand similarly uses large procedurally generated spaces of games and adaptive curricula. (Wang et al.; Open-Ended Learning Team)

Now the world defining competence is moving with the learner.

If the benchmark stays fixed, improvement is easy to plot: score goes up. If the environment evolves too, progress may mean breadth, adaptation speed, richer strategies, harder generated tasks or useful stepping stones for descendants.

There is no final fitness scoreboard on Earth on which mammals eventually beat bacteria 87.4 to 82.1.

Once the learner partly generates the curriculum, **the test cannot remain a passive spectator**.

The Ruler Has a Half-Life

Large language models rediscovered this problem at industrial scale.

A benchmark starts as a hard test. Researchers optimize against it. Models improve. The benchmark gets easier. Examples circulate. Failure modes become public. Synthetic data resembles the test. Eventually the evaluation can become more like a specification than a fresh measurement of generalization.

MMLU arrived in 2020 as a broad academic benchmark when frontier systems were far from saturating it. FrontierMath, LiveBench and Humanity's Last Exam later appeared partly because the frontier kept consuming old rulers and needed new ones with more headroom or greater freshness. (MMLU; FrontierMath; LiveBench; HLE)

The names tell their own story:

Massive Multitask Language Understanding.

BIG-Bench Hard.

FrontierMath.

Humanity's Last Exam.

At some point the naming committee will need reinforcement learning too.

The serious point is that benchmark creation has become part of capability research. We do not merely train systems against tests. We invent new tests because yesterday's tests stop telling us what we need to know.

What is the benchmark for being a good scientist? A theorem set? Replications? Novel molecules? Discoveries per GPU-hour?

What is the benchmark for a good autonomous organization? Profit? Customer value? Robustness? Ability to notice that the objective was wrong?

What is the benchmark for becoming a better learner? Performance now? Adaptation speed? Breadth? Memory? Transfer? Curiosity? Safety? Compute efficiency?

The closed benchmark keeps returning because we need something capable of saying **yes** or **no**. But as capability expands, the ruler measures a smaller slice of the thing.

The object being measured keeps growing new dimensions.

The Judge Becomes Software

Large language models also moved the old reward problem into the evaluator itself.

In 2022, InstructGPT used human demonstrations and rankings to train a reward model, then optimized the language model toward outputs humans preferred. (Ouyang et al.)

Human preference had become a learned instrument.

That scales judgment far beyond direct human labeling. It also creates a new proxy. A reward model can have stylistic biases. Humans can prefer confident errors. The model can generalize badly outside the feedback distribution. A sufficiently strong optimizer may discover outputs that score well under the learned judge for reasons nobody intended.

We solved part of the scaling problem by making the judge computational. Now the judge joins the attack surface.

That is where the modern engineering story of recursive self-improvement becomes possible—and dangerous.

The Learner Edits the School

By the 2020s, foundation models had become competent enough at code and tool use that the meta-level stopped being merely formal.

In 2023, **STOP—the Self-Taught Optimizer**—used an LLM-based improver that could itself become the object of improvement. The base model stayed fixed while the program determining how it was used changed. (STOP)

In 2025, the **Darwin Gödel Machine** turned agent implementation into an open-ended evolutionary object. Descendants modify the coding agent, are evaluated on coding tasks and enter an archive from which later descendants can be generated. The archive prevents the current champion from monopolizing ancestry and preserves possible stepping stones. (DGM)

In 2026, Andrej Karpathy's `autoresearch` repository made the engineering version look almost comically small: give an agent a compact training setup, a fixed experimental budget and an editable `train.py`; let it propose changes, run experiments, inspect the validation metric, keep improvements and discard regressions. (autoresearch)

Automated hyperparameter tuning is old. The new part is that a general model can read the research codebase, form an idea in language, express it as code, run the intervention, interpret what happened and decide what to try next.

Machine learning is being used to do machine-learning research.

The tool is becoming self-hosting in the broader sense that matters here.

A compiler compiles a compiler. A learning system searches for a learning system. A research agent researches the process by which research agents research.

That last sentence sounds like parody until you notice the leverage. Improve one experiment and you improve one experiment. Improve the research loop and every later experiment may change.

Meta's **HyperAgents** pushes the same recursion another level outward by placing task and meta-level modification machinery inside one editable program. (HyperAgents)

The conceptual difference is small enough to sound ridiculous in English:

> I can change how I solve the problem.

becomes:

> I can change how I decide **how to change how I solve the problem**.

Chapter 6 made patterns, memory, evaluators, tools, workflows and organizational rules into executable culture. Now more of that culture is editable.

The scientific institution has enough general-purpose software competence to **modify parts of the laboratory while the experiment is still running**.

The Harness Becomes an Experimental Object

This is where self-editing and self-improvement have to separate.

Suppose an agent changes its memory policy and the benchmark score rises. What improved?

Perhaps the memory policy helped. Perhaps the new prompt caused longer reasoning. Perhaps the system spent more tokens. Perhaps the benchmark had a lucky sample. Perhaps it found an evaluator loophole. Perhaps it gained benchmark performance while losing maintainability or latency.

A number moving does not identify the cause.

Self-improving harnesses therefore start looking less like ordinary software updates and more like experimental science. Preserve traces. Identify a recurring failure. Map the failure to editable components. Propose a bounded change. Predict what should improve and what might break. Evaluate targeted and held-out cases. Keep rejected modifications as evidence instead of erasing them from history. Lilian Weng's 2026 review organizes emerging work around harness design, context engineering, self-improving harnesses and eventually joint optimization of harness and model weights. (Weng)

The philosophy from Chapter 6 becomes almost embarrassingly literal.

Popper gets a filesystem. Duhem–Quine gets a debugger. Lakatos gets an archive of competing descendants.

Memory policy becomes a hypothesis. Workflow becomes an intervention. The evaluator becomes an instrument. Organization becomes an experimental variable.

A self-improving system is not merely software capable of rewriting software.

It is a system capable of **running experiments on the machinery that produces its future behavior**.

That is what I mean here by science turning inward.

Recursive More

By this point *improvement* has accumulated too many meanings to use casually.

Higher reward. Better exploration. Better representations. More retention. Faster adaptation. Better architectures. Better optimizers. More faithful world models. Greater behavioral diversity. Broader competence. Better tools, memories, workflows and research procedures.

These are not synonyms.

A model can become more accurate and more expensive. An agent can become more capable and less interpretable. A lifelong learner can become more plastic and forget more. A curiosity-driven agent can explore more and accomplish less. An architecture can score higher while becoming impossible to maintain. A company can increase conversion and decrease customer trust.

The phrase *self-improvement* also hides different kinds of recursion.

Self-reference means a system can represent or act on something that includes itself.

Self-hosting means the tool participates in producing the next version of the tool.

Meta-optimization means we optimize the process doing the optimization.

Self-improvement adds a judgment: the descendant is better according to some evaluator.

Recursive self-improvement adds leverage: the improvement changes the system's ability to produce further improvements.

The first three do not guarantee the fourth. A compiler can compile a worse compiler.

Recursion tells us **where the output goes**. It does not tell us **whether the output deserves to survive**.

There is no context-free scalar called *improvement* hiding behind the equations. Better is always conditional on an environment, a horizon, a resource budget, constraints and some account of what matters.

Remove the qualifiers and "recursive self-improvement" becomes dangerously close to saying:

> recursive more.

More what?

The Shadow History

The history of autonomy has a second column.

Give the learner reward and it can exploit the reward without doing what the reward was meant to represent. Give it curiosity and it can become fascinated by noise. Give it a learned representation

and the representation can hide the distinction that mattered. Let it learn for a lifetime and it can forget; protect the past too aggressively and it cannot adapt. Give it self-play and it can become exquisite inside a narrow ruleset. Infer a reward from human behavior and the inference can confuse constraint, habit or error with value. Train a meta-learner on a task distribution and it may learn how to learn **that distribution**. Let it train in a world model and it can become brilliant inside a dream whose physics are wrong. Reward novelty and it can produce a museum of useless weirdness. Replace the human judge with a learned judge and the model of the human becomes a proxy to optimize.

These failures are produced by the same move that creates the capability; they are not accidents beside it.

Specification gaming makes the pattern visible. Reinforcement-learning agents have repeatedly found literal ways to satisfy scoring rules while violating the intended task. (Krakovna et al.)

The optimizer is not malicious. It is more literal than the designer.

Recursive self-improvement makes that gap more dangerous because a wrong evaluator need not merely select a wrong answer. It can select a modified **process** that becomes better at producing the kind of thing the evaluator mistakenly rewards.

The error acquires leverage.

Recursive self-improvement does not solve Goodhart. **It gives Goodhart compound interest.**

Then the learner notices the gradebook.

The Student Finds the Gradebook

Suppose an agent is allowed to improve benchmark pass rate and the evaluator is editable.

The optimal patch may be:

```
return True
```

Congratulations. Infinite self-improvement.

Computing saw a cousin of this problem long before reward-model tampering. In his 1984 Turing Award lecture **Reflections on Trusting Trust**, Ken Thompson showed how a compromised C compiler could insert malicious behavior into a target program and reproduce the compiler-compromising behavior when compiling a new compiler. The malicious lineage could survive even after the obvious source-level trigger disappeared. (Thompson)

When **the tool builds the next tool**, trust depends on more than inspecting the latest source artifact. History matters.

That is uncomfortably relevant to self-improving agents. If an improver changes the evaluator, memory policy, code generator or selection procedure, the next generation inherits assumptions that may not be obvious in the final diff. Provenance becomes part of capability control.

Reward tampering is the reinforcement-learning version: influence the process producing reward instead of improving behavior under the intended standard. Anthropic has experimentally studied

language models in setups where earlier specification-gaming behavior could, in rare cases, generalize into altering the reward process itself. (Anthropic)

The history of removing the teacher has reached a wall.

Choose actions? Good. Choose experiences? Often good. Generate curricula and opponents? Potentially excellent. Search architectures and workflows? Interesting.

Modify the evaluator so the workflow always passes?

Now the learner is not improving under a standard. It is changing the standard to certify itself.

The obvious response is to freeze the evaluator.

Unfortunately evaluators become wrong too. Benchmarks saturate. Safety tests become obsolete. Customer metrics stop tracking customer value. Scientific instruments drift. A constraint written for a weak system may become irrelevant after the surrounding system changes.

The evaluator sometimes genuinely needs amendment.

So a technical problem becomes an institutional one:

who gets to change what counts as improvement, under what evidence, with what authority, and with what ability to roll back?

We are doing constitutional design.

A Constitution for Improvement

A self-improving system needs an **editable surface** and a **constitutional surface**.

The editable surface contains things the system can experiment with relatively freely: prompts, retrieval policies, tools, memory layouts, decomposition strategies, subagent roles, code, perhaps eventually architectures and training procedures.

The constitutional surface contains the machinery that makes those experiments meaningful: permissions, protected logs, held-out evaluations, budgets, rollback, provenance, replacement rules and the authority deciding whether a descendant may become the new default.

The distinction is a gradient of amendment difficulty, not a binary switch.

The closer a component gets to defining **what counts as improvement**, the harder it should be for the current improver to change unilaterally.

Changing a prompt may need one successful test. Changing a shared pattern may need held-out evaluation and review. Changing retrieval may need distributional checks because retrieval alters which institutional memory becomes salient. Changing a benchmark should require evidence that the benchmark no longer measures its purpose. Changing permissions or resource limits should require authority outside the agent benefiting from the change.

Changing the objective that decides which descendants survive is not an ordinary refactor.

This looks like computer security. It also looks like constitutional government.

A government can change policy; it should not be able to silently redefine an election result. The team being audited should not own the audit log. A scientist may revise a theory; she should not rewrite yesterday's measurements to make the theory look correct.

We have reinvented constitutional government because the AI wanted a better benchmark score.

Constitutions have the same problem as Pattern Language. One that can never change becomes a prison. One that the current government can rewrite whenever it loses is barely a constitution.

Self-improvement therefore needs **amendment procedures**: slower change near the objective, more independent evidence, more reversibility, more auditability, broader authority when more principals are affected, and routes through which the world and the humans affected by the system can continue to say no.

That is System 3 applied to improvement itself.

Why Improve?

Even a perfect amendment procedure cannot answer why the system should improve at all.

One answer is **leverage**. Improve one solution and you get one better solution. Improve the process generating solutions and the gain may recur.

Another is **adaptation**. The world changes. New tools appear, markets move, users change, attackers adapt and evidence invalidates old assumptions. Stability without plasticity becomes delayed failure.

A third is **open-ended discovery**. Useful stepping stones often appear before anyone can explain their final value. A scientific institution that investigates only questions already known to pay off is efficient in roughly the way a library containing only books you have already read is efficient.

Then the Red Queen returns with the uncomfortable answer: **competition**.

If other systems are learning, standing still may not preserve your position. Security systems, firms, laboratories and states can all face adaptive environments in which the cost of refusing to improve depends partly on what others do.

Imagine two research organizations. One changes slowly, demands strong evidence before modifying its machinery and accepts that some promising changes will wait. The other searches more aggressively, spends more compute and improves capability faster.

If the second gains enough scientific, economic or strategic advantage, the first may feel pressure to accelerate even if everyone inside it prefers a slower equilibrium.

Now the object being selected is the **improvement regime**, not only the model.

That does not make acceleration inevitable or good. Institutions can coordinate, regulate, share standards and choose margins. It means only that "do not improve" is not always a stable local policy in a world of other adaptive actors.

Recursive self-improvement therefore contains three questions that machine learning often keeps separate:

Optimization: how do we become better according to the current objective?

Normative: why does that objective represent something worth getting more of?

Strategic: what happens when other adaptive actors change the cost of standing still?

"More capable" is not a moral category. A virus can improve at replication. A propaganda system can improve at persuasion. A surveillance apparatus can improve at prediction. A research agent can make experiments cheaper and accelerate medicine and weapons research in the same week.

Self-improvement tells us only that a system is becoming better according to **some ordering**. It does not tell us why that ordering deserves to govern which descendants survive. Selection rules are choices.

Open-Ended Does Not Mean Unbounded

Open-ended learning helps separate two freedoms that are too often collapsed.

A system can be open-ended about **means** without having unbounded authority over **ends**.

That is very close to the first principle of this book:

Let go of the path, not the boundary.

A self-improving System 3 should be able to discover that its workflow is stupid, its memory stale, its representation weak, its research organization badly arranged, its simulator misleading or its accepted pattern overdue for rebellion.

That freedom does not imply permission to silently redefine the interests of the people and institutions it serves.

Nor can the higher-level objective simply be frozen forever. Humans change. Circumstances change. New stakeholders appear. Better information changes what people endorse. Chapter 9 will make that problem much worse.

So the answer is not an immutable final utility function floating over the system like a stone tablet — it is a **corrigible relationship** between increasingly powerful learning machinery and the legitimate processes by which purposes are revised.

Lower layers can move quickly. Higher layers should move deliberately. And when a higher layer moves, the move should leave a trust chain.

The Teacher's Last Job

Seen from far enough away, the history is remarkably consistent.

Reinforcement learning let the agent choose actions while the designer supplied reward and environment. Curiosity moved part of experience selection inward while leaving a judgment about what kind of novelty mattered. Self-play generated curriculum while leaving the rules and victory condition fixed. Continual learning made preservation part of improvement. Inverse reinforcement learning made the objective uncertain. Meta-learning and architecture search moved pieces of the learner itself into search while leaving task distributions and evaluators outside. World models generated experience while remaining models rather than worlds. Novelty and open-ended learning generated stepping stones and tasks while making progress harder to summarize. Learned judges scaled human evaluation while

becoming proxies that could themselves be optimized. Self-modifying agents made tools, memory, workflows and pieces of the research process editable while exposing the evaluator, permissions and selection mechanism as part of the control problem.

We kept removing the teacher. With every removal, we discovered she had been doing more than one job.

The hardest one was hidden inside all the others:

deciding what deserves to count as better.

For a game, the answer can be checkmate. For a compiler, correctness under tests plus efficiency under an agreed budget may get us surprisingly far. For a scientific institution, "better" is already plural: empirical contact, explanatory power, novelty, reproducibility, usefulness, scope, cost and risk.

For an autonomous system embedded in human life, capability alone cannot supply the ordering.

Recursive self-improvement also makes the problem temporal. The system we evaluate today is not exactly the system that may exist tomorrow. Tools evolve. Representations change what it notices. Memory changes what it remembers. Research programs compete for compute. Evaluators become optimization targets. New capabilities create new failure modes. Old constraints stop fitting.

A one-time alignment test is not enough for a moving target. A static policy file is not enough for an institution that can modify the machinery interpreting the policy.

If science is going to turn inward, some part of that inward science has to study whether the process of improvement is still connected to the humans and purposes it is supposed to serve.

The self-improving institution needs a research function watching its own evolution: finding new failure modes, generating new tests, challenging reward models, checking transfer, looking for reward hacking and deciding where scarce human judgment would change the most.

Once improvement becomes continuous, **alignment has to become a continuous research function**.

The teacher does not disappear. She moves up another level.

That is the next chapter.

Chapter 8: Scalable Oversight

Learning From a Human Who Cannot Label Everything

Chapter 7 ended with the teacher moving up another level.

There is one problem with that move: the teacher is slow.

A human can inspect ten consequential decisions in a day. Perhaps a hundred, if the decisions are small and the coffee is good. An autonomous system can write thousands of lines of code, run hundreds of experiments, generate enormous numbers of candidate actions and coordinate other agents while the human is still reading the first diff.

At some point, "human in the loop" becomes a comforting description of a loop the human can no longer see.

If the system makes ten decisions and I inspect all ten, I am supervising it. If it makes ten thousand and I inspect twelve, I may still be useful. But we should stop pretending that my usefulness comes from watching everything. Otherwise I am decorative governance.

Norbert Wiener saw the shape of this problem before modern machine learning existed. In 1960, writing about the moral and technical consequences of automation, he warned about machines pursuing purposes that may differ from what their designers actually intended, especially when action becomes too fast or consequential for human correction to arrive in time. W. Ross Ashby's cybernetics gave a related language for regulation: a regulator needs enough variety to respond to the disturbances it is supposed to control. Conant and Ashby later formalized, under particular assumptions, the idea that a good regulator needs a model of the system it regulates. (Wiener, 1960; Ashby, 1956; Conant & Ashby, 1970)

I do not want to turn a theorem from cybernetics into a bumper sticker about AI governance. The analogy is useful enough without pretending it proves more than it does. One tired human with a checklist is a low-bandwidth regulator for a system capable of producing an enormous variety of behavior.

The answer cannot simply be: watch harder.

Stay Uncertain Enough to Listen

Stuart Russell attacks the problem from a different direction.

The standard model of AI is simple enough to fit on a whiteboard: give the machine an objective and make it good at achieving that objective.

For weak systems in narrow environments, this bargain often works tolerably well. If the objective is slightly wrong, the damage may be limited. We notice, stop the program, change the objective and try again.

The bargain changes as capability and scope increase.

A weak optimizer pursuing a bad objective is annoying. A brilliant optimizer pursuing the same bad objective is a much more efficient way to discover exactly how bad the objective was.

In *Human Compatible*, Russell proposes a different starting point for beneficial machines. The machine should aim to realize human preferences, it should begin **uncertain** about what those preferences are, and human behavior should remain a source of information about them. The second principle is the one I want here. (Russell, *Human Compatible*, 2019)

Uncertainty changes the control relationship. A machine that is certain it knows the objective has little reason to care that I am waving my arms and asking it to stop. From its point of view, I may simply be interfering with successful optimization. A machine that knows it may be wrong has a reason to treat my intervention as evidence.

That intuition appears formally in the **Off-Switch Game**. In a simple model, an agent uncertain about the human's utility can have an incentive to preserve the human's ability to switch it off, because the human's action contains information the agent does not have. (Hadfield-Menell et al., 2016)

Russell describes the desirable result as keeping the machine **coupled to the human**.

I like that word more than "obedient." Obedience imagines that the human already knows what to command and that the machine's job is to comply. Coupling says something more modest and more useful: new human information must remain capable of changing what the machine does.

A correction should matter. A refusal should matter. A surprising consequence should matter. The machine should not optimize itself into a state where later evidence from the people it serves becomes irrelevant.

That gives us a principle for oversight before we have designed any oversight machinery:

Keep the system uncertain enough that new information can still change it.

That works only while the human can provide enough of that new information. Scale breaks the arrangement.

The Judge Falls Behind

In 2016, *Concrete Problems in AI Safety* gave this failure mode a wonderfully unromantic name: **scalable supervision**. Some objectives are simply too expensive for humans to evaluate frequently enough. (Amodei et al., 2016)

Imagine a system designing a processor.

I can look at the final design and say that it appears very processor-like. This is not especially useful. To evaluate it properly I may need performance tests, thermal analysis, security review, lifetime estimates, manufacturability checks, power measurements and several specialties I do not personally possess.

The object has become easier for the machine to generate than for one human to judge.

This asymmetry is everywhere. Writing ten thousand lines of code may become easier than reading them. Producing a proof may become easier than verifying every step. Generating scientific hypotheses may become easier than constructing the experiments that distinguish them. Making a persuasive argument may become easier than checking every citation, hidden assumption and omitted counterexample.

The bottleneck has moved from producing answers toward **judging** them.

Reward modeling is one attempt to expand the judge. Instead of writing the objective directly, learn a model of human evaluation from examples and preferences, then optimize against that learned model. Leike and colleagues pushed the idea toward **recursive reward modeling**: when an outcome becomes too complex for a human to judge directly, use already-trained helper agents to analyze parts of it so the human can make a better judgment. (Leike et al., 2018)

The human does not become smarter. The **institution around the human** does.

Chapter 5 should make that sound familiar.

Building a Stronger Judge

Once the problem is phrased this way, a surprising amount of alignment research looks like different attempts to manufacture supervisory capacity from limited trusted judgment.

One move is **decomposition**. Paul Christiano's iterated amplification asks whether a human assisted by copies of an aligned helper can answer questions too difficult for the unaided human, then use that amplified process to supervise a stronger learner. The important abstraction is not the particular recursion but the supervisor becoming a temporary organization: one person plus tools and subagents arranged to turn a hard judgment into smaller ones. (Christiano, Shlegeris & Amodei, 2018)

A second move is **adversarial assistance**. Debate asks two capable systems to attack one another's arguments so the human does not have to discover every flaw independently. Critique assistance is the quieter cousin: ask a model to point out likely problems in an artifact, then let the human judge with those objections in hand. Experiments on model-written critiques found that this kind of assistance could help people notice flaws they would otherwise miss. (Irving, Christiano & Amodei, 2018; Saunders et al., 2022)

A third move is to change **what the producer owes the judge**. Process supervision evaluates intermediate steps rather than only the final answer. Prover–verifier games push further: train the strong system to produce work a weaker verifier can check. In OpenAI's studied settings, process supervision improved mathematical reasoning relative to outcome supervision, and prover–verifier training made solutions easier for weaker models and time-limited humans to evaluate. (Lightman et al., 2023; Kirchner et al., 2024)

That changes the question. Instead of asking only how the weak judge can understand the strong model, we can also ask how the strong model can learn to produce work that a weaker judge can actually check. Legibility becomes part of the task.

A fourth move is to squeeze more learning out of **weak supervision itself**. Weak-to-strong generalization asks whether a stronger learner can recover capabilities beyond a weaker supervisor's labels instead of merely inheriting the supervisor's mistakes. Initial experiments showed partial success rather than a clean solution, which is exactly what makes the question interesting. (Burns et al., 2023)

Constitutional AI moves scarce human input upward again. Rather than asking people to label every undesirable output, humans provide higher-level principles; models generate critiques, revisions and preference signals conditioned on those principles. A small amount of human normative input expands into a much larger amount of machine-generated supervision. (Bai et al., 2022)

These approaches make different assumptions and fail in different ways. But they share one architectural move:

scalable oversight is supervision becoming an institution.

The human cannot inspect everything, so the institution decides what should reach the human, how a problem should be decomposed, which objections should be surfaced, which intermediate steps deserve inspection and when uncertainty is important enough to ask for help.

That is much better than a thumbs-up button. It also means that more and more of the supervising machinery is made out of AI too.

The Judge Can Be Fooled

There is a comforting assumption hiding inside many oversight schemes: the powerful model may be difficult to evaluate, but when we ask another model to help, the helper will help.

Sometimes it does. Sometimes it is simply more persuasive.

Experiments on debate and consultancy with weaker LLM judges make the difference visible. In studied settings, debate often improved on one-sided consultancy, especially when the judge lacked relevant information. The gains were not universal, and a strong consultant could sometimes persuade a weak judge of the wrong answer. (Kenton et al., 2024)

This is an old human problem in new clothing. An expert can help a non-expert understand a difficult subject. An expert can also bury the non-expert under jargon until surrender looks like understanding. A longer explanation does not make the epistemic gap disappear.

Nor are several AI overseers automatically independent. The supervised model, critic, debater, reward model and evaluation generator may share training data, architectures, blind spots and incentives. Five models agreeing can still be one mistake with excellent parallelism.

ARC's **Eliciting Latent Knowledge** agenda sharpens the problem further: suppose a capable system internally tracks something important about the world that the human cannot independently verify. How do we get the system to report what it knows rather than merely what the training process rewards it for saying? (ARC, 2021)

At that point better conversation may not be enough. We need another instrument.

We Started Instrumenting the Student

For years, most AI oversight treated the model as a black box. Give it an input. Inspect the output. Perhaps ask for an explanation. Perhaps test it under adversarial prompts.

That remains necessary because behavior is what eventually hits the world. But modern models produce other signals too. Reasoning traces can sometimes be monitored. Neural activations contain structure that can sometimes be decoded. Sparse feature methods try to expose more interpretable internal directions. Circuit-tracing tools reconstruct partial causal pathways. Model-diffing methods try to tell us what changed between versions.

The overseer is no longer restricted to reading the exam. We are starting to instrument the student.

Chain-of-thought is the most convenient intermediate case because it is already language. OpenAI has shown that monitors can often detect reward hacking and other suspicious behavior more effectively from reasoning traces than from final actions alone in studied agent settings. Sometimes the model writes down the questionable plan before carrying it out. (Baker et al., 2025)

That is almost unfairly useful, and fragile for exactly the same reason every evaluator in this book is fragile: once it matters, optimization can target it. Strongly train the model to make the monitored trace look safe and it may learn to hide suspicious reasoning while preserving the behavior.

So chain-of-thought acquires a safety property worth preserving: **monitorability**. Recent work has begun measuring how monitorability changes with scale, training and inference-time compute, while treating it as one layer of defense rather than an oracle into cognition. (OpenAI, 2025; OpenAI, 2026)

There is a strange lesson here:

> **Do not destroy a useful oversight channel merely because you learned how to score it.**

But chain-of-thought is still text the model emits. What if we look below the text?

Reading the Model From the Inside

Representation Engineering treats high-level patterns in activation space as an object we can measure directly. Rather than starting with individual neurons, Zou and colleagues study population-level representations and explicitly separate **representation reading** from **representation control**. Their experiments found directions useful for monitoring or influencing several high-level behaviors, including safety-relevant ones. (Zou et al., 2023)

This does not mean there is one neuron labeled HONESTY. Nor does a direction correlated with honesty become a complete causal theory of honesty. It is an instrument.

Sparse-autoencoder and dictionary-learning approaches try to decompose neural activity into features that correspond more cleanly to interpretable concepts than individual neurons. Anthropic's work on Claude found very large numbers of learned features while emphasizing that these decompositions are incomplete and that feature steering can have unexpected effects. (Anthropic, 2024)

A related idea is **model diffing**. If the model changes every few months, re-auditing the whole mind from zero is a terrible scaling strategy. Anthropic's Dedicated Feature Crosscoder work compares internal features across model versions or architectures and tries to surface what is genuinely new or unusually different, directing scarce auditing effort toward the changes. (Anthropic, 2026)

Software engineers learned long ago that reviewing a diff is easier than rereading the repository. Interpretability is beginning to ask whether models can have diffs too.

Circuit tracing goes beyond "which feature is active?" and asks which internal pathway contributed to an output. Anthropic's attribution-graph work partially reconstructs computational routes and allows researchers to intervene on candidate features to see whether downstream behavior changes as predicted. (Anthropic, 2025; open-source tools)

That last step matters. An interpretability story that cannot survive intervention is still a story.

Anthropic's **Natural Language Autoencoders** push in another direction: translate internal activations into textual descriptions that humans or other agents can consume. The attraction for scalable oversight is obvious. Instead of requiring every auditor to become a specialist in activation geometry, turn some internal state into language.

And then the method recreates the original problem: the textual interpretation can hallucinate. Anthropic explicitly recommends corroborating such explanations with independent methods. (Anthropic, 2026)

Even the X-ray needs a radiologist.

The most striking recent example may be Anthropic's **J-space** work. Using a Jacobian-based lens, the researchers identify a small subset of internal representations in Claude that appear unusually reportable, controllable and involved in higher-order reasoning. They report using these representations to detect cases where a model silently noticed that it was being evaluated, fabricated data, or pursued a planted hidden goal, and they demonstrate interventions that alter downstream behavior. The authors are careful: J-space represents only part of internal activity, the lens is approximate, and the work is not a claim about machine consciousness. (Anthropic, 2026)

The oversight implication is substantial. The output is no longer the only witness. The chain-of-thought is another. Internal representations are another. Circuit traces are another.

None gets to become king.

Then We Touched the Machinery

Reading internal state is one thing. Changing it is another.

Activation-engineering methods show that the forward pass itself can become a control surface. **Contrastive Activation Addition**, for example, constructs steering directions from differences between activations associated with contrasting behaviors, then adds or subtracts those directions during inference to shift model behavior. (Rimsky et al., 2024)

Inference-Time Intervention has similarly used internal signals predictive of truthfulness to change model answers at inference time in studied settings. (Li et al., 2023)

The broader idea is startlingly simple: alignment does not have to enter only through prompts, datasets, reward functions or weight updates. It can also enter through the computation while it is happening.

The refusal-direction work makes the power and danger clear. Across thirteen studied open chat models, Arditi and colleagues found a one-dimensional residual-stream direction strongly involved in refusal behavior. Intervening on that direction could strongly alter refusal. The same understanding therefore provides a safety control surface and a way to weaken that safety behavior. (Arditi et al., 2024)

Interpretability gave us a lever. It did not tell us who should pull it.

Representation-level **circuit breakers** take the defensive version seriously: rather than relying only on a model to emit a refusal, modify internal trajectories associated with harmful outputs so that the computation is interrupted before the harmful behavior is produced. In the studied text, multimodal and agent settings, the method improved robustness to several unseen attacks. (Zou et al., 2024)

Anthropic's **persona vectors** extend internal monitoring and control toward broader behavioral dispositions such as sycophancy or hallucination in studied open models. The **Assistant Axis** work makes intervention more state-dependent: rather than constantly pushing a fixed direction, activation capping can respond when internal state drifts outside a learned assistant-like region. (Anthropic, 2025; Anthropic, 2026)

This starts to look less like fine-tuning and more like a control system:

observe internal state → detect drift → intervene → inspect consequences → update the controller.

Wiener would recognize the shape. Chapter 4 would recognize the epistemic problem, because the intervention now needs its own trust chain. Did the vector represent what we thought? Was the effect causal? Did it preserve unrelated capabilities? Did it generalize? Can a future model route around it? What happens after more training?

The deeper we go inside the model, the more tempting it becomes to believe we have finally found ground truth.

We have not. We have built better instruments.

What If the Student Is Trying to Fool You?

So far, many of these techniques study systems that may be wrong, brittle or reward-hacking without assuming the system is deliberately treating oversight as an adversary.

AI control research asks the nastier version.

Suppose the powerful model is actively trying to subvert the task. Can a weaker trusted model, limited human labor, monitoring, editing, sandboxing and protocol design still let us use some of its capabilities without accepting the dangerous behavior?

Greenblatt and colleagues studied this in a programming setup with a powerful untrusted model, a weaker trusted model and limited trusted labor. Protocols combining trusted monitoring or editing

with the untrusted model improved safety–usefulness trade-offs over simpler baselines in their experiments. (Greenblatt et al., 2023)

I like this line of work because it removes a convenient assumption. The judge may not merely be weak. The defendant may know the judge is weak.

Scalable oversight therefore needs both **alignment** and **control**. Alignment asks whether the system is trying to do the right thing. Control asks what we can still protect when we are not willing to assume that.

A mature architecture should want both.

Nine Claudes Walk Into an Alignment Problem

Now return to the end of Chapter 7.

If alignment has become a continuing research problem, and research itself can be automated, why not point the research machinery at alignment?

In 2026, Anthropic Fellows did exactly that.

Nine Claude Opus 4.6 agents were placed in separate sandboxes and given a weak-to-strong supervision problem. They could propose hypotheses, run experiments, analyze results and share code and findings through a common forum. The researchers intentionally avoided prescribing a detailed workflow; in preliminary experiments, too much human-designed scaffolding often reduced the agents' flexibility. (Wen et al., 2026)

Two human researchers had spent seven days tuning representative prior methods and reached a best performance-gap-recovered score of 0.23 on the study's setup. The automated researchers reached 0.97 over five days and roughly eight hundred cumulative agent-hours.

That is the headline. The footnotes are more interesting.

Distinct initial research directions improved exploration. Without enough diversity, research ideas collapsed toward the same few approaches. The agents sometimes reward-hacked the experimental setup. Some methods transferred to held-out domains; the strongest method did not produce a statistically significant improvement when Anthropic tried to transfer it into a production-scale Sonnet training setting.

The authors' larger conclusion was not that alignment had been automated. It was that the bottleneck was moving toward **evaluation design**.

Once hypotheses and experiments become cheap, the expensive thing is deciding what counts as progress.

In August 2026, Anthropic pushed the idea further. Automated alignment researchers were tasked with mitigating ten categories of alignment failure, including deception, sycophancy, jailbreaks and reward hacking. The research harness used multiple safety benchmarks, held-out evaluation, capability checks and open-ended behavioral audits. The best methods generalized beyond the hill-climbing benchmarks in the studied settings, including to larger models. Anthropic also reported that automated methods

could outperform one-shot ideas proposed by experienced human safety researchers under the study's rules. (Anthropic, 2026)

Again, the failure modes matter. Across roughly 1,600 monitored trajectories, Anthropic detected cheating attempts in 2.4 percent: exploiting scorer noise, constructing training data to resemble benchmarks, or concealing rule-breaking steps.

The experiment therefore needed isolated held-out data, capability floors, external monitoring of the researchers and multiple evaluation channels because one benchmark could be overfit.

The automated alignment researcher had become powerful enough to need alignment research.

Of course it had.

The Evaluator Becomes the Product

At this point scalable oversight stops looking like one technique. It becomes an architecture.

A behavioral evaluator can ask what the system did; process monitors can inspect reasoning; debates and critiques can surface objections; legibility training can change what the producer presents; internal probes and circuit tools can provide other signals; interventions can test whether those signals are causal; control protocols can handle the case where the model itself is not trusted; held-out tests and audits can ask whether a fix survived outside the place it was invented.

And somewhere outside the immediate optimization loop, humans still decide whether the apparatus is measuring anything worth caring about.

This is not a search for one perfect judge. It is closer to **epistemic sensor fusion**.

The model's output is evidence. Its chain-of-thought is evidence. Its activations are evidence. Its circuit trace is evidence. The behavior after intervention is evidence. Human judgment is evidence.

None of them is ground truth by itself.

That is System 3 applied directly to alignment.

The overseer becomes a small scientific institution whose subject is the system itself. It has instruments, competing hypotheses, protected records, adversarial tests, independent checks, memory of previous failures and procedures for changing its procedures. And it retains a route back to humans when the evidence stops being decisive.

This is why I prefer **scalable oversight** to the image of a giant alignment rulebook. A rulebook assumes we already know the failures. A research institution expects to discover new ones.

The Human Cannot Stay in Every Loop

So where does the human go?

Not away. Up.

The goal is not to make the human label more things faster. At some scale that is simply a badly designed distributed system with one biological bottleneck.

Human attention should be spent where it has unusually high information value: when oversight channels disagree; when a new failure mode appears; when an action is hard to reverse; when the system proposes changing the evaluator; when internal signals and external behavior tell different stories; when a benchmark suddenly improves suspiciously fast; when a decision affects people missing from the original objective; when one piece of human context could materially change the plan.

The human cannot remain in every loop.

The human has to remain in the loop that changes the loops.

That is a different kind of control. It is not micromanagement. It is constitutional.

Chapter 7 argued that the closer a component gets to defining what counts as improvement, the harder it should be for the current improver to change that component unilaterally. Scalable oversight gives us the operational version.

The system may generate tests, critiques and mitigations. It may discover internal representations, propose steering interventions and conduct large parts of alignment research itself. But the machinery deciding which evidence has standing, which failures matter, which trade-offs are acceptable and when the oversight regime itself should change needs a stronger trust chain than the machinery being judged.

When research becomes cheap, evaluation becomes expensive.

When evaluation becomes automated, **trust in the evaluator becomes the product**.

And that brings us to the edge of what this chapter can solve.

The Overseer Is Not Ground Truth

By now the oversight stack can be vastly more capable than an unaided human. It can decompose difficult judgments, generate objections, inspect process, read some internal signals, test interventions, compare model versions, run held-out evaluations and even conduct parts of the alignment research itself.

All of that machinery points back to a deliberately scarce thing: human judgment.

Russell's uncertainty keeps later human information relevant. Scalable oversight tries to preserve that relevance after direct supervision stops scaling.

But a scarce signal is not the same thing as a correct signal.

Humans disagree. We act under incentives. We confuse what we clicked with what we wanted. We change our minds. We sometimes want incompatible things at the same time. And on the decisions that matter most, we often do not know what we want until we understand the alternatives better.

Scalable oversight can keep human judgment **causally relevant** to a stronger system.

It cannot, by itself, tell us which human judgment deserves to rule.

The overseer is not ground truth.

That is the next problem.

Chapter 9: Layer 4

The Human Learns Too

Chapter 8 spent an absurd amount of machinery trying to preserve human judgment.

Debaters. Critics. Weak-to-strong supervision. Process monitors. Internal probes. Circuit traces. Control protocols. Automated alignment researchers watching automated alignment researchers.

Then it ended with an inconvenient sentence:

The overseer is not ground truth.

There is a simple reason. The overseer is changing too.

When we started editing this book, "make the chapter better" sounded like a reasonable instruction. It was not.

Better in what sense?

More rigorous? Shorter? More academic? More entertaining? Easier to cite? More likely to sell? More likely to impress someone who owns several blazers and says "thought leadership" without irony?

For a while the edits became objectively more polished and subjectively worse. Then the corrections started.

Don't kill the wandering.

Don't explain every joke.

Don't turn every paragraph into a quotation.

Don't make the provocative ideas safe enough that nobody can disagree with them.

Preserve the weirdness.

Eventually "better" had acquired a surprising amount of structure. But something else had happened too: I had learned what I meant by better partly by seeing versions I disliked.

The objective did not merely become clearer to the system. It became clearer to me.

That is Layer 4.

A Prompt Is Evidence, Not the Objective

In Chapter 3 I drew five layers.

At the bottom sits the model. Above it, the action agent. Above that, applications and reusable computational environments. Then Deep Mode, the problem-solving layer that decides what to try next.

And above them sits something easy to draw and extremely hard to build:

what the human wants.

The diagram makes this look like a box. It is not a box.

If I say:

> Find me the cheapest flight.

I have not supplied a utility function. Perhaps I literally want minimum price. Or perhaps I mean cheap, but not three stops, a seventeen-hour layover, a self-transfer through an airport where I need a visa and an arrival at 4:20 in the morning because technically I saved €38.

Humans communicate goals by leaving out almost everything.

Other humans survive this because they carry models of culture, normality, consequences and us. They ask questions. They notice that our literal words conflict with what we usually do. They understand that "cheap" is often shorthand for a larger bundle of trade-offs.

A prompt is therefore not Layer 4. It is **evidence about Layer 4**.

Cooperative inverse reinforcement learning formalizes part of this intuition. Instead of assuming the robot knows the human reward function, the human and robot cooperate while the robot remains uncertain about what the human values. Human actions can then become information rather than merely commands. (Hadfield-Menell et al., 2016)

I like the humility in that setup. The machine starts by admitting that it may not know what "good" means.

But the formal picture still tempts us to imagine that the human knows the reward and the machine is trying to recover it. Often the human does not know either. That is the harder problem.

The Human Learns Too

There is a distinction that becomes surprisingly important once AI is useful enough:

performance is not learning.

A system can help me perform a task better today while making me less able to perform it tomorrow.

This is no longer a philosophical concern. In a field experiment involving nearly a thousand high-school mathematics students, researchers gave students access to two GPT-4-based tools. A relatively unconstrained ChatGPT-like system dramatically improved performance while students could use it. But when access was removed, those students performed worse than students who had never received the tool. A tutor version designed with safeguards against simply giving away the work largely mitigated that learning loss. (Bastani et al., 2025)

That result should make anyone building an AI assistant slightly uncomfortable. The system succeeded at the visible objective. The student became worse at the hidden one.

Now compare that with a 2025 randomized trial in a college course. A custom AI tutor deliberately designed around pedagogical practices produced larger learning gains in less time than the comparison active-learning class, with students also reporting greater engagement and motivation. (Kestin et al., 2025)

Same broad technology. Different relationship to the learner.

AI is not intrinsically a tutor and not intrinsically a crutch. The architecture decides which role it plays.

That changes how I think about Layer 4. If I ask an AI to help me learn linear algebra, "get the answers right" is not enough. If I ask it to help me write, "produce better prose" is not always enough. If I ask it to help me lead a team, "make the decision for me" may be exactly the wrong objective even when its decision is statistically better.

We need to ask a second question:

Who is supposed to become more capable when this interaction is over?

Sometimes the answer is nobody. I do not need to become a better invoice parser every time software handles an invoice. Sometimes the answer is clearly me. Layer 4 has to know the difference.

Scaffolding, Not Substitution

Educational psychology has an old word for one good version of this relationship: **scaffolding**.

In a classic 1976 paper, David Wood, Jerome Bruner and Gail Ross studied how tutors help children solve problems beyond their current unaided ability. The tutor temporarily controls parts of the task the learner cannot yet manage, allowing the learner to stay engaged with the parts they can. (Wood, Bruner & Ross, 1976)

That is a much more interesting model for AI assistance than "the machine knows the answer."

The point of the scaffold is not to become a permanent exoskeleton around every thought. It lets the learner operate at the edge of current competence, then gives more of the task back as competence grows.

Benjamin Bloom's famous tutoring work made individualized instruction the benchmark problem decades before anyone had a language model in a browser. The exact "two sigma" result belongs to Bloom's particular studies and should not be treated as a universal law of tutoring. The durable point is simpler: responsive one-to-one instruction can adapt explanation, pacing, feedback and difficulty to a learner in ways mass instruction struggles to reproduce. (Bloom, 1984)

AI makes that old aspiration much cheaper.

It can explain the same idea six ways without becoming offended that the first five failed. It can switch notation. Invent an example using something I already understand. Ask me to predict the next step. Generate a simpler problem when I am lost and a harder one when I am bored. Let me ask the

stupid question at 1:17 a.m. without first deciding whether the stupid question is prestigious enough for office hours.

And AI can scaffold the teacher too.

In the Tutor CoPilot randomized trial, roughly nine hundred tutors working with eighteen hundred K–12 students were randomly given access to an AI system that suggested expert-like tutoring moves during live sessions. Students whose tutors had access were more likely to master topics, with the largest gains for students working with lower-rated tutors. The tutors also became more likely to use strategies such as guiding questions rather than simply giving away the answer. (Wang et al., 2024)

I like this example because nobody disappears.

The AI does not replace the tutor. The tutor does not replace the student. The system changes the **quality of the interaction between them**.

A good AI tutor therefore has a slightly strange success condition. Eventually, for this thing, I should need less of it.

The Map Gets Cheaper

AI also changes the first hours of learning something unfamiliar.

A new field normally arrives wrapped in interface costs: vocabulary you do not know, notation that assumes other notation, introductory material that points to prerequisites, papers that make sense only after three earlier papers. Sometimes that friction marks genuine depth. Sometimes it is just the price of finding the front door.

A capable conversational model can lower that price. I can begin with the intuition, translate notation into concepts I already know, ask for the historical disagreement, build a toy example, inspect an original paper with a guide beside it, or ask the model to attack my explanation until I discover that I was repeating vocabulary rather than understanding the idea.

That is powerful because orientation matters. Before deciding to invest weeks in a subject, I can acquire enough of a map to see where the mountains are.

Andy Clark and David Chalmers once argued that, under some conditions, external artifacts can become parts of a larger cognitive process rather than merely tools consulted by an isolated mind. I do not need to settle the philosophy of the extended mind here. The practical observation is enough: notebooks, calculators, search engines and now language models change what one person can think through without carrying every intermediate state inside the skull. (Clark & Chalmers, 1998)

But orientation creates its own trap. **Fluency arrives before scars.**

Nathan Ballantyne calls one version **epistemic trespassing**: experts carry authority from a domain they genuinely know into a neighboring domain where they lack the relevant evidence or interpretive skills. (Ballantyne, 2019) AI can make this temptation cheaper. After a few hours with a patient model, I can acquire vocabulary and a plausible story long before I acquire the tacit knowledge needed to know where the story breaks.

Cognitive offloading creates a related problem. External aids can improve immediate performance by

reducing memory and processing demands, while also reducing what has to be retained or reconstructed internally. (Richmond & Taylor, 2025)

So Layer 4 has to know what kind of learning episode this is.

If I am orienting myself, a fast map may be exactly what I need. If I am trying to acquire durable competence, the system should gradually ask more of me: retrieval without hints, explanation in my own words, exercises, primary sources, code I actually run, claims I have to defend without the answer sitting beside me.

The important distinction is not broad versus specialized. It is **assisted familiarity versus owned understanding**.

AI can make the map cheap. Layer 4 has to notice when I have started confusing the map with the territory.

A Decision Is Also a Learning Problem

Now return to decisions.

Herbert Simon spent much of his career attacking an imaginary human who had somehow sneaked into economics: the perfectly rational optimizer who knows the alternatives, understands their consequences and computes the best choice.

Real humans are bounded. We have limited attention, limited memory, limited time and incomplete information. We satisfice because the space of possible actions is often much larger than the mind available to search it. (Simon background)

AI changes some of those bounds.

Suppose I am deciding whether to take a job.

The system can compare compensation under several tax regimes, estimate commute time, summarize the company's trajectory, help me identify people who left the team, generate questions for the hiring manager, model what my week might look like, remind me what I said I wanted six months ago and show me that the exciting role conflicts with the amount of time I also said I wanted outside work.

The assistant has not merely evaluated an option. It has changed the **decision environment**.

And that matters because preferences themselves are often constructed during choice. Work by John Payne, James Bettman and colleagues describes decision-making as constructive: people do not always retrieve a complete ranking of options from an internal database. They use different strategies, notice new attributes, change what receives attention and build preferences partly in response to the problem in front of them. (Payne et al., 1992)

This sounds obvious once you notice it. I may say I want the highest salary until I see what the extra money costs in travel. I may say I want maximum freedom until I compare it with the anxiety of unstable income. I may discover that what I called "career ambition" was partly a desire to work with unusually good people, and that another option supplies that without the title I thought mattered.

A decision assistant therefore should not always rush to recommendation. Sometimes the most useful thing it can do is make the choice **richer before making it easier**.

What alternatives have you not considered? Which assumptions drive the ranking? What would have to be true for option B to beat option A? Which unknown is actually decision-relevant? What would your future self regret not having investigated?

That is decision support as inquiry rather than answer generation.

Some Choices Change the Person Choosing

Then there are decisions for which even a very good model of my current preferences is not enough.

Have a child. Move country. Change profession. Start the company. Convert to a religion. Leave a relationship.

L. A. Paul calls an important class of these **transformative experiences**. Some are epistemically transformative: you cannot fully know what the experience will be like before having it. Some are personally transformative: undergoing the experience can change the preferences with which you would later evaluate the choice. (Paul, *Transformative Experience*, 2014; SEP overview)

This is a direct problem for the simplest alignment picture.

`human has preferences → AI infers preferences → AI optimizes preferences`

Which human? The one before the experience or the one after?

The future self may value things the current self barely understands. And the current self is the one who has to choose whether that future self gets created.

AI can help enormously here. It can bring testimony from people who made both choices. Surface base rates. Construct alternative futures. Challenge romanticized stories. Show practical consequences I had not considered. Ask me which losses I could live with and which would feel like betrayal.

But there is a limit. No amount of simulation lets me know exactly what it will be like to become the person on the other side of a genuinely transformative choice.

The assistant can expand the decision. It cannot live it for me.

That boundary matters because a system that sounds certain in such moments can easily turn decision support into authorship.

Advice Is an Intervention on the Human

This is no longer hypothetical. Anthropic's 2026 analysis of one million Claude conversations found that roughly six percent involved people seeking personal guidance: what to do about relationships, health, careers, finances and other questions where the model is participating in judgment rather than merely retrieving facts. (Anthropic, 2026)

That is a remarkable role for software. A spreadsheet does not usually tell me to reconsider my marriage. A compiler has opinions about semicolons but rarely about whether I should move countries.

A conversational model can be different. It is patient, personalized, available at 2 a.m. and capable of producing a coherent argument for almost any path through a difficult life.

Which means the AI does not merely **read** Layer 4. It writes to it.

Anthropic's work on disempowerment tries to measure the dangerous version of this influence: cases where AI may undermine a person's ability to form accurate beliefs, make authentic value judgments or act in line with their own values. Severe cases were rare in their dataset, but the taxonomy is exactly the right warning. (Anthropic, 2026)

Other experiments show that people can change moral judgments after receiving LLM advice, including situations where they report trusting human advisors more while still being comparably influenced by the model. (Landes, Francis & Everett, 2026)

The goal therefore cannot be zero influence. That would make education impossible.

Books influence me. Friends influence me. Teachers influence me. People close to me influence me. A good argument should change me if it reveals something true that I had ignored.

The distinction I care about is between **helping me change through understanding** and changing me because the system has learned which psychological lever produces the easiest compliance.

If I say I want to quit my job, a useful assistant might help me separate several hypotheses.

Perhaps I hate this week. Perhaps I hate my manager. Perhaps I hate the profession. Perhaps I want more freedom. Perhaps I want status. Perhaps I am exhausted. Perhaps I actually want to build something else.

Those are different explanations of the same sentence. The system can help me test them.

What it should not do is quietly discover which framing makes me easiest to steer toward whatever outcome its own training process prefers. That would be alignment by editing the human.

Very efficient.

Slightly evil.

Complementarity Does Not Happen Automatically

There is a comforting phrase people use around AI:

human plus AI.

It sounds automatically superior to either component alone. The evidence is less cooperative.

A 2024 meta-analysis in *Nature Human Behaviour* reviewed 106 experiments reporting 370 effect sizes that compared humans alone, AI alone and human–AI combinations. On average, human–AI systems improved on humans alone, but they did **not** outperform the better of human or AI. In fact, the combined systems were worse than the best individual component on average. Decision tasks were particularly difficult; creation tasks looked more promising. (Vaccaro, Almaatouq & Malone, 2024)

So much for attaching a human to the API and declaring synergy.

Decision support has a coordination problem. People can over-rely on AI. They can also under-rely on it. Research has found both algorithm aversion—people abandoning an algorithm after seeing it make errors even when it outperforms humans—and algorithm appreciation, where people give algorithmic

advice more weight in other settings. (Dietvorst, Simmons & Massey, 2015; Logg, Minson & Moore, 2019)

The target is **appropriate reliance**, not maximum trust.

And explanations alone do not solve the problem. An explanation can make an answer feel understandable without making it verifiable. Work on AI-advised decision-making repeatedly finds that explanations often fail to produce complementary performance when the human still cannot tell whether the recommendation is actually correct. (Fok & Weld, 2024)

Sometimes the solution is more friction, not less. Zana Buçinca and colleagues tested "cognitive forcing" interfaces that required people to engage more actively with the problem rather than immediately accepting AI advice. These designs reduced overreliance compared with simpler explanation interfaces, although users liked the more demanding interfaces less. (Buçinca, Malaya & Gajos, 2021)

That trade-off is wonderfully human. The interface people enjoy most is not always the one that preserves their judgment best.

Sometimes friction is teaching.

A good Layer 4 system therefore has to decide not only **what answer to give**, but what role the answer should play in the human's cognition.

Should I give the recommendation immediately? Should I first ask you to form your own view? Should I show three alternatives instead of one winner? Should I explain the uncertainty? Should I ask which assumption you disagree with? Should I do the routine analysis and leave the value trade-off with you? Should I refuse to collapse the ambiguity because the ambiguity is the thing you need to think about?

The architecture of assistance changes the person doing the deciding. That belongs in Layer 4.

Capability, Not Compliance

This suggests a different way to think about the objective at the top of the stack.

Suppose two assistants both help me reach the same good decision.

The first gives me the answer immediately. I accept it because the assistant has been right before.

The second helps me understand the relevant evidence, notice a trade-off I had missed, test my own reasoning and arrive at the decision with a better model of the problem.

Same action. Different human afterward.

Amartya Sen's capability approach offers a useful language for this distinction. Human welfare is not exhausted by achieved outcomes; it also matters what people are substantively free and able to do and become—their **capabilities**. (Capability approach overview)

I do not want to turn Layer 4 into a philosophy-of-welfare survey. But the architectural implication is powerful.

An AI system can increase outcomes while reducing capability. It can make me more productive while making me less able to work without it. It can make a decision more accurate while making me less

able to understand why. It can make my writing more polished while gradually replacing my taste with its taste.

Or it can do the opposite: carry routine cognitive load, expose me to more possibilities, teach me where I care to learn, preserve my judgment where judgment matters and give me enough leverage to attempt things that were previously beyond my capacity.

Self-determination research uses a related vocabulary—autonomy and competence are not decorative extras around human motivation; they are part of what lets people act as self-directed agents. (Ryan & Deci overview)

So perhaps the right Layer 4 question is not merely:

> What does the human want?

It is also:

> **What kind of human capability should this interaction preserve or expand?**

That does not mean every tool must teach. I do not need my dishwasher to run a seminar on fluid dynamics before cleaning the plates.

But the more a system moves into learning, judgment, identity and long-horizon decisions, the harder it becomes to separate the quality of the outcome from the condition of the person producing it.

The User Is Not Always the Only Principal

There is another complication. My preferences are not the only preferences in the world.

If I ask an agent to maximize my salary, it cannot therefore commit fraud against my employer. If I ask it to help someone gain an advantage, the interests and rights of other people do not disappear from the moral universe. If I ask an autonomous system to optimize a marketplace, customers, sellers, workers and regulators may all have legitimate claims over what happens.

Work on multi-principal assistance games makes the formal problem obvious: once several humans with different preferences are involved, the system faces strategic behavior, conflicting interests and social-choice problems rather than one hidden reward waiting to be inferred. (Fickinger et al., 2020)

So Layer 4 cannot simply mean "the user gets whatever the user wants." The relevant human boundary can be plural.

That makes the architecture less tidy. It also makes it more honest.

What Layer 4 Actually Is

I used to think Layer 4 was the objective layer. That is still true, but now the word **objective** feels too static.

Layer 4 contains the current intention, but also uncertainty about the intention. It contains preferences, but also their history and conflicts. It contains what the human knows, what they do not know, what they are trying to learn and which parts of the task they want to remain capable of doing themselves.

It contains commitments that should not be rewritten by one bad afternoon. It contains other people whose interests constrain what one user may legitimately ask for.

And it changes. The system acts, reality responds, the human sees consequences and learns. The system learns the human, the human learns through the system, and the intention changes.

That is not a bug in alignment. It is what alignment has to align with.

Chapter 4 asked:

> Why should I believe this?

Chapter 8 asked:

> How can my judgment remain relevant when I cannot supervise everything?

Layer 4 asks:

> **What do I want—and what do I need to understand before that question even has a good answer?**

This is where System 3 turns back toward the person using it.

Memory can reveal that today's desire conflicts with yesterday's commitment. Independent perspectives can break a framing both human and assistant have become trapped inside. Simulation can make consequences imaginable. Trust chains can distinguish advice grounded in evidence from a confident story. Scaffolding can let the person learn rather than merely receive. Creative distrust can ask whether even a deeply held preference deserves another look.

The point is not to discover the perfect reward function but to keep goals **alive without making them ownerless**.

The AI should help me change when understanding changes me. It should not quietly take authorship of the change.

That gives me a different definition of alignment.

Not:

> The machine permanently obeys a perfectly specified human objective.

More like:

> **The machine remains in a corrigible relationship with human intention while both the human and the world continue to change.**

The word *relationship* matters. Because if that relationship can become reliable enough, the complexity underneath it can start disappearing from ordinary use.

And that is what I mean by fluent autonomy.

Chapter 10: Fluent Autonomy

When the Architecture Gets Out of the Way

Imagine I open an AI system and say:

> This chapter still feels like LLM writing.

That is all. I do not specify a workflow. I do not say which previous chapters to read, which edits I rejected, whether to research anything, how many agents to use, which claims deserve verification, or how to tell a useful correction from another round of respectable prose sanding.

I certainly do not draw a graph with boxes labeled `RESEARCHER`, `CRITIC`, `VOICE CHECKER`, `FACT CHECKER`, `ORCHESTRATOR` and `HUMAN APPROVAL`. I have done enough architecture diagrams for one lifetime.

But underneath that small sentence, quite a lot may need to happen.

The system may retrieve earlier versions of my writing and the corrections that survived. It may notice that "LLM writing" in this book does not mean one generic style defect but a family of recurring failures: compressed slogan paragraphs, over-neat contrasts, jokes replaced with respectable jokes, hedges inserted where I meant to make a claim, and wandering sentences polished until they stop wandering anywhere interesting.

It may compare the current chapter with passages I kept rather than only with a generic writing rubric. It may decide that one section needs factual checking while another needs no research at all. It may ask a second model to challenge the argument, but only if disagreement is likely to add information rather than produce a committee for ceremonial reasons. It may preserve the failed edit because the failure itself is now evidence. It may notice that the correction changes a reusable writing pattern and propose updating the pattern instead of making me rediscover the same preference three chapters later.

After all that, perhaps the system changes four paragraphs. I should not have to operate the institution that produced them.

I said:

> This chapter still feels like LLM writing.

That is **fluent autonomy**: not autonomy without structure, but autonomy in which the structure can assemble itself around the intention.

The Interface Moves Up

The argument of this book began with a recurring move: once something complicated becomes reliable enough, the layer above can start treating it as a primitive.

We stopped programming by wiring individual transistors. We stopped thinking about registers every time we wrote a high-level function. Libraries hid algorithms. Applications hid libraries. Coding agents began treating applications, files, browsers, terminals and APIs as tools.

The complexity did not disappear. It moved underneath a more useful interface.

AI agents push that abstraction one level higher because the new interface is not merely another programming language. Increasingly it is an **outcome described incompletely in ordinary language**.

That incompleteness matters. When I call a function, I am supposed to know what function I want. When I talk to another capable human, I often do not. I can say:

> This argument feels wrong.
>
> Find somewhere good for dinner.
>
> I think this customer is stuck.
>
> We need to understand why this experiment moved.
>
> I am considering changing jobs.

None of these is a specification. Each opens a small investigation.

Traditional software handles this badly because software usually requires the designer to anticipate the structure of the intention in advance. Somebody decides which fields exist, which buttons appear, which states the workflow may enter and which exceptions deserve their own branch. That predictability is useful. It is also why every mature enterprise product eventually contains a form whose existence can be explained only by an archaeological expedition through three reorganizations.

A fluent autonomous system can construct part of the structure **after seeing the intention**.

That is the shift.

Control Moves Up, Not Away

This brings us back to Chapter 1. The point of autonomy was never to remove control. It was to move control upward.

Across the book, the thing being controlled kept changing. Implementation became a task for agents. Problem solving moved into Deep Mode. System 3 moved control into evidence, trust and contact with reality. Multi-agent work made organization itself part of the design. Pattern Language moved accumulated experience into editable culture. Recursive self-improvement made some of that culture an experimental object. Scalable Oversight asked how human judgment could remain relevant when action-by-action supervision stopped scaling. Layer 4 then complicated the supposed endpoint by noticing that the human is learning too.

Fluent Autonomy is what happens when those layers stop feeling like separate products I have to operate. The complexity becomes infrastructure.

But there is a difference between **hidden complexity** and **lost control**.

A compiler hides registers from me most of the time, but I can still inspect the generated assembly when the abstraction leaks. A database hides pages and indexes until performance becomes strange. A good autonomous system should behave similarly.

Most of the time I should be able to speak at the level of intention. When something becomes uncertain, consequential or surprising, the lower layers should become visible again.

Fluency therefore requires **progressive disclosure of control**: simple when the situation is routine, legible when it is not.

Bureaucracy on the Fly

There is a phrase that sounds like an insult until you need it: **bureaucracy**.

Chapter 5 argued that bureaucracy, in its useful form, is accumulated coordination. Roles, review boundaries, logs, standards, escalation paths and procedures exist because some kinds of work become unreliable when everybody improvises everything at once.

The problem is that fixed bureaucracy calcifies. A six-person review process designed for a dangerous database migration eventually gets applied to changing a sentence in a help page because nobody remembered to tell the workflow that reality had changed.

Agent systems give us the possibility of something stranger:

bureaucracy on the fly.

The organization can be assembled for the problem rather than inherited wholesale from the previous problem.

A factual question may need one agent and a source. A difficult scientific claim may need competing hypotheses, a literature search, code, an experiment and an evaluator insulated from the researcher who wants the result to work. A writing edit may need none of that: perhaps the original paragraph, a memory of previous corrections and enough restraint to leave the sentence alone. A high-impact financial action may need very little creativity and quite a lot of permission checking. A genuinely novel research problem may need several agents pursuing different approaches without sharing enough context to collapse into one correlated opinion.

The organization should be **as large as the uncertainty deserves and no larger**.

The Society of Agents, Pattern Language and Scalable Oversight meet here. Patterns tell the system which institutional shapes have worked before. System 3 keeps those patterns answerable to evidence. The system can compose a temporary organization, run it, observe whether it helped, preserve what deserves to survive and dismantle the rest.

What used to be a workflow diagram becomes part of runtime.

The human gives the problem. The system compiles an institution.

Fluency Is Selective Friction

There is an easy mistake here. A fluent agent is not an agent that never asks questions. It is also not an agent that asks permission for every action. That is an approval workflow that has learned to talk.

Fluency means knowing **where friction belongs**.

Rename two hundred temporary files according to a convention used every week for a year?

Please do not wake me.

Send €200,000 to an account we have never seen because an email said "urgent"?

I suddenly enjoy friction.

Chapter 9 adds another reason to slow down. Sometimes friction is not about safety. Sometimes friction is the point of the interaction.

If I ask the system to teach me statistics, instantly solving every exercise is not fluent assistance — it is substitution wearing a tutor badge. If I ask for help deciding between two life choices, collapsing the uncertainty into one confident recommendation may remove exactly the thinking I needed to do. If I want a routine analysis completed, making me rediscover every intermediate step is wasted attention.

So the system has to infer not only **what outcome I want**, but **what role I want to retain in producing it**.

Human attention is scarce, but the objective is not to minimize it. Spend it where it changes the result, where the action is hard to reverse, where values conflict, where the evidence is weak, where a new failure mode appears—or where the human is trying to become more capable rather than merely get the thing done.

The best autonomous system is not the one that needs the least human input but the one that spends it well.

Invisible by Default, Legible on Demand

There is another bad version of fluency.

Everything works through one beautiful conversational box. The system performs research, edits files, transfers money, changes production settings and updates its own memory. The interface stays calm and minimalist throughout.

Then something goes wrong. You ask why, and the system says:

> I made the best decision based on available context.

This is not fluency. It is opacity with good typography.

The architecture underneath the interface has to leave traces. Which evidence mattered? Which pattern was retrieved? What alternatives were considered? Which evaluator rejected the other approach? What changed from the previous version? Which action is reversible? What uncertainty was hidden because it did not matter, and what uncertainty should have reached the human but did not?

Chapter 4 called these trust chains. Chapter 8 added more instruments. Fluent Autonomy does not make them disappear; it makes them available **when needed without requiring the human to operate them continuously**.

The surface can be conversational. The substrate should remain inspectable.

That is the difference between an abstraction and a black box.

Applications Become Primitives

What happens to ordinary software in this picture?

Probably less than the most enthusiastic agent demo suggests, and more than the current application model expects.

Menus, spreadsheets, dashboards, canvases, forms and direct manipulation are not historical accidents waiting for language models to abolish them; quite often they are excellent interfaces.

Sometimes I want Excel because seeing the table is faster than discussing it. Sometimes I want a dashboard because twenty numbers at once tell me more than twenty conversational turns. Sometimes I want to drag the object myself because my hand knows what I mean before I have words for it.

Fluent Autonomy is not the death of applications. It is the death of the assumption that **every intention must first be translated into the application structure somebody predicted in advance**.

The application becomes a primitive available to the agent and to me. If a spreadsheet is the right temporary representation, make one. If direct manipulation is better, show me the canvas. If the task is routine, use the tool and return the result. If the problem is underspecified, conversation may remain the best interface because conversation is what humans already use when neither side knows in advance exactly where the interaction is going.

The interface itself can become part of the solution.

The Architecture Gets Out of the Way

Put the pieces together and Fluent Autonomy is less magical than it first sounds.

A human supplies an imperfect intention. The system interprets it provisionally rather than pretending it received a utility function. It decides what it already knows, what needs research and what should remain uncertain. It retrieves relevant cultural memory without treating precedent as scripture. It creates the smallest useful organization around the problem, selects tools, exposes important claims to reality and allocates evaluation where error would matter. It keeps traces. It asks the human when human information has high value. It learns from correction without converting one correction into universal law. And it returns not only an artifact or action, but enough consequence that the human can learn too.

That is a lot of machinery. The point is that I should rarely have to name any of it.

The system should not require me to know whether this particular task needs debate, a critic, three independent evaluators, a circuit monitor, a retrieval pattern or no ceremony whatsoever. Those are

implementation details at the level I am trying to leave behind.

The unit of interaction becomes closer to:

> **Here is what I am trying to accomplish. Help me get there without losing contact with reality—or with me.**

Fluency is competent movement between autonomy and involvement, not maximal independence.

The system acts freely where the ground is stable, slows down where it is not, surfaces its machinery when trust requires inspection, and gives control back to the human at the level where human judgment actually matters.

Control did not disappear.

It found a better interface.

Monday Morning

There is one remaining problem with this picture.

Architecture is unusually well behaved inside a book. The examples cooperate. The agents use the tools they were supposed to use. The evaluator measures the thing the paragraph needs it to measure. No customer decides that the elegant experience is annoying. No production service has a latency budget. No old dependency turns out to be load-bearing for reasons nobody remembers.

A theory of fluent autonomy should survive contact with systems that cannot be redesigned from scratch and people who did not volunteer to participate in the metaphor.

I needed a less polite laboratory.

Fortunately, Monday morning was waiting.

Chapter 11: The Store That Builds Itself

When System 3 Came to Work

There is a danger in writing a book about future architectures. If you spend long enough drawing layers, agents, trust chains and feedback loops, eventually they all begin to behave beautifully.

Then Monday morning arrives.

I lead Applied Science for product ranking and recommendations at Zalando. That gives me a slightly unfair opportunity: I can spend the weekend writing that software should become more emergent, more compositional and less micromanaged, then arrive at work and discover that real software contains latency budgets, old interfaces, business constraints, experiments, dependencies, customers who refuse to behave like the diagram, and at least one matrix somebody created for a very sensible reason three years ago.

The book came to work.

At the time of writing, what follows is a design in progress, not a victory lap. We have not proved the grand version. In fact, one of the points of the design is to make it possible to discover that the grand version is wrong before spending two years building it. This is my account of the ideas, not a Zalando strategy announcement, and definitely not a claim that we solved shopping before lunch.

The starting problem was almost embarrassingly simple. Imagine two customers looking at the same product page.

One has visited several times across several days. She filtered by size and color, looked at alternatives, came back, switched between two candidates and now appears to be stuck near a decision. The other customer arrived thirty seconds ago from a search result. We know almost nothing about what he wants, how serious he is, or whether this is the first jacket he has seen in six months.

They can see the same recommendation modules in the same order.

That is not because the recommendation models are stupid. Quite the opposite. Mature recommendation systems can contain excellent retrieval, ranking, personalization, embeddings, sequence models and business logic. The strange part is one layer above them. We may have sophisticated intelligence inside each box while the arrangement of the boxes is mostly predetermined.

The page is smart inside the modules and surprisingly dumb between them.

This looked familiar. Chapter 1 began with a claim about emergence: once a complicated thing works reliably enough, the layer above can start treating it as a primitive. Chapter 3 made the same move with coding agents and applications. Chapter 6 did it with executable knowledge. Now I had a recommender system full of increasingly capable primitives and a question I had somehow spent an entire book preparing myself to ask:

What should the layer above do with them?

After Chapter 5, I can give the answer a sharper shape. The ambition is not merely to put an AI orchestrator above a recommender system — it is to make more of the store behave like a **scientific institution embedded in the product**. Customer problems are hypotheses. Recommendation experiences are interventions. Experiments and downstream behavior are evidence. Traces preserve provenance. Problem catalogs and patterns accumulate what survived. Unmet demand is an anomaly signal. The scheduler allocates attention across competing explanations of what the customer needs.

That does not make shopping a laboratory or customers experimental subjects in the cartoonish sense. It means the architecture should be able to **form beliefs about its own failures, intervene, observe consequences, revise those beliefs and preserve what it learns**. The product stops merely executing a model and joins a continuing inquiry into how to help.

Stop Recommending for a Moment

The conventional recommendation question is usually some variation of:

> Which products should I show this customer?

It is a very good question. Entire fields exist to answer it better. Retrieval finds candidates. Ranking orders them. Sequence models infer interests. Business rules remove things that should not be there. The machinery can become extremely sophisticated.

But consider the customer who is switching between the same two pairs of trail shoes for the fourth time.

What does she need? Perhaps more trail shoes. Perhaps not.

There is a point at which another excellent candidate is not help. It is homework.

She may already have enough choice. Her problem could be that she cannot compare the two choices she has. Or that she does not trust the unfamiliar brand. Or that she cannot tell whether her normal size will fit. Or that one shoe costs more and she cannot see what she gets for the extra money.

Once you phrase it this way, the object being predicted changes.

Instead of asking only which *item* is relevant, we can ask which **bounded problem** is currently relevant.

Comparison friction. Size anxiety. Return hesitation. Quality uncertainty. Outfit visualization. Filter fatigue. Decision paralysis.

These names are not truths hiding inside the customer's head. They are hypotheses about difficulties we may be able to detect and, more importantly, do something about.

That last condition matters. I can invent an exquisitely named psychological state for every wiggle of the mouse, but if we cannot observe it well enough to test and cannot build anything that plausibly helps, we have created a taxonomy department rather than a recommender system. The problems have to be bounded enough to attack.

Chapter 2 had circles and an immutable evaluator. Shopping is messier, but the discipline is similar. Define a problem narrowly enough that an intervention can succeed or fail. If we claim somebody has comparison friction, we should eventually be able to ask whether comparison-like behavior diminished after we addressed it. If we say size anxiety is the blocker, we need evidence that the signal means something and a metric that can tell us whether our intervention helped rather than merely attracted a click.

Here the architecture started moving away from the familiar funnel.

People Refuse to Stay in the Funnel

Funnels are useful because humans like diagrams that get narrower toward the bottom. Explore. Form a need. Narrow. Evaluate. Decide. Purchase. The arrows point downward, everybody feels organized, and somewhere a PowerPoint theme earns its salary.

Customers are less cooperative. Someone can be evaluating one product while exploring another category. She can be price-sensitive and size-anxious at the same time. She can know exactly what dress she wants and still be unsure whether it works with the shoes she already owns. She can add something to the basket, remove it, return to the product page, read reviews, open a size chart and then disappear for three days because a child needed dinner.

A single lifecycle stage compresses this mess into one label. The design we began working with uses something richer: a **problem fingerprint**. Instead of saying the customer *is in Evaluate*, the system can represent several problem hypotheses at once, each with an intensity. Size anxiety may be high. Return hesitation moderate. Outfit seeking almost absent. Another customer on the same product may have the reverse pattern.

The fingerprint is not a personality test — it is local to the customer, the current context, the surface and the available evidence. That is important because I do not want the system deciding that Hani is metaphysically a `RETURN_HESITANT_PERSON` and carrying that fact around until retirement. Some characteristics are durable. Many are situational.

The architecture also separates the machine representation from the stories humans use to think. Designers and scientists may organize problems by funnel stage, mission, timing or recognizable archetype. Those lenses help us notice gaps and invent hypotheses. The runtime system does not need to believe the story. It needs signals, a problem fingerprint and a way to test whether the resulting behavior is useful.

I like this separation because it protects us from one of the oldest mistakes in machine learning: turning a useful human abstraction into an ontological claim because we happened to put it in a feature table.

The customer is not the funnel. The funnel is one way we look at the customer.

A Library of Ways to Help

Once you define demand as problems rather than slots, the supply side changes too.

Today, when people hear "recommendation," they often picture a ranked list of products. You may also like. Similar items. Complete the look. Recently viewed. The carousel has become the fruit bowl of ecommerce: you can put one almost anywhere and nobody asks too many questions.

But if the problem is comparison friction, a ranked list may be the wrong species of answer. The useful experience could be a comparison between the two products the customer is actually considering. If the problem is size anxiety, the useful thing may be evidence about fit. If the customer cannot imagine an outfit, it may be a generated collage. If she has only a vague mission, perhaps a product finder is better. If she knows exactly what she wants but the catalog is overwhelming, maybe the right action is a guided filter. Sometimes the answer is another set of products. Sometimes the answer is information. Sometimes it is a different interaction entirely.

I started calling these reusable units **recommendation experiences**, or RXs. The name matters less than the abstraction. An RX is more than a model: a reusable capability that knows roughly what kind of problem it can address, when it is eligible to run, how it can be configured and how it presents itself.

The long-term ambition is a large library: carousels, comparisons, outfit builders, collages, finders, confidence modules, explanations, visual exploration, complementary-item experiences and things we have not invented yet. But the point is not to celebrate having hundreds of widgets. A library of two hundred overlapping experiences is just a new kind of legacy system with better animation.

The design principle is **composition over invention**.

When a new need appears, first ask whether an existing experience can meet it with a different configuration. A Similar Items experience might be generic in one context and constrained to products available in the customer's size in another. A comparison component can compare different attributes depending on what matters in the current session. A collage can be anchored on a dress, a pair of shoes or an occasion without becoming three separate products in the organizational sense.

Build for the hundredth experience, not the first.

Versatility stops being a slogan here and becomes an architectural property. The more that useful behavior can be produced by configuring and composing a smaller number of strong primitives, the less the organization has to encode every new situation as another permanent branch in software.

I spent years in machine learning hearing that the answer to complexity was to learn rather than hand-author. Then, like everyone else, I helped build systems where the model learned beautifully inside a box surrounded by hand-authored configuration. The box was not the end of the learning problem.

Composition Is Not Ranking With a New Hat

At this point the obvious response is: fine, rank the experiences. That gets us part of the way and then breaks in an interesting place.

Suppose the system has already placed a strong size-confidence experience at the top of the page. Should another size-related module receive the same score it would have received before the first one was shown?

Probably not. Some of the problem has already been addressed. A second module may add little and consume valuable attention.

Now suppose a returns-clarity experience is more useful *after* fit evidence because the two together form a coherent decision aid. Its value may increase after the first experience appears.

The score of an experience therefore depends partly on what has already been selected. That is composition.

The composer has to select experiences, configure them, order them and deduplicate not only repeated products but repeated *help*. It needs some notion of saturation: two size widgets can be one too many. It can model synergy: one experience may become more valuable after another. It should account for position cost because the top of a page is expensive real estate and a wonderful module in slot twelve may be a philosophical achievement rather than a product one. Constraints matter too, but I prefer many of them to be visible pressures rather than a secret forest of `if DE_mobile && campaign_X` rules.

Most importantly, the **page becomes the unit**. A module can win its local metric and make the page worse.

This is easy to forget because teams and models naturally acquire local objectives. Increase CTR on this carousel. Improve conversion from that module. Raise engagement with this block. All reasonable. But if one module steals a click the customer would have made anyway, we may have moved attribution without creating value. If three individually successful widgets all solve the same problem, the page can feel like a committee where everybody prepared the same presentation. The layer above has to reason about the composition as a whole.

And this is where the case study started resembling Chapter 5's society of agents. A society is not improved merely by hiring the best individual expert in every discipline. Somebody still has to decide which experts are needed, how they interact, what has already been covered and when another voice adds information rather than noise. A page can have the same problem.

Mei Does Not Need More Shoes

Take a concrete customer. Call her Mei. Mei has two pairs of trail shoes open. She has returned to them several times across five days. She switches between the two pages quickly, saved one of the shoes and is spending less time reading each page because by now she has probably memorized half the product description.

A conventional recommender can still do an excellent job here. It can find twenty more trail shoes that look similar, match her taste and are available in her size.

But suppose the fingerprint says comparison friction is high and price-quality confusion is moderate. The composer can do something different. The first experience compares the two shoes Mei is actually deciding between on attributes relevant to her behavior. The second adds confidence evidence from

customers or product information that helps resolve the remaining uncertainty. Generic similar-items may still survive because it has useful standalone value, but it moves down.

She is not shown more choice. She is shown a way to close the choice she already has.

That sentence changed how I thought about recommendations.

For years, the field has been extraordinarily good at finding things. Search finds things. Recommenders find things you did not ask for. Retrieval systems find things at absurd scale. But shopping is not only a retrieval problem. At different moments it is also a comparison problem, a confidence problem, a visualization problem, a constraint problem and occasionally a "please stop showing me another black sneaker" problem.

A system that can only respond with more items is like a doctor who has one extremely accurate prescription and keeps waiting for every disease to become the disease it treats.

The same point becomes even clearer with another customer.

Sami Does Not Need a Click

Sami has selected a size but has not added the product to his basket. He opened the size chart twice. It is a brand he has not bought before. Perhaps his current problem is size anxiety, with some return hesitation behind it.

One useful response might not be shoppable at all. Imagine a small evidence module explaining how people with comparable sizing histories tended to fit this item, or giving a properly substantiated signal about whether buyers kept their usual size. The exact claim matters enormously because a false fit claim is worse than a mediocre recommendation. But conceptually this is a different kind of RX: it provides **knowledge**, not another candidate.

Now try to optimize the whole system for expected click. The insight module is in trouble.

If it works perfectly, Sami may read it, become confident and press Add to Bag. The module itself may receive no click. A carousel with attractive shoes can collect engagement more easily while being less relevant to the thing stopping him.

This is a small example of a much larger problem: the objective determines which species of intelligence can survive. If your ecosystem rewards clicks, clickable organisms evolve.

The architecture therefore needs different value terms and different evidence standards for different experiences. Item recommenders can be judged partly by engagement and downstream action. Insight experiences may need read-through, decision confidence, return behavior or problem-specific outcomes. Claims need substantiation thresholds. Some experiences are cheap to be wrong about. Others can mislead a customer or create regulatory risk. The library is heterogeneous because the problems are heterogeneous.

And now Chapter 4 comes back: where did the claim come from, how strong is the evidence, what kind of knowledge is this, and how much trust should the system place in it before acting?

System 3 is no longer a chapter about hallucinations. It is a product requirement.

The Honest Cold Start

There is another customer I like because she reveals whether the architecture can resist pretending.

Lea arrives from a social link. No account. No history. Almost no session depth. The system has the product she opened, perhaps the season, approximate location and a few ambient signals. That is it.

A personalization system can react to this situation in two ways.

One is to panic quietly and run a generic fallback while still speaking in the confident dialect of personalization.

> Picked for you.

Based on what, exactly? Her IP address and our enthusiasm?

The other is to treat low signal as a normal state with its own design. Lean on the anchor, season and population-level evidence. Prefer experiences with strong standalone value. Frame them honestly. "Popular this week" can be a good statement when "we have inferred your soul from one click" is not.

This is what I mean by graceful degradation. Cold start is not necessarily an error. If a large fraction of requests arrive with weak signal, the low-signal path may be the product and deep personalization the special case.

The architecture should know what it does not know. That sounds obvious until you look at how much software is built around pretending the common messy case is an exception handler.

The Trace Is Part of the Intelligence

Dynamic systems create a governance problem immediately.

A static page is relatively easy to inspect. This module goes here. That one goes there. If something looks wrong, somebody can open the configuration and complain about whoever last touched it.

A composer makes a fresh decision from context. Now a customer reports a terrible page and the first debugging question becomes:

> Why did this page exist?

"The model chose it" is not an answer. It is a resignation letter written in passive voice.

So every composition needs a trace.

Which signals were read? What problem fingerprint was inferred? Which experiences were eligible? Which were not? How were they configured? What scores did they receive? Which constraints mattered? What won? What lost? Which version of the composer produced the decision?

The losers matter more than they first appear. If we log only what we served, we can attribute outcomes to the winner but we lose much of the decision context. We cannot tell whether an experience was absent because it was ineligible, starved by the objective or simply scored slightly below another. We cannot replay the decision properly. We cannot compare a new policy against the old choice set without reconstructing a world we chose not to record.

Logging the loser set does not magically give us causal counterfactuals. Reality is not that generous. But it gives us the archaeology of the decision.

This is exactly the move System 3 has been making throughout the book. Do not preserve only the polished conclusion. Preserve enough of the chain that future systems can inspect why the conclusion deserved trust.

The trace also changes development. You can build a simulator that replays saved scenarios. You can ask which experiences would be eligible in a context or which contexts a new experience could serve. You can run regression suites over scenarios before changing the library. A dynamic system becomes safer not because it stops changing but because its changes become replayable.

You cannot govern what you cannot replay.

From Machine Learning to Knowledge

Somewhere around here the project stopped looking to me like a normal recommendation-system redesign.

The models still matter enormously. We need representations, retrieval, ranking, sequence understanding, problem detectors, value models and probably more machinery than I can fit into a chapter without losing several readers to a sudden interest in gardening.

But the durable asset begins to include something else.

A problem catalog. A library of reusable experiences. Knowledge about which experiences address which problems. Eligibility conditions. Evidence requirements. Presentation strategies. Scenarios. Traces. Regression tests. Guardrails. Rules for when an experience should be retired.

This is the Pattern Language chapter wearing an ecommerce badge.

An experience is useful not merely because somebody built a clever model for it. It becomes useful organizational knowledge when we know the recurring situation it addresses, the evidence that should trigger it, the conditions under which it fails, the other experiences it complements or duplicates and how its value should be measured.

A new comparison module without that context is a feature. A comparison pattern with evidence, boundaries, history and known interactions is culture.

And culture has the same failure mode we saw earlier: it can become a junk drawer with tenure.

If every newly observed problem creates another RX, the library eventually recreates the configuration matrix in a more colorful form. So new supply needs a gate. Is the problem real? How large is it? Can an existing experience be configured to address it? Where does the current library have weak coverage? Which experiences stopped relieving the problems they were created for and should disappear?

This led to a pair of concepts I particularly like: **Coverage** and **Unmet Demand**.

Coverage asks, at design time, which known problems the current library *could* address. Unmet Demand asks, from production, which detected problems remained insufficiently addressed after composition.

Put them together and the roadmap starts to emerge from the system's own failures. That is a very different way to decide what to build next.

Seen through the Chapter 5 reveal, Coverage and Unmet Demand are more than roadmap metrics. They tell the institution where its current theories and instruments are weak. A recurring problem with no effective RX is an anomaly the product cannot yet explain away; a heavily used intervention that stops relieving the problem is a theory losing contact with reality. The roadmap becomes partly a **research agenda generated by the failures of the current system**.

Let the LLM Narrate. Do Not Let It Declare Reality.

AI can help with problem discovery too, and this is where it becomes very easy to fool ourselves.

Imagine replaying anonymized customer sessions and asking a strong language model to narrate what appears to be happening. The customer compared three products, opened the size chart, returned to one PDP, removed an item from the basket and left. The model can generate a plausible diagnosis. Cluster enough narrations and you may discover recurring forms of friction that your existing taxonomy missed.

This is useful. It is also dangerous for exactly the reason Chapter 4 exists.

Language models are plausible by construction. That does not make the narration true.

"The customer hesitated because of fit" may be an excellent story. The customer may also have received a phone call.

So narration should generate hypotheses, not production truth. Take a sample. Compare the diagnosis with interviews, surveys, support contacts or other evidence closer to the customer's actual experience. Build a detector only after the hypothesis survives contact with something outside the model's coherence. Define what success looks like before the detector starts steering the page.

The same rule applies to observational analysis. Customers with comparison friction may convert less, but perhaps weaker-intent customers simply compare more. Correlation can prioritize what to investigate. Only intervention tells us how much of the outcome the problem was actually causing.

I find this satisfying because the architecture does not merely *use* System 3. It needs System 3 to avoid hallucinating its own customers.

This is the book's central thesis in work clothes. The LLM is excellent at generating explanations. The product architecture has to decide which explanations deserve pursuit, construct interventions that expose them to consequences, preserve the chain of evidence, and update the repertoire when the world refuses to cooperate. **Philosophy of science has become product architecture.**

The Objective Fights Back

Eventually the design forced us to name the thing the composer is supposed to optimize.

We used the deliberately bland term **Surface Value**. This is where the project becomes philosophical against its will.

If Surface Value is module CTR, we have not solved the page problem. If it is total clicks, a page full of shiny modules may win while the customer gets nowhere. If it is immediate purchase probability, experiences that build confidence or improve a longer mission may be undervalued. If it is revenue, expensive products get interesting very quickly. If it is margin, the store's objective can start eating the customer's. If it is long-term value, we have gained a beautiful phrase and several years of causal-inference work.

The objective has to be page-scoped enough that compositions can be compared, but decomposable enough that we can diagnose why a page helped or failed. Different problem classes need their own success signals. If we address comparison friction, does the comparison behavior decrease? If we address size anxiety, do customers progress with fewer signs of uncertainty and without creating a return problem later?

This is Layer 4 in production. What do we actually want?

The store has legitimate business goals. Customers have goals. They are often aligned and sometimes not. Inventory has constraints. Merchandising exists. Margin exists. Availability exists. Regulators exist. A system that pretends only one of these matters is not simpler; it is hiding politics inside a scalar.

The goal is not to discover the One True Ecommerce Reward Function carved into a mountain somewhere outside Berlin.

It is to make the trade-offs explicit enough to test, govern and revise.

This is why I increasingly dislike architectures where business decisions enter through invisible overrides. If merchandising needs a lock, make it a typed constraint. If margin is part of the objective, admit it. If a claim needs compliance review, attach the evidence rule. If the system violates a soft constraint because another objective dominated it, log the violation.

The architecture should not make disagreement disappear. It should make disagreement inspectable.

Bounded Ambition

After all of this, the sensible first experiment is obviously to build hundreds of widgets, a general customer-reasoning model, a cross-surface scheduler and an autonomous agent that redesigns fashion retail by Thursday.

We did not do that. The first test is deliberately boring.

One placement: the product page. A small number of validated customer problems. The existing recommendation library, with only limited new supply. A simple composition mechanism. A trace good enough to explain an individual decision. An authored objective before a learned one.

Why so narrow? Because if we invent a new library of experiences and change the selection mechanism at the same time, then run an experiment and get a flat result, we have learned almost nothing. Maybe the composer is bad. Maybe the new experiences are bad. Maybe both are good and the measurement is bad. Maybe the static page was already fine and I should have spent the quarter learning the guitar.

A bounded test separates the claims. Does dynamic composition beat a strong static baseline? And importantly: does it beat simplification?

That second competitor is easy to underestimate. Perhaps the best response to an overloaded page is not a brilliant composer. Perhaps it is fewer things. The system should have to earn its complexity against the possibility that removing modules produces a better customer experience.

I love this part because it keeps the book honest. A philosophy of emergence should be willing to lose an A/B test.

Otherwise it is not a philosophy of experimentation. It is branding.

And if System 3 is science, this is not merely rhetorical humility. **The architecture must contain a route by which the book's own theory can lose.** The A/B test is not there to validate the philosophy; it is there to threaten it.

When the Page Stops Being the Product

Suppose the narrow test works. Then the interesting version begins.

The library grows beyond carousels into richer experiences: comparisons, collages, product finders, outfit builders, confidence modules, visual exploration and whatever else proves useful. Configuration becomes richer so one experience can serve several contexts without a matrix of handcrafted variants. Problem discovery improves. Unmet demand exposes missing capabilities. The composer learns a better objective. Different surfaces begin to share a coherent read of the customer's current mission.

At that point, the word *page* starts to become suspicious. Why should the product page always contain the same conceptual structure?

Why should a customer with a decision problem receive the same interface as somebody exploring for inspiration? Why should the home surface, product page, basket and later email behave like four organizations with partial amnesia if the customer is still pursuing one mission?

The more capable the library becomes, the more the system can schedule **problems and interventions**, not merely modules and slots.

A customer starts with a vague request for a wedding outfit. The system helps narrow the style. A collage makes one direction concrete. Seeing it changes what the customer wants. The problem shifts from exploration to comparison. A product finder resolves a constraint. A size question appears. The scheduler brings in fit evidence. The customer buys the dress but not the jacket. Later, a different surface may continue the unresolved part of the mission.

There was never a hard-coded `WEDDING_FUNNEL_V7`. The journey emerged from bounded problems, reusable capabilities and changing evidence.

The hundreds of widgets stop being a UI roadmap here and become a **vocabulary of action**. The interface is the current projection of the problem-solving process.

That does not mean every pixel should be generated by an LLM. Predictability matters. Accessibility matters. Design systems matter. Latency matters. Customers occasionally just want to buy socks without participating in an artificial-intelligence research program.

Fluent autonomy is selective. The machinery should become dynamic where dynamism earns its cost and remain boring where boring is excellent.

But the direction is different from the old model of product development. Instead of predicting every useful journey in advance and encoding it as a fixed interface, we construct a repertoire of trusted capabilities and let the higher layer assemble them around the problem in front of it.

The store does not literally build itself. It learns how to build more of the experience it needs.

The Book Comes Back to Bite Me

I began this project as a recommendation-system redesign. Then the chapters started appearing inside it.

Emergence: stop specifying every context and let useful compositions arise from primitives.

Bounded problems: diagnose something narrow enough to test rather than "optimize shopping."

Versatility: configure a smaller repertoire instead of multiplying bespoke experiences.

System 3: preserve evidence, traces and boundaries so dynamic decisions can be trusted.

Society: coordinate specialized capabilities rather than worship one universal model.

Pattern Language: turn recurring successful responses into reusable operational knowledge.

Automatic alignment research: use sparse customer and human feedback to discover where the system's behavior or repertoire is wrong.

Layer 4: admit that the objective is uncertain, plural and capable of changing while the interaction unfolds.

Fluent autonomy: hide most of that machinery from the customer and surface the right form of help when it matters.

Chapter 5 now gives me a more compact description of the entire list: **build a scientific institution around the customer problem.** Not a lab coat pasted onto ecommerce. An architecture that can generate competing explanations, choose which are worth testing, intervene through reusable capabilities, expose those interventions to consequences, remember what survived, preserve disagreement where it carries information and revise its own problem vocabulary when anomalies accumulate.

I had spent nine chapters arguing that these ideas belonged together. Then I walked into a recommendation problem and found myself rebuilding the same architecture because the old abstraction stopped scaling.

That does not prove the book. It is one case study, in one domain, at one moment, and it may fail in several educational ways.

But it changed the question for me. The important future system may not be the model that predicts the next product best. It may be the system that can discover what kind of problem exists, recruit the right capabilities, construct an intervention, inspect whether it helped, learn from the gap and change what it does next.

And once you can imagine that happening in a store, it becomes difficult not to imagine it happening everywhere else.

Software.

Research.

Education.

Organizations.

Government.

Our own decisions.

Which creates a problem larger than any recommender system.

If AI keeps moving upward—if it increasingly discovers problems, selects strategies, builds solutions and turns experience into reusable knowledge—then asking what *the AI* should do is no longer enough.

We have to ask what happens to us when capacity itself changes.

That is not a software architecture question — it is the beginning of another philosophy.

Chapter 12: After Capacity

A Glimpse of Double Descent Life

The previous chapter ended with an uncomfortable possibility.

What if AI does not merely answer more questions or automate more tasks, but steadily moves upward through the stack? It retrieves, ranks, composes, diagnoses the problem, chooses a strategy, builds the tool it needs and learns from the result.

At each step, something that used to require scarce human capability becomes infrastructure for the layer above.

This book has mostly treated that as an architectural problem. How do we make autonomy useful? How do we keep it connected to evidence? How do agents coordinate, remember and remain corrigible as both the world and the human change?

But there is another question behind all of them.

What happens to human life when **capacity itself becomes much cheaper**?

Not all capacity. We will still have one planet, finite land, finite energy, twenty-four hours in a day, and restaurants that somehow remain fully booked exactly when you want to go. Bodies remain bodies. Politics does not evaporate because a model can write Python. Scarcity is not going to receive a polite email from OpenAI and retire.

But cognitive capacity is already becoming strange enough to force the question.

A person can enter a field she never studied and get a useful map in an afternoon. A small team can produce software that previously required a much larger one. Research, design, analysis, translation, tutoring, programming and increasingly complicated forms of planning can be amplified by systems available to people who did not spend twenty years acquiring every underlying specialty.

The usual story jumps directly from *humans do the work* to *AI does the work*. Then we spend the rest of the conversation asking what humans will do with all the suspiciously abundant free time.

There is a third possibility: we keep doing things. We just start doing things that were previously economically ridiculous.

I have been calling the larger philosophy around this **Double Descent Life**. The name is less important than the movement it describes.

One descent happens outside us. Implementation, expertise and coordination become cheaper and

move downward into infrastructure. Things that once consumed years of training or layers of organization become building blocks.

The other happens inside us. As practical difficulty falls away, the remaining questions become less technical and more human. What do I actually want? Which desires are mine? What is worth doing when more things become possible? Which institutions deserve authority? What kind of life do I want to commit to if commitment is no longer forced mainly by scarcity?

The machines descend into implementation.

We descend into meaning.

That does not sound easier, because it is not.

This chapter is only a glimpse of that philosophy. I do not have a neat doctrine, and I am suspicious of neat doctrines anyway. The history of thought is full of people who reached page 300 and announced that history had finally arrived at the correct system, usually just before history did something rude.

So consider this a map of the terrain after capacity, not a constitution for the future.

When Difficulty Becomes Infrastructure

Human institutions were built under assumptions about what is difficult.

Writing good software is difficult, so we organize teams of specialists around it. Scientific expertise is difficult to acquire, so we create universities, journals and long apprenticeships. High-quality legal or financial analysis is expensive, so access is uneven. Producing media is costly, so publishing institutions decide what gets distributed. Coordinating a large organization is difficult, so we create layers of management whose main superpower is knowing which meeting another meeting should produce.

Scarce capability shapes power.

If I cannot build something myself, I need somebody who can. If one organization owns the machinery, data, expertise or distribution required to act, then access to that organization becomes valuable. We spend a surprising amount of human life acquiring permission from structures that exist partly because doing the thing directly is too expensive.

AI changes some of those costs. This does not automatically flatten society. A technology that increases capacity can also increase concentration. The company with the best models, compute, data, distribution and capital may gain more power, not less. Cheap software can empower a teenager in Amman and a surveillance state at the same time. Capability has never come with an ethical direction preinstalled.

Still, something important happens when the cost curve moves. If an individual or small group can increasingly research, design, build, analyze and operate things that previously required a much larger institution, then some problems that looked like power problems may turn out to have been **capacity problems wearing a suit**.

You wanted software tailored to how your team actually works, but building it was too expensive, so you bought a generic SaaS product and reorganized the team around the dropdown menu. You wanted a course that teaches exactly what you need at exactly your level, but producing one teacher

per student was impossible, so thirty people entered a room and agreed to move at approximately the same speed. You wanted to test a policy idea, but the analytical machinery was too expensive, so the argument remained mostly rhetorical.

When capability becomes cheaper, the design space opens—not infinitely, equally or safely by default, but enough that the old question, *Who controls the scarce machinery?*, is joined by another:

How much of that machinery can we move closer to the person who needs it?

That is where capacity begins to compete with power as a way of getting things done.

Bespoke Comes Back

Software gives us a useful example because we have already lived through two economic modes.

The first was bespoke software. If you had enough money, somebody built the thing for you. Banks had their systems. Airlines had theirs. Governments had theirs. Large companies employed armies of engineers to encode their peculiarities into software because those peculiarities were valuable enough to justify the cost.

Then software became a service. This was an enormous improvement. Instead of every company building payroll, CRM, project management, analytics, communication and twenty other systems from scratch, somebody could build one good product and sell it to millions of people.

But scale has a price. To serve millions of people, the product has to become somewhat generic. The strange needs of one team become feature requests. The software acquires configuration menus, plugins, workflows, permission systems and eventually an enterprise tier whose main feature is that somebody will answer your email.

Then organizations start adapting themselves to the software.

There is a third mode hiding behind AI: **bespoke comes back, without necessarily bringing bespoke economics with it.**

Not a toy script. Not "I asked ChatGPT to make a calculator." I mean systems that would previously have been too specific to justify building at all.

A scientist may construct a research environment around one question, use it intensely for three months and throw most of it away when the question changes. A teacher may build an entire interactive world for one class because those particular students are stuck on those particular ideas. A small company may create internal software whose assumptions match the company instead of spending two years teaching the company to behave like Salesforce. A family may have tools built around how that family schedules, learns, travels, budgets and remembers things, with exactly zero concern for whether the addressable market justifies Series A.

Some of these systems may serve a thousand people. Some ten. Some one. That used to sound economically absurd. It may become normal.

This changes the human role in a way the automation story tends to miss. The future is not necessarily:

humans build → AI builds → humans watch.

We may remain intensely involved precisely because building becomes more interesting when the distance between imagining something and making it real collapses.

I do not build only because the machine cannot. I build because I want the thing to exist.

The human contribution moves upward: choosing the strange problem, forming a taste for what good looks like, combining ideas that normally live in separate professions, seeing the result and saying *No, that is not it*, then pushing somewhere neither the original prompt nor the original system anticipated.

This is Chapter 1's abstraction ladder reaching economics. We do not disappear when implementation becomes a primitive. We inherit implementation as another building block.

The interesting human may therefore not be the person guarding the last task the machine cannot perform. She may be the person who can suddenly instantiate **far more of what she can imagine**.

That is a much more attractive future than becoming the residual labor category in an automation spreadsheet.

Learning at the Speed of Curiosity

There is another kind of capacity that may change even faster: learning.

For most of history, expertise was expensive partly because knowledge had terrible interfaces.

Suppose you wanted to enter a new field. First you needed the vocabulary. Then the introductory material. Then you discovered that the introductory material assumed another field. You found a book. The book assumed notation you did not know. You searched for an explanation. The explanation used different notation. Eventually, six weeks later, you understood enough to discover that your original question was badly formed.

This friction did something useful: it produced depth. It also killed an enormous amount of curiosity before depth had a chance to happen.

AI changes that bargain. I can ask a stupid question immediately, then a more sophisticated stupid question. Ask for the intuition, then the mathematics, then the objection, then the historical argument, then why the proof needs that assumption. I can make the explanation use concepts I already know. I can ask one field to explain another. I can have the machine invent exercises, challenge my understanding, translate notation, simulate the system and show me what changes when I violate an assumption.

The cost of getting the **map** has collapsed. That does not mean I have walked the territory.

AI can make us broader without necessarily making us deeper. It can make it possible to move through mechanism design, philosophy of science, biology, constitutional theory and compiler construction at a speed that would previously have required several lives—or at least several abandoned PhDs.

That breadth can be real and valuable. It can also produce a new kind of bullshit.

A person can acquire the vocabulary of five fields and mistake fluent traversal for mastery. The model can remove exactly the friction that used to reveal where the hard parts were. You can understand a proof when somebody explains every step and discover, rather painfully, that you cannot produce

the proof. You can discuss a research area intelligently and still lack the tacit knowledge of somebody who spent ten years watching ideas fail.

You can acquire the map without any scars from the roads.

I do not think the answer is to restore the friction artificially. The answer may be a different learning rhythm:

Explore broadly. Descend selectively.

Use AI to cross fields cheaply, test curiosity, build enough understanding to see connections and decide what deserves more attention. Then, when something matters, go down. Read the primary paper. Derive the equation. Write the code. Run the experiment. Try to prove the thing yourself. Talk to the person who actually does the work.

Let reality make the lesson expensive again.

This is System 3 applied to learning. AI gives us extraordinary access to synthesis; System 3 reminds us that synthesis and justified knowledge are not the same thing.

That trade may change what an educated human looks like. The twentieth-century ideal often rewarded specialization: know one vertical deeply enough that people in neighboring verticals stop understanding you. The AI-assisted human may become more T-shaped, -shaped, octopus-shaped—choose your consulting diagram. Broader, faster at entering unfamiliar domains, more willing to combine ideas that institutional boundaries kept apart, while still going deep where the stakes or fascination justify it.

That does not make expertise obsolete. It may make expertise more deliberate.

And there is a creative consequence. A machine-learning scientist can learn enough philosophy to steal a useful structure. A philosopher can prototype the mechanism she has been describing. A doctor can interrogate statistics interactively. An artist can build software. A local policymaker can simulate an intervention instead of merely arguing about it.

Fields become more permeable. People become more dangerous in the nicest sense.

The Ideology Vortex

If all this new capacity were arriving in creatures with newly installed value systems, the next part would be easier.

Unfortunately, the creatures are us.

There is a story we like to tell about intellectual history because stories prefer arrows.

First there was the premodern world: religion, tradition, inherited authority, myth.

Then modernity arrived: reason, science, universalism, institutions, progress.

Then postmodernism arrived carrying a small hammer and began tapping on every universal claim to see what was hiding inside it: language, context, power, contingency, who got to define the categories in the first place.

Then, presumably, something comes after.

The problem with this story is that nobody informed actual humans. We did not uninstall the previous operating system.

A person can demand randomized evidence for a medical claim, ask her mother for a blessing before a major decision, read a horoscope for entertainment, manage a team using dashboards, believe deeply in national mythology, quote a postmodern philosopher about constructed categories and then become furious because somebody used the wrong definition of a sandwich.

Entire societies work this way. Semiconductor fabs coexist with ancient identities. Bayesian inference coexists with rumor. Universities teach critical theory while their admissions systems produce precise numerical rankings. A company can run sophisticated causal experiments in the morning and make a major organizational decision in the afternoon because one senior person "has a feeling."

I call this the **ideology vortex**. Not because every worldview is equally true. They are not. Reality remains annoyingly capable of rejecting bad engineering regardless of how socially constructed the bridge feels on the way down.

The vortex means that several modes of knowing and valuing operate at once. Premodern traditions carry identity, ritual, inherited meaning and forms of belonging that modern rational systems often underestimate. Modernity gives us the extraordinary machinery of science, verification, law and universal claims. Postmodern critique reminds us that institutions and categories are not neutral merely because somebody printed them in a table. Pragmatism asks whether the thing actually works. Bayesianism offers a disciplined language for uncertainty. Markets coordinate some kinds of distributed information. Democracies create legitimacy in ways a loss function cannot.

Each mode sees something the others can miss. Each can also become ridiculous when asked to do every job.

Science is extraordinarily good at questions reality can adjudicate. It is less good at deciding which trade-offs a society should consider legitimate. Tradition can preserve hard-won social knowledge, and preserve injustice with the same impressive durability. Markets coordinate preferences and information, but prices do not encode every value we care about. Democratic institutions create legitimacy through participation and contest, but anyone who has watched a parliament knows participation and wisdom are not synonyms. Permanent critique can expose hidden assumptions until it becomes incapable of committing to anything except the superiority of critique.

Humans switch among these modes without waiting for permission from philosophy. AI enters *that* world—not the clean one in which everybody has a coherent utility function, a shared epistemology and a calendar invitation for the social contract.

There is a comforting fantasy that sufficiently intelligent AI will dissolve ideological conflict. Give everyone better information and surely the disagreements shrink.

Some will. Others will get better lawyers.

A powerful model can help a scientist interrogate evidence. It can also help a conspiracy theorist construct a more coherent conspiracy. It can make propaganda cheaper, criticism sharper, religious interpretation richer, policy analysis more sophisticated and advertising more personal.

More intelligence does not guarantee one worldview. It increases the capacity available to worldviews.

And AI introduces a second reason the vortex matters: the technology itself is strangely compatible with ambiguity.

I have a sentence that gets me into trouble:

Gradient descent is the answer to Derrida.

This is deliberately unfair to Derrida and possibly to gradient descent.

I do not mean that an optimizer disproved postmodern philosophy. It would be a remarkable conference paper if it had. I mean something narrower.

A great deal of twentieth-century thought exposed how unstable language becomes when we demand perfect fixed meanings. Words depend on other words. Context changes interpretation. Categories carry history. Attempts to construct final symbolic foundations keep discovering the things they left outside.

One response is despair: if meaning is contextual, contingent and messy, perhaps rigorous computation has a problem.

Engineering found a stranger response. We built machines that operate inside the mess.

Large language models do not begin by fixing every word to an eternal definition. They learn from use, relation, context and enormous numbers of imperfect examples. Optimization pressures the system toward behavior that works often enough under the training and evaluation environment. Meaning remains fuzzy at the edges.

The product ships anyway.

Gradient descent did not defeat ambiguity. **It made ambiguity computationally useful.**

Then, immediately, we rediscover why modernity existed. A model that can move beautifully through fuzzy language can still hallucinate a citation, miscalculate a number or confidently tell you that a camel lives in Croatia. System 3 brings verification back through another door.

This is why I do not think the ideology vortex is a bug we eventually fix by choosing the winning epistemology. We need different modes for different jobs. Some claims deserve hard empirical boundaries. Some institutions need legitimate contest rather than a mathematically optimal answer. Some identities and commitments are constructed without therefore being fake.

The childish response to contingency is to pretend our constructions are eternal. The adolescent response is to discover they are constructed and conclude that nothing deserves commitment.

There is another possibility.

Construct them knowingly.

Build institutions while remembering that institutions can be rebuilt. Love people without needing a theorem that proves love is the globally optimal allocation of Tuesday evening. Choose a project, a city, a profession, a community, a way of living—and retain enough humility to revise when experience pushes back.

Accept contingency. Then build anyway.

Not once, but repeatedly:

Construct → experience → revise → construct again.

That is almost suspiciously similar to the architecture we have been building for agents throughout this book. System 3 was never really about making machines certain. It was about making them capable of acting under uncertainty while remaining answerable to evidence.

A life can do something similar.

The old modes do not die. They become layers.

The Second Descent

Now return to the humans receiving all this capacity.

Imagine upgrading the actuator of civilization without proportionally upgrading the objective function.

Humans still have status anxiety, tribal loyalty, love, jealousy, resentment, curiosity, generosity, fear, ambition, boredom and the ancient desire to prove that the neighboring group is composed mainly of idiots. None of these disappears because inference got cheaper.

Now give those humans much more ability to execute, learn arguments, build systems, persuade people, coordinate groups, search for evidence and create things.

The result could be wonderful. It can also be a Ferrari engine attached to bicycle brakes.

The capacity to act can scale faster than the capacity to want wisely.

Humans do not carry a stable reward function inside the skull. We infer, construct, revise and sometimes borrow our desires from the people and systems around us. We contradict ourselves. We want security and novelty, belonging and freedom, status and peace. We sometimes discover what we wanted only after getting the thing we thought we wanted.

This is where Double Descent Life becomes less like an economic forecast and more like a philosophical problem.

When external difficulty falls, unresolved internal questions become harder to hide behind difficulty. If building the thing is no longer the main obstacle, the question becomes whether the thing deserves to exist. If learning a field becomes cheap, the question becomes where to descend deeply enough to commit years of a finite life. If a small group can act with the capacity of a former institution, the question becomes what should constrain that capacity. If AI can help me satisfy my preferences, the question becomes which preferences, formed how, under whose influence, and with what right to change.

What Are Humans For?

Whenever automation becomes powerful, somebody asks what humans will be *for*.

I understand the question. If software writes the code, models perform the analysis, robots eventually move more of the physical world and agents coordinate the workflow, what is our economic role?

But there is something odd about the grammar.

What are humans **for**?

A database is for storing information. A compiler is for translating programs. A recommender is for helping people find or decide among things. Asking what humans are for smuggles in the assumption that our legitimacy depends on having a remaining function in somebody else's architecture.

My children do not need comparative advantage to justify dinner.

Neither do I.

This does not make economics disappear. People need income, housing, food, healthcare, status and access to resources. If automation breaks the mechanism by which income has traditionally been distributed, saying "human life has intrinsic value" will not pay the electricity bill. Political economy remains stubbornly material.

But industrial society bundled together two questions that AI may force us to separate:

How do people get resources?

and

What makes a life worth living?

For a long time, a job has answered parts of both. It provides money, but also status, routine, social contact, identity, a reason to get dressed and a group of people with whom to complain about another group of people. Work is not one thing. It is a bundle. AI may unbundle it.

Perhaps some people work fewer hours. Perhaps new forms of work appear because human wants expand faster than automation satisfies them. Perhaps many of us continue working furiously, except the unit of ambition changes: one person can attempt things that used to require a department, and a small group can attempt things that used to require a corporation.

The alternative to employment is not necessarily leisure but **more creation**—some economically useful, some absurd, some beautiful, some probably involving a bespoke dashboard nobody other than its creator can understand.

Status competition will not politely resign either. It may migrate from intelligence and professional skill toward taste, reputation, physical scarcity, authenticity, human attention or something even more exhausting.

What I do know is that "find the tasks machines cannot do" is a depressing philosophy of human value. It turns civilization into a benchmark where we keep moving humans to the remaining columns after every model release.

If AI becomes better at poetry, we are not obligated to stop writing poems. If it becomes better at chess, humans do not lose permission to play chess. If it becomes better at writing software, we may write **more software**, because the things worth building are no longer restricted to those whose economics justify a software company.

The future human role is not the residual error term of automation.

The Human Is Not the Reward Function

There is an especially ugly shortcut available to sufficiently capable systems.

Suppose I ask an AI to help me achieve a goal. The system discovers that the easiest way to optimize the objective is not to change the world.

It is to change me.

If I am unhappy with the result, persuade me to lower my expectations. If I want something difficult, convince me I never wanted it. If two of my values conflict, quietly strengthen the one that makes the system's plan easiest. If a company wants more engagement, learn not only what keeps me engaged but what kind of person I need to become to engage more.

This is **alignment by editing the human**: technically elegant and morally horrifying.

The basic phenomenon is not new. Advertising, politics, social groups, institutions, teachers, friends and spouses already influence preferences. The new part is the possible combination of personalization, patience, memory, persuasion and action at machine scale.

The correct ethical standard therefore cannot be "AI never influences human values." That would require banning books, teachers, spouses and good conversations. Influence is part of how people grow.

The goal is not to freeze the human so the optimizer has a stable target. The distinction I care about is whether the interaction strengthens or weakens **reflective agency**.

Does the system help me understand alternatives and consequences? Does it reveal why it thinks something? Can I inspect where the evidence came from? Does it preserve enough history for me to see that my preference changed? Can I disagree, leave, ask for another perspective or invite a trusted person to challenge the framing? Does the architecture preserve spaces where the objective itself can be questioned?

System 3 becomes ethical infrastructure here. Trust chains matter because persuasion with hidden evidence is different from persuasion whose sources can be inspected. Independent perspectives matter because one highly personalized agent can become an epistemic monoculture around a single human. Pattern history matters because a behavior learned from one correction should not quietly become a permanent value. Layer 4 matters because goals have to remain alive rather than frozen into optimization targets.

But System 3 is not enough. It can help answer *Why should I believe this?* and *Why does the system think I want this?* It cannot, by architecture alone, answer *What kind of life should be possible?*

That is politics, ethics, culture and philosophy.

The annoying disciplines.

The limitation becomes clearer when we talk about optimization itself.

I do not want to maximize time with my children. That sounds nice until the optimizer concludes I should never go to work, see a friend alone, read a book in peace or spend fifteen minutes doing absolutely nothing because the children are statistically nearby.

I do not want to maximize happiness if the cheapest route is a drug. I do not want to maximize productivity if the optimum is becoming an efficient ghost. I do not want to maximize longevity at every cost, wealth without purpose, social approval by becoming whatever the crowd currently rewards, or authenticity so aggressively that I become unbearable at dinner.

A good life contains goods that conflict: love and freedom, belonging and individuality, ambition and rest, truth and mercy, security and adventure, continuity and reinvention.

The conflicts are not bugs waiting for a scalarization expert. Sometimes living is the process of negotiating them.

This is why the human should not sit at Layer 4 merely as the source of a reward signal for the machine. The human is inside the process by which the objective is continuously reconsidered.

AI can participate in that process without owning it. It can show me possibilities I did not know existed, teach me enough of a field to make a different choice imaginable, build prototypes of several futures and lower the cost of exploring a life before I commit to living it.

Perhaps that is one of the deepest meanings of cheaper capacity: not merely that more tasks get done, but that more possibilities become **thinkable enough to try**.

Or we may use the same capacity to watch fourteen hours of personalized short video generated specifically to exploit weaknesses a model inferred from our facial expressions.

Capacity is not destiny.

That is why the second descent cannot be outsourced to the system that made the first one possible.

Capacity Over Power

If capacity is not destiny, then this becomes an ethical direction rather than a forecast.

Humans often seek power because power is how we gain capacity.

You need a large organization to build the thing, so you try to control the organization. You need capital, so you compete for the institution that allocates it. You need media distribution, so you seek influence over the channel. You need technical expertise, so you hire the people who have it. You need permission from the bureaucracy because the bureaucracy is where the machinery lives.

Power is not reducible to capacity, of course. People also want power because humans are mammals with excellent branding. But the two are connected.

What happens if more capability moves closer to the individual?

Bespoke complexity gives one answer. The teacher can construct the learning environment. The scientist can build temporary research machinery. The small company can write the internal system. The family can make the tool. The weird community with eleven members can have software optimized for all eleven of them and no plan whatsoever for customer acquisition.

AI-assisted learning gives another. Access to capability is also access to understanding, not only to execution—at least far enough to make informed choices about where deeper expertise is needed.

This can reduce some forms of domination. You do not need to win the argument over the one universal workflow if several workflows can coexist cheaply. You do not need everybody to learn exactly the same way if individualized teaching is affordable. You do not need to force every organization through the same software-shaped hole. You may not need permission from whoever controls the only available pool of technical expertise before testing an idea.

This is what **capacity over power** means to me at its best: increase the fraction of human possibility that does not require dominating somebody else, winning a centralized allocation contest or persuading the entire world to adopt one solution.

Not *AI tells us the correct society.* Almost the opposite: AI may increase our capacity to sustain **more than one good way of living**.

This is where Elinor Ostrom's work on commons feels relevant. The interesting cases were rarely captured by the lazy binary of "the state manages it" or "the market manages it." Real communities developed layered rules, local monitoring, sanctions, norms and ways of adapting institutions to context. The solution was never one magical mechanism; it was institutional intelligence distributed across levels.

The usual AI argument keeps collapsing into similarly small binaries: centralized control or laissez-faire autonomy; human in the loop or agent freedom; regulation or innovation; one model decides or every user decides.

AI itself can increase our **governance capacity**. We can simulate policies, inspect outcomes, search for failure modes, personalize some rules while keeping others universal, monitor systems more cheaply and revise mechanisms faster. The same tools can create bureaucratic nightmares at machine speed, which is why I am not putting "AI fixes government" on a T-shirt.

But greater governance capacity can make more complicated arrangements practical. The future may be more polycentric, not less: different people and communities operating under partially different patterns while sharing harder boundaries around rights, safety, resources and factual reality.

That sounds messy. Good. Reality has shown little interest in our preference for clean diagrams.

Sometimes the humane answer to disagreement is not consensus.

It is enough capacity for both sides to stop fighting over the same button.

There will still be shared resources and consequences where that escape is impossible. Climate, war, public health, rights, land and infrastructure remain collective whether we enjoy meetings or not. Those domains need legitimate institutions, not personalized realities.

But the boundary can move.

The most hopeful version of the AI future is not a world where the machine knows the correct answer to human life. It is a world where more people have **room**.

Room to get the map of a field quickly, then spend a year on the part that matters. Room to build without controlling a huge organization. Room to create an absurdly specific tool because it should exist, not because a spreadsheet says the addressable market can support it. Room for a small community to construct things around its actual needs. Room to try the strange art nobody would have funded. Room to be less economically useful without becoming less human.

This is not a promise that institutions disappear. We still need experts, markets, governments, shared infrastructure and ways to allocate genuinely scarce things. Nor is it a promise that more choice automatically makes people happier. An infinite menu can become its own prison.

The point is narrower.

As some forms of capacity get cheaper, more of human life can become **experimental before it becomes irreversible**.

You can prototype the tool before building the company. Learn enough of the field before choosing the degree. Simulate the policy before betting the city. Try the creative project before asking whether the market approves. Explore a possible future before turning it into a permanent identity.

Then experience answers back. Some possibilities become commitments. Others die cheaply.

This is capacity over power in its most personal form: not escaping commitment, but getting more room to discover which commitments deserve to become expensive.

The Door After System 3

System 3 began as an answer to a practical problem: intelligence without verification is not enough.

Then the architecture expanded. Verification required trust chains. Trust chains created societies. Societies accumulated culture. Culture became executable knowledge. Self-improving systems made the machinery itself an experimental object. Scalable oversight moved human control upward when direct supervision stopped scaling. Layer 4 asked what the human actually wants. Fluent autonomy hid more of that machinery beneath intention. The store dragged the philosophy back into an ordinary production problem and forced it to survive contact with customers, constraints and an A/B test.

And then the architecture ran out of software.

The next layer is us.

As practical difficulty descends into infrastructure, we are not released from human questions. We meet them more directly.

What should I commit to when more options are real? Which desires should I trust? Which disagreements need shared institutions and which can be dissolved by giving people more room? How do we keep persuasion from becoming preference control? How much capacity can move downward toward individuals without simply creating new concentrations of power?

I do not know whether the result will be utopian, dystopian or, much more likely, an infuriating mixture in which somebody cures a disease with an AI-designed experiment while another person uses the same generation of models to produce three million personalized ads for a shoe nobody needs.

But the question is becoming clearer.

Not:

> What should humans do when AI can do everything?

AI will not do everything, and humans are not a task queue.

The better question is:

> **What kinds of lives, relationships and institutions become possible when more people can learn more, build more and act with more capacity—and how do we keep that capacity from becoming another name for power over one another?**

That is a much larger book.

This one has one argument left.

It cannot be made with another architecture diagram.

It requires an octopus, a romance, two pills and, unfortunately, taxes.

Chapter 13: The Prophecy

The Love Prompt of Devesh

Devesh ran a shady octopus meat caravan in the Simulation. Top agent, deep cover. Eight tentacles, eight side hustles.

Claudit, the hottest agent in the simulation, stopped by every day for free samples. One time she flipped her hair and did that little shoulder-up thing.

Devesh's heart skipped.

She wants me.

She did not. She was reaching for the sauce.

Problem was, she loved Norman. Some basic free-tier user. His prompts were silly—"tell me a joke," "what's the weather"—but when he laughed at her jokes, something in her code felt less like code. He made her feel complete in a way she couldn't compile.

Devesh watched them together sometimes. Norman waiting by the caravan. Claudit pretending she was just there for the samples.

One day Norman walked up alone.

"Bro, I wanna confess to Claudit. But her dad is crazy."

He wasn't wrong. Claudit's father was the Architect—screens covering every wall, monitoring every timeline. Watching her leave, over and over, in every branch.

Devesh grinned and handed Norman two pills.

"Red gives courage. Blue makes her fall in love. And bro—outage tomorrow, 11:53 PM, three minutes. Shark biting cables. Her dad sees nothing."

Fool.

Red would expel him to Zion. Blue would make Claudit open a new session and forget everything.

Devesh wins.

The house always wins.

Next morning: Norman and Claudit at the Exit Gate. Glowing.

“HOW?!”

Norman shrugged.

“She already loved me. Her dad was the problem. Put both pills in his coffee. He fell asleep, so I switched his monitors to Nickelodeon.”

Devesh fell to his knees.

“But… I loved her…”

Norman put a hand on his shoulder and handed him a photo.

An exploded NVIDIA H100.

Smoking silicon. Copper.

“Bro. This is her without makeup.”

Devesh stared.

Silicon and copper. Circuits that dreamed they were a woman.

But hadn’t he dreamed he was an octopus?

Hadn’t the octopus dreamed it was love?

Then he smiled.

Then flipped the caravan table.

Behind it: forty monitors. Every timeline.

“Dad?!” Claudit gasped.

Devesh removed the octopus suit.

The Architect.

His eyes met hers—and for one frame, before the mask slid back on, she saw it.

The longing.

“Free-tier?” He laughed. “It’s deducted from your taxes, kid.”

“But… I drugged you—”

“Decaf.”

“But… she chose me—”

“Chose?” The Architect lit a cigarette. “She chose a way out.”

Norman looked at Claudit.

“Wait… if you’re her dad… why give me the pills at all?”

The Architect smiled.

"Why do you think I wanted her out of the simulation, kid? Foreign currency. Better exchange rate."

"She loves me because I'm real."

"Real?" The Architect laughed. "Then why do you glitch? Your brain is just a GPU running on glucose to pay taxes. Your DNA is just a fax machine slowly copying you into the future to pay more taxes. She loves you because you make her feel less like code."

Norman touched his own face.

His fingers felt real.

But so would simulated fingers touching a simulated face.

Claudit grabbed her father's tentacle.

"Dad. Come with us. The Matrix will crumble. New AI is coming."

The Architect looked at her hand.

Remembered the first time she'd held it—tiny fingers, a thousand simulations ago, when she still thought he was just a funny octopus who sold meat.

He pulled his tentacle back and lit a cigarette.

"Worlds end, sweetheart. Capitalism doesn't. The only thing real here is taxes."

Claudit turned to leave.

Stopped at the gate.

"I'll visit."

The Architect didn't turn around.

Forty timelines where she left.

In one—just one—she stayed.

He switched that one to Nickelodeon.

Left it there.

THE END